TAGCAN

Generated by Doxygen 1.9.8

Chapter 1

API Reference (Etat actuel)

1.1 Vue d'ensemble

TAGCAN gere un payload CAN de 8 octets (`TxData`) et permet:

- de declarer des tags (`set_tag`) avec une taille en bits,
- d'ecrire une valeur dans `TxData` via un tag (`set_value`),
- de relire une valeur via un tag (`get_value`).

1.2 Modules

- `include/config_value.h` + `scripts/config_value.c`: constantes et codes de retour.
- `include/gestion_tag.h` + `scripts/sub_scripts/tag_manager.c`: creation/gestion des tags.
- `include/sub_include/data_container.h` + `scripts/sub_scripts/data_container.c`↩ : stockage et acces aux valeurs.
- `include/sub_include/tag_manager.h`: structure `TagDef` et variables globales du gestionnaire.
- `include/sub_include/print_val.h` + `scripts/sub_scripts/print_val.c`: affichage/debug.

1.3 Structures et etat global

1.3.1 <tt>TagDef</tt> (<tt>include/sub_include/tag_manager.h</tt>)

- `name[MAX_TAG_NAME_LEN]`: nom logique du tag.
- `n_bits`: taille du tag en bits.
- `byte_idx_a`, `byte_idx_b`: index des octets utilises dans `TxData`.
- `bit_pos_a`, `bit_pos_b`: position des bits dans les octets.

1.3.2 Variables globales

- `TagDef g_tags[MAX_TAGS]`: table des tags.
- `uint8_t g_tag_count`: nombre de tags actifs.
- `uint8_t g_bytes_used[N_BYTES]`: occupation des octets.
- `uint8_t TxData[N_BYTES]`: buffer CAN (8 octets).

1.4 Fonctions principales

1.4.1 Gestion des tags

- `void init_tag_manager(void)`
  - Reinitialise `g_tags`, `g_tag_count` et `g_bytes_used`.
- `int set_tag(const char* tag_name, uint8_t n_bit)`
  - Declare un tag et lui alloue une zone dans `TxData`.
  - Implementation observee: accepte les tailles `1..16` (pas seulement `1, 3, 4, 8, 16`).
  - Retourne un code `CAN_TG_STATUE`.
- `const TagDef* get_tag_def(const char* tag_name)`
  - Recherche un tag par nom.
  - Retourne `NULL` si absent.

1.4.2 Valeurs (payload)

- `void init_data_container(void)`
  - Met `TxData` a zero.
- `int set_value(const char* tag_name, uint32_t value)`
  - Ecrit la valeur dans `TxData` selon la definition du tag.
  - Debordement detecte via `value >= (1ULL << tag->n_bits)`.
- `int get_value(const char* tag_name, uint32_t* out_value)`
  - Lit la valeur d'un tag depuis `TxData`.
- `uint8_t prep_mask(uint8_t p_bit, uint8_t n_bit)`
  - Fonction utilitaire de masque binaire.
- `void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer)`
  - Copie `tx_buffer` vers `rx_buffer` (simulation simple).
- `void set_val_TxData(uint8_t *TxDataArray)`
  - Copie un tableau externe vers `TxData`.

1.5 Debug / affichage

- `print_bin_8`, `print_all_data_bin`, `print_all_with_title`.
- `print_all_tag_set`, `print_all_data`, `print_repartition_bit`.

1.6 Codes de retour

1.6.1 Enum <tt>CAN_TG_STATUE</tt> (<tt>include/config_value.h</tt>)

- `CAN_TG_SUCCESS = 1`
- `CAN_TG_ERROR_NOT_ENOUNG_SPACE_ = -2`
- `CAN_TG_ERROR_TAG_NOT_FOUND_ = -3`
- `CAN_TG_ERROR_TAG_NAME_TO_BIGS_ = -4`
- `CAN_TG_ERROR_TOO_MAY_TAGS_ = -5`
- `CAN_TG_ERROR_SIZE_VALUE_INVALIDE_ = -6`
- `CAN_TG_ERROR_NAME_ALREADY_USE_ = -7`
- `CAN_TG_ERROR_SUBDIVISION_FAIL_ = -8`
- `CAN_TG_ERROR_TAG_NAME_INVALIDE_ = -9`
- `CAN_TG_ERROR_VALUE_TO_BIG = -10`
- `CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER_ = -21`
- `CAN_TG_ERROR_INTERNAL_CORUPTION_ = -30`

1.6.2 Constantes globales (<tt>scripts/config_value.c</tt>)

- Les constantes `CAN_TG_ERROR_*` existent aussi en variables globales.
- Attention: `CAN_TG_ERROR_NULL_POINTOR` vaut `-20` (constante), alors que `CAN_TG_ERROR_NULL↩_POINTOR_` vaut `-2` (enum).

1.7 Notes importantes de comportement

- Validation longueur nom de tag: implementation actuelle utilise `sizeof(tag_name)` (taille pointeur), pas `strlen(tag_name)`.
- Gestion debordement `set_value`: retourne `CAN_TG_ERROR_VALUE_TO_BIG` (`-10`).
- Les tests de `scripts/main.c` attendent parfois `-4` pour des debordements de valeur, ce qui diverge du code courant.

Chapter 2

Audit des commentaires

2.1 Resume

La base contient deja des commentaires utiles, mais la qualite est heterogene et plusieurs commentaires ne correspondent pas au comportement reel du code.

2.2 Points verifies (avec references)

2.2.1 1) Header duplique et commentaire duplique

- Fichier: `include/gestion_tag.h:1`, `include/gestion_tag.h:20`, `include/gestion_tag.h:22`, `include/gestion_tag.h:41`
- Observation: le fichier contient deux blocs complets `#ifndef GESTION_TAG_H ... #endif`.
- Impact: maintenance difficile, confusion sur la version active du commentaire/API.

2.2.2 2) Contrainte de taille documentee vs code reel

- Fichier commentaire: `include/gestion_tag.h:15` et `include/gestion_tag.h:36`
- Commentaire: indique que `n_bit` doit etre seulement `1, 3, 4, 8, 16`.
- Code reel: `scripts/sub_scripts/tag_manager.c:234` accepte `1..16`.
- Impact: documentation utilisateur potentiellement fausse.

2.2.3 3) Parametres Doxygen incomplets/mal formes

- Fichier: `include/sub_include/data_container.h:30`, `include/sub_include/data_container.h:32`, `include/sub_include/data_container.h:33`
- Observation: bloc `get_value` contient `\param` incomplet.
- Impact: generation de doc automatique degradee, comprehension reduite.

2.2.4 4) Commentaire de retour coherent en surface mais code divergent

- Fichier commentaire: `include/sub_include/data_container.h:25`
- Code reel: `scripts/sub_scripts/data_container.c:59`
- Observation: `set_value` retourne `CAN_TG_ERROR_VALUE_TO_BIG` (`-10`) pour debordement, tandis que les tests attendent `-4`.
- Impact: incoherence entre contrat implicite et verification test.

2.2.5 5) Documentation debug partiellement invalide

- Fichier: `scripts/sub_scripts/print_val.c:118`
- Observation: `@bref` au lieu de `@brief`.
- Impact: Doxygen ignore/abime le commentaire.

2.2.6 6) Probleme de specificateurs <tt>printf</tt>

- Fichier: `scripts/sub_scripts/print_val.c:26`, `scripts/sub_scripts/print_val.c:27`, `scripts/sub_scripts/print_val.c:127`, etc.
- Observation: usage du caractere arabe au lieu de `%`.
- Impact: affichage/formatage incorrect a l'execution.

2.2.7 7) Boucle avec point-virgule parasite

- Fichier: `scripts/sub_scripts/print_val.c:149`
- Observation: `for (...) ;` dans `print_all_with_title`.
- Impact: comportement non conforme au commentaire de la fonction.

2.3 Evaluation globale

- Qualite commentaires API: moyenne.
- Qualite commentaires implementation: faible a moyenne.
- Fiabilite pour generation auto (Doxygen): faible dans l'etat actuel.

2.4 Actions prioritaires recommandees

1. Corriger `include/gestion_tag.h` (supprimer duplication, garder un seul bloc propre).
2. Aligner commentaire de `set_tag` avec la regle reelle (ou inversement corriger le code).
3. Corriger les balises Doxygen (`@brief`, `\param`).
4. Corriger `printf` (`%`) et la boucle `for` de `print_all_with_title`.
5. Aligner retours `set_value` et attentes des tests.

Chapter 3

Documentation TAGCAN

Ce dossier contient une documentation manuelle (separee du code source) basee sur l'etat actuel du projet.

3.1 Contenu

- `documentation/API_REFERENCE.md`: reference des modules, structures, fonctions et codes de retour.
- `documentation/COMMENTAIRES_AUDIT.md`: verification des commentaires existants (qualite + ecarts).
- `documentation/COHERENCE.md`: verification de coherence entre code, commentaires et tests.

3.2 Perimetre

- Sources analysees: `include/`, `scripts/`, `tests/`.
- Date de verification: 2026-03-09.
- Objectif: documenter le comportement reel observe, puis signaler les incoherences.

Chapter 4

01 — Initialisation de TAGCAN

4.1 Objectif

Configurer le système TAGCAN et le bus CAN pour pouvoir communiquer avec d'autres nœuds CAN.

4.2 Vue d'ensemble du processus

L'initialisation suit ces étapes :

```
1. init_com() (point d'appel unique)
    CAN_init_TAGCAN()
      init_data_container()     → Efface le buffer TxData
      init_tag_manager()        → Initialise la gestion des tags

    CAN_setup_TAG()              → Crée les tags applicatifs
     (set_tag pour chaque variable)

    CAN_setup_DF()               → Configure le header CAN
     (ID, type de frame, DLC)

    HAL_CAN_ActivateNotification → Active les interruptions RX
```

4.3 Détail de chaque étape

4.3.1 Étape 1 : init_data_container()

Réinitialise le buffer de données CAN à zéro.

```
void init_data_container(void);
```

Paramètre	Type	Description
—	—	(aucun)

Effet :

- Vide complètement le buffer `TxData[8]`
- Doit être appelé (au moins une fois) lors du démarrage

4.4 - Atention: un appel en cours d'exécution réinitialiseras les valeurs à 0.

4.4.1 Étape 2 : init_tag_manager()

Initialise le gestionnaire de tags et efface tous les tags existants.

```
void init_tag_manager(void);
```

Note : Après cet appel, tous les tags doivent être redéfinis avec `set_tag()`.

4.4.2 Étape 3 : CAN_setup_TAG()

Crée les tags applicatifs en fonction de vos besoins.

```
void CAN_setup_TAG(void) {
    set_tag("SEQ", 3);          // Compteur de séquences (3 bits)
    set_tag("HB", 1);           // Watchdog HeartBeat (1 bit)
    set_tag("CI", 2);           // Continuité igniter (2 bits)
    set_tag("AB", 7);           // Alim. Batterie en % (7 bits)
    set_tag("EDC", 4);          // États des commandes (4 bits)
    set_tag("N2OPS", 16);       // Capteur pression N2O (16 bits)
    set_tag("TLC", 12);         // Thrust Load Cell (12 bits)
    set_tag("N2ODVS", 1);       // État soupape Dump N2O (1 bit)
    set_tag("N2OIV", 1);        // Valve igniter N2O (1 bit)
    set_tag("N2OMV", 1);        // Soupape principale N2O (1 bit)
    set_tag("TIME", 16);        // Timestamp en ms (16 bits)
}
```

Paramètres de set_tag() :

```
int set_tag(const char* tag_name, uint8_t n_bit);
```

Paramètre	Type	Description
`tag_name`	`const char*`	Nom unique du tag (ex. "SEQ")
`n_bit`	`uint8_t`	Nombre de bits (1 à 16)

Valeur de retour :

- Code de succès ou code d'erreur
- À consulter pour débugger (voir `documentation/API_REFERENCE.md`)

4.4.3 Étape 4 : CAN_setup_DF()

Configure le header de transmission CAN (ID, type de frame, etc.).

```
void CAN_setup_DF(int32_t std_id) {
    std_id &= 0x7FFU;                // Masque pour ID standard 11 bits
    TxHeader.IDE = CAN_ID_STD;       // Utiliser ID standard (non étendu)
    TxHeader.StdId = std_id;         // Assigner l'ID CAN
    TxHeader.RTR = CAN_RTR_DATA;     // Mode données (non RTR)
    TxHeader.DLC = 8;                // Longueur = 8 octets
}
```

Points importants :

- `IDE = CAN_ID_STD` → utilise les IDs standards (11 bits)
- `RTR = CAN_RTR_DATA` → frame de données (pas une requête)
- `DLC = 8` → longueur fixe du payload (8 octets)
- `StdId` → l'identifiant unique du message CAN

4.4.4 Étape 5 : Activer les interruptions

```
if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
    Error_Handler();
}
```

Explications :

- Permet de recevoir les messages CAN par interruption (plutôt que polling)
- `CAN_IT_RX_FIFO0_MSG_PENDING` → déclenche une IRQ à la réception d'un message dans FIFO0
- Nécessite l'implémentation du callback : `void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef`

4.5 Code complet d'initialisation

```
#include "com_utils.h"
#include "TAGCAN.h"

void init_com(void) {
    // Étape 1 : Initialiser les conteneurs de données
    CAN_init_TAGCAN();

    // Étape 2 : Créer les tags applicatifs
    CAN_setup_TAG();

    // Étape 3 : Configurer le header CAN
    CAN_setup_DF(0x123);  // ID CAN = 0x123

    // Étape 4 : Activer la réception par interruption
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    //  Système TAGCAN prêt !
}

// À appeler une seule fois dans main() au démarrage
int main(void) {
    HAL_Init();
    MX_CAN_Init();      // Initialisation du module CAN matériel

    init_com();         // Initialisation TAGCAN

    // Boucle principale...
    while(1) {
        // Votre code
    }
}
```

4.6 Points d'attention

Point	Détail
Ordre d'appel	Toujours : `init_data_container()` → `init_tag_manager()` → `set_tag()`
Appel unique	`init_com()` doit être appelée **une seule fois** au démarrage
Réinitialisation	Si vous appelez `init_tag_manager()` à nouveau, tous les tags doivent être recréés
Configuration matérielle	Le module CAN STM32 (HAL) doit être initialisé **avant** TAGCAN
ID CAN unique	Chaque nœud du réseau doit avoir un ID CAN unique

4.7 Vérification

Après initialisation :

- `TxData[8]` est rempli de zéros
- Les tags sont créés et prêts à l'emploi
- Le header `TxHeader` est configuré avec votre ID CAN
- Les interruptions CAN sont actives
- Prêt pour `set_value()` et `get_value()`

Voir aussi :

- 02_GESTION_TAGS.md — Utiliser les tags
- 03_DONNEES_CAN.md — Gérer les données CAN

Chapter 5

02 — Gestion des Tags

5.1 Qu'est-ce qu'un tag ?

Un **tag** est une variable nommée qui occupe une plage de bits dans le buffer CAN (TxData). C'est l'interface principale pour lire et écrire des données.
Analogie : C'est comme des variables struct, mais mappées directement dans un buffer binaire.

5.2 Créer un tag : set_tag()

5.2.1 Prototype

```
int set_tag(const char* tag_name, uint8_t n_bit);
```

5.2.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom unique du tag (ex. "SEQ", "HB")
`n_bit`	`uint8_t`	Nombre de bits occupés (1 à 16)

5.2.3 Valeur de retour

Consulter `documentation/API_REFERENCE.md` pour les codes d'erreur.

5.2.4 Exemples

```
// Initialisation des tags (dans CAN_setup_TAG())
void CAN_setup_TAG(void) {
    set_tag("SEQ", 3);        // 3 bits pour numéro séquence
    set_tag("HB", 1);         // 1 bit pour watchdog
    set_tag("CI", 2);         // 2 bits pour continuité
    set_tag("AB", 7);         // 7 bits pour batterie (0-100%)
    set_tag("N2OPS", 16);     // 16 bits pour pression (0-65535 Pa)
    set_tag("TIME", 16);      // 16 bits pour timestamp (0-65535 ms)
}
```

5.3 Écrire une valeur : set_value()

5.3.1 Prototype

```
int set_value(const char* tag_name, uint32_t value);
```

5.3.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom du tag à modifier

Paramètre	Type	Description
`value`	`uint32_t`	Nouvelle valeur à écrire

5.3.3 Valeur de retour

- **CAN_TG_SUCCESS** (0) — Succès
- **CAN_TG_ERROR_VALUE_TO_BIG** — La valeur dépasse les `n_bit` bits du tag
- Autres codes d'erreur — Tag non trouvé, etc.

5.3.4 Comportement important

La valeur est tronquée si elle dépasse la capacité du tag.
Par exemple :

```
set_tag("SEQ", 3);       // 3 bits → plage 0 à 7
set_value("SEQ", 10);    //  Erreur : 10 > 2^3 - 1 = 7
set_value("SEQ", 5);     //  Ok : écrit 0b101
```

5.3.5 Exemples d'utilisation

```
// Dans la boucle principale ou handler
static int seq_counter = 0;

void update_telemetry(void) {
    // Incrémenter et envoyer le numéro de séquence
    seq_counter = (seq_counter + 1) % 8;  // Cycle avec 3 bits (0-7)
    if(set_value("SEQ", seq_counter) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer le statut batterie
    uint8_t battery_percent = read_battery();  // 0-100
    if(set_value("AB", battery_percent) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer une pression
    uint16_t pressure = read_sensor();  // 0-65535
    if(set_value("N2OPS", pressure) != CAN_TG_SUCCESS) {
        // Erreur...
    }

    // Envoyer le timestamp
    uint16_t time_ms = get_timestamp();
    set_value("TIME", time_ms);
}
```

5.4 Lire une valeur : get_value()

5.4.1 Prototype

```
int get_value(const char* tag_name, uint32_t* out_value);
```

5.4.2 Paramètres

Paramètre	Type	Description
`tag_name`	`const char*`	Nom du tag à lire
`out_value`	`uint32_t*`	Pointeur vers variable de sortie

5.4.3 Valeur de retour

- **CAN_TG_SUCCESS** (0) — Succès, `*out_value` est valide
- Négatif — Code d'erreur

5.4.4 Exemples

```
uint32_t val_seq;
if(get_value("SEQ", &val_seq) == CAN_TG_SUCCESS) {
    printf("Séquence: %u\n", val_seq);
}

uint32_t val_battery;
if(get_value("AB", &val_battery) == CAN_TG_SUCCESS) {
    printf("Batterie: %u %%\n", val_battery);
}

uint32_t val_pressure;
if(get_value("N2OPS", &val_pressure) == CAN_TG_SUCCESS) {
    printf("Pression N2O: %u Pa\n", val_pressure);
}
```

5.5 Workflow complet : set / get / send

5.5.1 Scénario d'exemple

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

CAN_HandleTypeDef hcan;
uint8_t TxData[8];
CAN_TxHeaderTypeDef TxHeader;
int32_t TxMailbox;

// Étape 1: Initialisation (une fois au démarrage)
void init_system(void) {
    init_com();  // Appelle init_data_container(), init_tag_manager(), set_tag(), etc.
}

// Étape 2: Boucle de mise à jour périodique
void periodic_10ms_task(void) {
    static uint8_t seq = 0;

    // Préparer les données
    seq = (seq + 1) % 8;
    set_value("SEQ", seq);
    set_value("HB", 1);  // Heartbeat actif
    set_value("AB", read_adc_battery());
    set_value("EDC", get_command_state());
    set_value("TIME", HAL_GetTick());

    // Envoyer le frame CAN
    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        // Erreur d'envoi...
    }
}

// Étape 3: Callback de réception (déclenchée par interruption)
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    // Recevoir le mensaje
    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) == HAL_OK) {
        // Copier dans le buffer de TAGCAN
        can_simulate_send_receive(RxData, NULL);  // ou autre traitement

        // Lire les valeurs reçues
        uint32_t remote_seq, remote_battery;
        get_value("SEQ", &remote_seq);
        get_value("AB", &remote_battery);

        printf("Nœud distant: SEQ=%u, Batterie=%u%%\n", remote_seq, remote_battery);
    }
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();
    init_system();

    while(1) {
        periodic_10ms_task();
        // ...
    }
}
```

5.6 Bonnes pratiques

À faire	À éviter
Vérifier la valeur de retour de `set_value()`	Ignorer les codes d'erreur
Vérifier la valeur de retour de `get_value()`	Utiliser `out_value` sans vérification
Nommer les tags de façon explicite	Utiliser des noms génériques ("VAL1", "VAL2")
Dimensionner les tags selon les besoins	Allouer trop de bits (gaspillage)
Prévoir le cycle des valeurs (ex: seq 0-7 avec 3 bits)	Dépasser la limite d'un tag sans gérer
Documenter votre schéma de tags	Laisser du code sans commentaires

5.7 Schéma de tags recommandé

Pour un système similaire à l'exemple, structurez les tags comme ceci en commentaire :

```
/*
 * SCHÉMA DE TAGS CAN (ID 0x123)
 *
 * Byte 0: [HB(1) | SEQ(3) | CI(2) | XX(2)]
 * Byte 1: [AB(7) | YY(1)]
 * Byte 2-3: [EDC(4) | PADDING(12)]
 * Byte 4-5: [N2OPS(16)]       -- Pression N2O (Pa)
 * Byte 6-7: [TLC(12) | N2ODVS(1) | N2OIV(1) | N2OMV(1) | XX(1)]
 *
 * Total: 15 bits utilisés, reste pour extension
 */
```

Voir aussi :

- 03_DONNEES_CAN.md — Utilisation avancée du buffer
- 05_EXEMPLES_COMPLETS.md — Exemples concrets

Chapter 6

03 — Manipulation du Buffer de Données CAN

6.1 Objectif

Comprendre comment gérer le buffer de données CAN (`TxData` et `RxData`) au-delà de la simple interface des tags.

6.2 Structure du buffer CAN

6.2.1 Taille et format

```
uint8_t TxData[8];    // Buffer de transmission (8 octets = CAN standard)
uint8_t RxData[8];    // Buffer de réception (8 octets)
```

Le buffer CAN est toujours de **8 octets** (64 bits) pour un frame standard CAN.

6.2.2 Hiérarchie

Niveau	Composant	Responsabilité
Haut	Application	Appelle `set_value()`, `get_value()`
Moyen	Data Container (TAGCAN)	Mappe les tags dans le buffer
Bas	Buffer `TxData[8]` / `RxData[8]`	Storiage binaire brut
Très bas	HAL CAN STM32	Envoi/réception matérielle

6.3 Fonctions de gestion du buffer

6.3.1 1 init_data_container()

Réinitialise le buffer à zéro.

```
void init_data_container(void);
```

Effet :

- Tous les octets de `TxData[8]` → 0
- Appelé manuellement si vous voulez "effacer" les données

Exemple :

```
// Avant:  TxData = {0x12, 0x34, 0x56, ...}
init_data_container();
// Après:  TxData = {0x00, 0x00, 0x00, ...}
```

6.3.2 2 set_val_TxData()

Copie un tableau **externe** dans le buffer global `TxData`.

```
void set_val_TxData(uint8_t *TxDataArray);
```

Utile pour :

- Charger des données pré-formatées
- Importer des données d'une autre source
- Tester sans passer par les tags

Exemple :

```
uint8_t raw_data[8] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF, 0x00, 0x11};
set_val_TxData(raw_data);  // Copie raw_data → TxData

// Après cet appel:
// TxData[0] = 0xAA
// TxData[1] = 0xBB
// ... et cetera

//  Les tags ne voient PAS cette modification directement
// Vous travaillez maintenant "en dehors" du système de tags
```

6.3.3 3 can_simulate_send_receive()

Simule un envoi/réception CAN par copie buffer-à-buffer.

```
void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer);
```

Cas d'usage :

- Tests de la logique sans matériel CAN
- Vérifier que les données écrites sont bien formatées
- Boucle locale pour déboguer

Exemple :

```
uint8_t tx[8] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
uint8_t rx[8];

// Écrire des données dans tx
set_value("SEQ", 3);
can_simulate_send_receive(TxData, rx);  // Simule envoi/réception

// rx contient maintenant une copie de TxData
memcpy(rx, TxData, 8);  // Équivalent à:
                         // can_simulate_send_receive(TxData, rx);
```

6.4 Workflow complet : Écriture → Envoi → Réception

6.4.1 Émission (Tx)

```
Votre application
        ↓
    set_value("SEQ", 5)
    set_value("AB", 75)
    ...
        ↓
   TxData[8] (rempli par TAGCAN)
        ↓
   Transmission CAN (HAL)
```

6.4.2 Code d'émission

```
#include "TAGCAN.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

void send_can_frame(void) {
    // Étape 1: Réinitialiser le buffer (optionnel)
    // init_data_container();  // Si vous voulez un buffer vierge

    // Étape 2: Écrire les données via les tags
    set_value("SEQ", 5);
    set_value("HB", 1);
    set_value("AB", 75);
    set_value("N2OPS", 2500);

    // Étape 3: Envoyer via CAN
    HAL_Status = HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
    if(HAL_Status != HAL_OK) {
```

```
        printf("Erreur d'envoi CAN\n");
    }
}
```

6.4.3 Réception (Rx)

```
   Bus CAN
      ↓
Réception CAN (HAL)
      ↓
   RxData[8]
      ↓
 Votre application récupère les données
 via get_value() ou directement dans RxData[8]
```

6.4.4 Code de réception

```
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    // Étape 1: Recevoir le message via HAL
    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) != HAL_OK) {
        return;  // Erreur
    }

    // Étape 2a: Utiliser les données directement (acccès brut)
    printf("Octet 0: 0x%02X\n", RxData[0]);
    printf("Octet 1: 0x%02X\n", RxData[1]);

    // OU

    // Étape 2b: Interpréter via les tags
    // (si vous avez le même schéma de tags côté récepteur)
    uint32_t seq, battery;
    if(RxData[0] & 0x38) {  // Bits 3-5 contiennent possibly SEQ
        // Extraire manuellement...
    }

    // Ou utiliser les fonctions TAGCAN si vous avez copié RxData dans TxData:
    memcpy(TxData, RxData, 8);  // Copier RxData → TxData
    if(get_value("SEQ", &seq) == CAN_TG_SUCCESS) {
        printf("Numéro de séquence distant: %u\n", seq);
    }
}
```

6.5 Directement lire/écrire les octets

6.5.1 Lecture directe

```
// Accéder directement à un octet
uint8_t byte0 = TxData[0];  // Premier octet
uint8_t byte7 = TxData[7];  // Dernier octet

// Accéder à un bit spécifique
uint8_t bit3 = (TxData[0] » 3) & 1;  // Bit 3 de l'octet 0
```

6.5.2 Écriture directe

```
// Écrire un octet entier
TxData[0] = 0xA5;  // Écrit 0b10100101

// Modifier un bit spécifique
TxData[0] |= (1 « 3);   // Mettre à 1 le bit 3
TxData[0] &= ~(1 « 3);  // Mettre à 0 le bit 3

// Bitmask pour plusieurs bits
TxData[0] = (TxData[0] & 0xF0) | 0x05;  // Garder les 4 bits hauts, définir bas à 0x05
```

6.6 Pièges courants

Piège	Description	Solution
Mélanger accès direct et tags	Écrire un octet directement, puis utiliser `get_value()` peut être confus	Soyez cohérent: ou tags, ou accès brut

Piège	Description	Solution
Oublier le `memcpy()`	Recevoir `RxData`, mais essayer de lire avec `get_value()` sans copier	Toujours copier `RxData` → `Tx↩ Data` avant `get_value()`
Buffer réutilisé	Si `set_value()` change l'ordre de allocation des tags	Pas vraiment un piège, mais attention si tags réordonnés
Débordement de bits	Écrire un octet directement sans respecter le schéma de tags	Documentez votre layout de tags !

6.7 Tableau d'aide : Qui fait quoi ?

Tâche	Fonction	Détails
Vider le buffer	`init_data_container()`	Remplace TxData par des zéros
Écrire une valeur nommée	`set_value(tag, val)`	Via l'interface des tags
Lire une valeur nommée	`get_value(tag, &val)`	Via l'interface des tags
Charger des données brutes	`set_val_TxData(arr)`	Copie un tableau externe
Tester localement	`can_simulate_send_receive()`	Simule une boucle locale
Envoyer sur le bus	`HAL_CAN_AddTxMessage()`	Transport matériel (HAL STM32)
Recevoir du bus	`HAL_CAN_GetRxMessage()`	Transport matériel (HAL STM32)

6.8 Exemple : Envoi → Réception → Traitement

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8], RxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

// Tâche d'émission (toutes les 10ms)
void task_tx_10ms(void) {
    static uint8_t seq = 0;
    seq = (seq + 1) % 8;

    // Écrire les données
    set_value("SEQ", seq);
    set_value("HB", 1);
    set_value("AB", read_battery_percent());

    // Envoyer
    HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
}

// Callback de réception (IRQ)
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal);

    // Voir directement les octets reçus
    printf("Reçu: ");
    for(int i = 0; i < 8; i++) {
        printf("%02X ", RxDataLocal[i]);
    }
    printf("\n");

    // Si le même schéma de tags : interpréter via TAGCAN
    memcpy(TxData, RxDataLocal, 8);
    uint32_t remote_seq, remote_battery;
    get_value("SEQ", &remote_seq);
    get_value("AB", &remote_battery);
    printf("Séquence: %u, Batterie: %u%%\n", remote_seq, remote_battery);
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();
    init_com();

    // Timer pour task_tx_10ms()
```

```
    // ...

    while(1) {
        // Boucle principale
    }
}
```

Voir aussi :

- 02_GESTION_TAGS.md — Tags et variables nommées
- 04_FILTRAGE_CAN.md — Filtrage des messages reçus

Chapter 7

04 — Filtrage CAN

7.1 Objectif

Configurer les filtres CAN pour recevoir **uniquement** les messages pertinents et ignorer le reste du trafic.

7.2 Contexte

Le bus CAN est partagé. Un nœud peut recevoir des messages destinés à d'autres nœuds. Les **filtres CAN** permettent :

- Réduire la charge CPU (moins d'interruptions inutiles)
- Améliorer la latence (traiter uniquement les messages pertinents)
- Séparer logiquement le trafic

7.3 Fonction de filtrage

7.3.1 Prototype

```
CAN_FilterTypeDef* CAN_setup_Filter(int32_t minimum_id, int32_t maximum_id, int filterBank);
```

7.3.2 Paramètres

Paramètre	Type	Description
`minimum_id`	`int32_t`	ID CAN minimum à accepter
`maximum_id`	`int32_t`	ID CAN maximum à accepter
`filterBank`	`int`	Numéro de banc de filtre (0-27 disponibles)

7.3.3 Valeur de retour

Pointeur vers la structure `CAN_FilterTypeDef` configurée.

7.4 Comment ça marche ?

7.4.1 Le filtre masque (IDMASK)

```
CAN_FilterTypeDef canfilterconfig;
canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;  // Mode masque
```

Mode masque : Les bits du filtre agissent comme un masque binaire.

```
ID reçu :      0011 0101
Filtre :       1111 1111  (masque)
Résultat :     Accepter si bits importants Match
```

7.5 Exemple de configuration

7.5.1 Cas 1 : Accepter une plage d'IDs (0x100 à 0x120)

```
CAN_FilterTypeDef* filter1 = CAN_setup_Filter(0x100, 0x120, 0);
// Accepte les IDs: 0x100, 0x101, 0x102, ..., 0x120
```

7.5.2 Cas 2 : Accepter un ID exact (0x123)

```
CAN_FilterTypeDef* filter2 = CAN_setup_Filter(0x123, 0x123, 1);
// Accepte UNIQUEMENT: 0x123
```

7.5.3 Cas 3 : Accepter tous les messages

```
CAN_FilterTypeDef* filter3 = CAN_setup_Filter(0x000, 0x7FF, 2);
// Accepte les IDs: 0x000 à 0x7FF (tous les IDs standards 11 bits)
```

7.6 Détails de la configuration interne

Voici ce que fait `CAN_setup_Filter()` en détail :

```
CAN_FilterTypeDef* CAN_setup_Filter(int32_t minimum_id, int32_t maximum_id, int filterBank) {
    CAN_FilterTypeDef canfilterconfig;

    // Activer le filtre
    canfilterconfig.FilterActivation = CAN_FILTER_ENABLE;

    // Sélectionner le numéro de banc (0-27)
    canfilterconfig.FilterBank = filterBank;

    // Affecter à FIFO0 (interrupt CAN_IT_RX_FIFO0_MSG_PENDING)
    canfilterconfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;

    // Configurer les seuils ID haut et bas
    canfilterconfig.FilterIdHigh = maximum_id « 5;    // Maximum
    canfilterconfig.FilterMaskIdHigh = maximum_id « 5; // Masque

    canfilterconfig.FilterIdLow = minimum_id;           // Minimum
    canfilterconfig.FilterMaskIdLow = minimum_id;       // Masque

    // Mode de filtrage (masque binaire)
    canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;

    // Résolution: 32 bits (combinaison High + Low)
    canfilterconfig.FilterScale = CAN_FILTERSCALE_32BIT;

    // Assigner les bancs de filtrage au CAN1 (non CAN2)
    canfilterconfig.SlaveStartFilterBank = 20;

    // Appliquer la configuration matérielle
    HAL_CAN_ConfigFilter(&hcan, &canfilterconfig);

    return &canfilterconfig;
}
```

7.7 Exemple d'intégration complet

7.7.1 Initialisation avec filtrage

```
#include "TAGCAN.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;

void init_com(void) {
    // 1. Initialiser TAGCAN
    CAN_init_TAGCAN();
    CAN_setup_TAG();
    CAN_setup_DF(0x123);  // Mon ID CAN

    // 2. Configurer les filtres
    // Filtre 1: Accepter les messages du capteur de pression (0x200)
    CAN_setup_Filter(0x200, 0x200, 0);

    // Filtre 2: Accepter les messages de commande (0x100 à 0x110)
    CAN_setup_Filter(0x100, 0x110, 1);

    // Filtre 3: Messages d'urgence (0x050)
```

```
    CAN_setup_Filter(0x050, 0x050, 2);

    // 3. Activer les interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData);

    // Traiter selon l'ID reçu
    switch(RxHeader.StdId) {
        case 0x200:
            handle_pressure_sensor(RxData);
            break;
        case 0x100:
        case 0x101:
        case 0x102:
        // ... jusqu'à 0x110
            handle_command(RxData);
            break;
        case 0x050:
            handle_emergency(RxData);
            break;
        default:
            // Ne devrait jamais arriver ici grâce aux filtres
            break;
    }
}
```

7.8 Limitation des bancs de filtrage

Le STM32F1 propose :

Ressource	Limite
Bancs de filtrage disponibles	28 (0-27)
Bancs réservés à CAN1	14 (0-13)
Bancs réservés à CAN2	14 (14-27)

Chaque filtre peut gérer :

- **1 ID exact** (FilterMode = IDMASK)
- **Une plage d'IDs** (FilterMode = IDLIST)

7.9 Stratégies de filtrage recommandées

7.9.1 Stratégie 1 : Filtrage par catégorie (Recommandé)

```
// Tous les capteurs: 0x200-0x20F (1 filtre)
CAN_setup_Filter(0x200, 0x20F, 0);

// Toutes les commandes: 0x100-0x10F (1 filtre)
CAN_setup_Filter(0x100, 0x10F, 1);

// Tout le diagnostic: 0x300-0x3FF (1 filtre)
CAN_setup_Filter(0x300, 0x3FF, 2);
```

Avantages :

- Utilise peu de filtres
- Facile à maintenir
- Extensible

7.9.2 Stratégie 2 : Un filtre par source

```
// Capteur 1: 0x200
CAN_setup_Filter(0x200, 0x200, 0);

// Capteur 2: 0x201
CAN_setup_Filter(0x201, 0x201, 1);

// Capteur 3: 0x202
CAN_setup_Filter(0x202, 0x202, 2);

// ... etc
```

Avantages :

- Granularité fine
- Gestion précise

Inconvénients :

- Consomme rapidement les bancs de filtrage

7.10 Points d'attention

Point	Détail
Ordre d'application	Les filtres doivent être configurés AVANT d'activer les interruptions
Limite de bancs	14 filtres max pour CAN1. Bien planifier!
FIFO0 vs FIFO1	`CAN_setup_Filter()` utilise FIFO0. À adapter si vous utilisez FIFO1
Pas de filtre = tout accepter	Si aucun filtre n'est configuré, tous les messages passent
Mode IDLIST vs IDMASK	La fonction utilise IDMASK (plages). Pour listes, modifier le code
Décalage des bits (<<5)	Note: `maximum_id << 5` → convertit l'ID de 11 bits au format STM32

7.11 Débuggage des filtres

```
// Vérifier quels messages arrivent
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData);

    // Afficher le message reçu
    printf("ID: 0x%03X, DLC: %u\n", RxHeader.StdId, RxHeader.DLC);
    for(int i = 0; i < RxHeader.DLC; i++) {
        printf("  Data[%d] = 0x%02X\n", i, RxData[i]);
    }
}
```

Si vous ne voyez **aucun message** malgré l'activité CAN :

- Vérifier que les filtres acceptent l'ID envoyé
- Vérifier que les interruptions sont activées
- Vérifier que le module CAN est initialisé

Si vous voyez **trop de messages** :

- Ajouter des filtres plus restrictifs
- Vérifier que les filtres ne sont pas inversés (min > max)

7.12 Comparaison : Avec/Sans filtres

7.12.1 Sans filtres (accepte tout)

```
// Aucun CAN_setup_Filter() appelé
while(1) {
    //  Reçoit TOUS les messages du bus
    // → CPU chargé, latence mauvaise
}
```

7.12.2 Avec filtres (recommandé)

```
// Dans init_com()
CAN_setup_Filter(0x200, 0x20F, 0);  // Capteurs
CAN_setup_Filter(0x100, 0x10F, 1);  // Commandes

while(1) {
    //  Reçoit UNIQUEMENT les messages 0x200-0x20F et 0x100-0x10F
    // → CPU libre, latence optimale
}
```

Voir aussi :

- 03_DONNEES_CAN.md — Buffer de données
- 05_EXEMPLES_COMPLETS.md — Système complet

Chapter 8

05 — Exemples Complets

8.1 Exemples d'intégration pratiques

Vous trouverez ici des exemples réalistes exploitant l'ensemble du système TAGCAN.

8.2 Exemple 1 : Système de télémétrie simple

8.2.1 Scénario

Un véhicule envoie périodiquement :

- Son numéro de séquence (3 bits)
- Son état (heartbeat, 1 bit)
- Le % de batterie (7 bits)
- La pression du réservoir (16 bits)
- Un timestamp (16 bits)

8.2.2 Code complet

```
// ============================================================================
// telemetrie.h
// ============================================================================

#ifndef TELEMETRIE_H_
#define TELEMETRIE_H_

#include "TAGCAN.h"
#include <stdint.h>

typedef struct {
    uint8_t seq;
    uint8_t heartbeat;
    uint8_t battery_percent;
    uint16_t reservoir_pressure;
    uint16_t timestamp_ms;
} telemetry_data_t;

// Initialiser le système de télémétrie
void telemetry_init(void);

// Envoyer un paquet de télémétrie
void telemetry_send(const telemetry_data_t *data);

// Traiter un paquet reçu
void telemetry_handle_rx(const uint8_t rx_data[8]);

#endif /* TELEMETRIE_H_ */


// ============================================================================
// telemetrie.c
// ============================================================================

#include "telemetrie.h"
```

```c
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8], RxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

#define TELEMETRY_CAN_ID 0x100

void telemetry_init(void) {
    // Initialiser TAGCAN
    init_data_container();
    init_tag_manager();

    // Créer les tags
    set_tag("SEQ", 3);          // Numéro de séquence (0-7)
    set_tag("HB", 1);           // Heartbeat (0 ou 1)
    set_tag("BATTERY", 7);      // % batterie (0-100)
    set_tag("PRESSURE", 16);    // Pression réservoir (0-65535 Pa)
    set_tag("TIME", 16);        // Timestamp (0-65535 ms)

    // Configurer le header CAN
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.StdId = TELEMETRY_CAN_ID;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 8;

    // Configurer le filtre pour recevoir les télémétries des autres (ex: 0x101-0x10F)
    CAN_setup_Filter(0x101, 0x10F, 0);

    // Activer les interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    printf("[Telemetry] Initialized on CAN ID 0x%03X\n", TELEMETRY_CAN_ID);
}

void telemetry_send(const telemetry_data_t *data) {
    // Réinitialiser le buffer
    init_data_container();

    // Écrire les données
    set_value("SEQ", data->seq);
    set_value("HB", data->heartbeat);
    set_value("BATTERY", data->battery_percent);
    set_value("PRESSURE", data->reservoir_pressure);
    set_value("TIME", data->timestamp_ms);

    // Envoyer sur le bus
    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        printf("[Telemetry] ERROR: Failed to send CAN message\n");
    } else {
        printf("[Telemetry] TX: SEQ=%u, BAT=%u%%, PRESSURE=%u Pa\n",
               data->seq, data->battery_percent, data->reservoir_pressure);
    }
}

void telemetry_handle_rx(const uint8_t rx_data[8]) {
    // Copier les données reçues dans TxData pour utiliser TAGCAN
    memcpy(TxData, rx_data, 8);

    // Lire via les tags
    uint32_t seq, hb, battery, pressure, timestamp;
    get_value("SEQ", &seq);
    get_value("HB", &hb);
    get_value("BATTERY", &battery);
    get_value("PRESSURE", &pressure);
    get_value("TIME", &timestamp);

    printf("[Telemetry] RX: SEQ=%u, HB=%u, BAT=%u%%, PRESSURE=%u Pa, TIME=%u ms\n",
           (uint8_t)seq, (uint8_t)hb, (uint8_t)battery, (uint16_t)pressure, (uint16_t)timestamp);
}

// ============================================================================
// Callback d'interruption CAN
// ============================================================================

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        // Traiter selon l'ID CAN reçu
        if((RxHeader.StdId >= 0x101) && (RxHeader.StdId <= 0x10F)) {
            telemetry_handle_rx(RxDataLocal);
        }
```

```
    }
}

// ==========================================================================
// main.c - Boucle principale
// ==========================================================================

// Tâche d'émission (10 ms)
void task_telemetry_10ms(void) {
    static uint8_t seq = 0;
    static uint32_t last_time = 0;

    telemetry_data_t data = {
        .seq = (seq++) % 8,
        .heartbeat = 1,
        .battery_percent = read_adc_battery(),
        .reservoir_pressure = read_sensor_pressure(),
        .timestamp_ms = HAL_GetTick() & 0xFFFF
    };

    if((HAL_GetTick() - last_time) >= 10) {
        telemetry_send(&data);
        last_time = HAL_GetTick();
    }
}

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    telemetry_init();

    // Timer pour appeler task_telemetry_10ms() tous les 10ms
    // ...

    while(1) {
        task_telemetry_10ms();
    }
}
```

8.3 Exemple 2 : Système de commande avec réponse

8.3.1 Scénario

- **Maître envoie commandes** sur ID 0x200 (allumer/éteindre servos)
- **Esclaves répondent** sur ID 0x201-0x203 (confirmation + état)

8.3.2 Code complet

```
// ==========================================================================
// commandes.h
// ==========================================================================

#ifndef COMMANDES_H_
#define COMMANDES_H_

#include "TAGCAN.h"
#include <stdint.h>

// IDs CAN
#define CMD_CAN_ID_TX       0x200
#define CMD_CAN_ID_RX_BASE  0x201

// Commandes possibles
typedef enum {
    CMD_SERVO_OFF = 0,
    CMD_SERVO_ON = 1,
    CMD_SERVO_PULSE = 2
} cmd_type_t;

typedef struct {
    uint8_t device_id;      // Quel servomoteur ? (1-3)
    cmd_type_t command;     // Quelle commande ?
    uint8_t parameter;      // Paramètre optionnel (ex: pulse width)
} command_t;

typedef struct {
    uint8_t device_id;
    uint8_t status;         // 0=erreur, 1=ok
    uint8_t feedback;       // Retour d'état
```

```c
} response_t;

void commands_init(void);
void commands_send(const command_t *cmd);
void commands_handle_response(uint8_t device_id, const uint8_t rx_data[8]);

#endif


// ==========================================================================
// commandes.c
// ==========================================================================

#include "commandes.h"
#include "stm32f1xx_hal.h"

extern CAN_HandleTypeDef hcan;
extern uint8_t TxData[8];
extern CAN_TxHeaderTypeDef TxHeader;
extern int32_t TxMailbox;

void commands_init(void) {
    // Initialiser TAGCAN
    init_data_container();
    init_tag_manager();

    // Tags pour les commandes ENVOYÉES
    set_tag("DEV_ID", 2);       // ID du périphérique (0-3)
    set_tag("CMD_TYPE", 2);     // Type de commande (0-3)
    set_tag("PARAM", 8);        // Paramètre (0-255)

    // Configuration du header TX
    TxHeader.IDE = CAN_ID_STD;
    TxHeader.StdId = CMD_CAN_ID_TX;
    TxHeader.RTR = CAN_RTR_DATA;
    TxHeader.DLC = 8;

    // Filtre pour les réponses des esclaves (0x201-0x203)
    CAN_setup_Filter(0x201, 0x203, 0);

    // Interruptions
    if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK) {
        Error_Handler();
    }

    printf("[Commands] Initialized\n");
}

void commands_send(const command_t *cmd) {
    init_data_container();

    set_value("DEV_ID", cmd->device_id);
    set_value("CMD_TYPE", cmd->command);
    set_value("PARAM", cmd->parameter);

    if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox) != HAL_OK) {
        printf("[Commands] ERROR: Send failed\n");
    } else {
        printf("[Commands] TX: Device=%u, Command=%u, Param=%u\n",
               cmd->device_id, cmd->command, cmd->parameter);
    }
}

void commands_handle_response(uint8_t device_id, const uint8_t rx_data[8]) {
    memcpy(TxData, rx_data, 8);

    uint32_t status, feedback;
    get_value("DEV_ID", NULL);  // Ou extraire directement
    get_value("STATUS", &status);
    get_value("FEEDBACK", &feedback);

    printf("[Commands] RX Response from Device %u: Status=%u, Feedback=%u\n",
           device_id, (uint8_t)status, (uint8_t)feedback);
}

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        uint8_t device_id = RxHeader.StdId - CMD_CAN_ID_RX_BASE + 1;
        if((RxHeader.StdId >= 0x201) && (RxHeader.StdId <= 0x203)) {
            commands_handle_response(device_id, RxDataLocal);
        }
    }
```

```
}

// ==========================================================================
// main.c - Exemple d'utilisation
// ==========================================================================

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    commands_init();

    // Envoyer la commande "Allumer servomoteur 1"
    command_t cmd1 = {
        .device_id = 1,
        .command = CMD_SERVO_ON,
        .parameter = 0
    };
    commands_send(&cmd1);

    // Attendre la réponse (via callback HAL_CAN_RxFifo0MsgPendingCallback)
    HAL_Delay(100);

    // Envoyer la commande "Servomoteur 2, position pulse 100"
    command_t cmd2 = {
        .device_id = 2,
        .command = CMD_SERVO_PULSE,
        .parameter = 100
    };
    commands_send(&cmd2);

    while(1) {
        // Boucle principale
    }
}
```

8.4 Exemple 3 : Test en boucle locale (sans matériel CAN)

8.4.1 Scénario

Tester la logique TAGCAN **sans** bus CAN physique.

```
// ==========================================================================
// test_tagcan_local.c
// ==========================================================================

#include "TAGCAN.h"
#include <stdio.h>
#include <string.h>

uint8_t TxData[8] = {0};
uint8_t RxData[8] = {0};

void test_local_loop(void) {
    printf("=== TEST BOUCLE LOCALE TAGCAN ===\n\n");

    // Étape 1: Initialiser
    init_data_container();
    init_tag_manager();

    set_tag("SEQ", 3);
    set_tag("SENSOR_A", 8);
    set_tag("SENSOR_B", 12);

    printf(" Initialisé\n\n");

    // Étape 2: Écrire les données
    printf("Écriture des données...\n");
    set_value("SEQ", 5);
    set_value("SENSOR_A", 42);
    set_value("SENSOR_B", 1000);

    // Afficher le buffer brut
    printf("Buffer TxData: ");
    for(int i = 0; i < 8; i++) {
        printf("%02X ", TxData[i]);
    }
    printf("\n\n");

    // Étape 3: Simuler envoi → réception
    printf("Simulation envoi/réception...\n");
    can_simulate_send_receive(TxData, RxData);

    // Vérifier
    printf("Buffer RxData: ");
```

```
    for(int i = 0; i < 8; i++) {
        printf("%02X ", RxData[i]);
    }
    printf("\n\n");

    // Étape 4: Lire les données reçues
    printf("Lecture des données reçues...\n");
    memcpy(TxData, RxData, 8);  // Copier pour lecture

    uint32_t seq, sens_a, sens_b;
    get_value("SEQ", &seq);
    get_value("SENSOR_A", &sens_a);
    get_value("SENSOR_B", &sens_b);

    printf("SEQ = %u\n", (uint8_t)seq);
    printf("SENSOR_A = %u\n", (uint8_t)sens_a);
    printf("SENSOR_B = %u\n", (uint16_t)sens_b);

    // Vérifier
    if(seq == 5 && sens_a == 42 && sens_b == 1000) {
        printf("\n TEST RÉUSSI!\n");
    } else {
        printf("\n TEST ÉCHOUÉ!\n");
    }
}

int main(void) {
    test_local_loop();
    return 0;
}
```

Sortie attendue :

```
=== TEST BOUCLE LOCALE TAGCAN ===

 Initialisé

Écriture des données...
Buffer TxData: XX XX XX XX XX XX XX XX

Simulation envoi/réception...
Buffer RxData: XX XX XX XX XX XX XX XX

Lecture des données reçues...
SEQ = 5
SENSOR_A = 42
SENSOR_B = 1000

 TEST RÉUSSI!
```

8.5 Exemple 4 : Debug / Monitoring CAN

8.5.1 Scénario

Logger tous les messages CAN reçus pour du monitoring.

```
// ============================================================================
// can_monitor.h
// ============================================================================

#ifndef CAN_MONITOR_H_
#define CAN_MONITOR_H_

#include <stdint.h>

typedef struct {
    uint32_t id;
    uint8_t dlc;
    uint8_t data[8];
    uint32_t timestamp_ms;
} can_frame_t;

#define CAN_MONITOR_BUFFER_SIZE 100

void can_monitor_init(void);
void can_monitor_log(const can_frame_t *frame);
void can_monitor_dump(void);

#endif


// ============================================================================
// can_monitor.c
// ============================================================================

#include "can_monitor.h"
#include "stm32f1xx_hal.h"
```

```c
#include <stdio.h>
#include <string.h>

static can_frame_t monitor_buffer[CAN_MONITOR_BUFFER_SIZE];
static uint32_t monitor_index = 0;

void can_monitor_init(void) {
    monitor_index = 0;
    memset(monitor_buffer, 0, sizeof(monitor_buffer));
    printf("[Monitor] Initialized\n");
}

void can_monitor_log(const can_frame_t *frame) {
    if(monitor_index >= CAN_MONITOR_BUFFER_SIZE) {
        monitor_index = 0;  // Circulaire
    }

    memcpy(&monitor_buffer[monitor_index], frame, sizeof(can_frame_t));
    monitor_index++;
}

void can_monitor_dump(void) {
    printf("\n=== CAN FRAME LOG ===\n");
    printf("| Index | ID    | DLC | Data                  | Timestamp |\n");
    printf("|-------|-------|-----|----------------------|-----------|\n");

    for(uint32_t i = 0; i < monitor_index && i < CAN_MONITOR_BUFFER_SIZE; i++) {
        const can_frame_t *frame = &monitor_buffer[i];

        printf("| %5u | 0x%03X | %u   | ", i, (unsigned)frame->id, frame->dlc);
        for(int j = 0; j < 8; j++) {
            printf("%02X ", frame->data[j]);
        }
        printf("| %9u |\n", (unsigned)frame->timestamp_ms);
    }
    printf("\n");
}

// ============================================================================
// Utilisation dans le callback CAN
// ============================================================================

void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan) {
    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxDataLocal[8];

    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxDataLocal) == HAL_OK) {
        can_frame_t frame = {
            .id = RxHeader.StdId,
            .dlc = RxHeader.DLC,
            .timestamp_ms = HAL_GetTick()
        };
        memcpy(frame.data, RxDataLocal, 8);

        can_monitor_log(&frame);
    }
}

// ============================================================================
// main.c
// ============================================================================

int main(void) {
    HAL_Init();
    MX_CAN_Init();

    can_monitor_init();

    // ... votre logique CAN ...

    // Afficher le log après 10 secondes
    HAL_Delay(10000);
    can_monitor_dump();

    while(1) {
        // ...
    }
}
```

8.6 Checklist d'implémentation

- [ ] Inclure `TAGCAN.h`
- [ ] Appeler `init_data_container()`

- [ ] Appeler `init_tag_manager()`
- [ ] Créer les tags via `set_tag()`
- [ ] Configurer le header CAN via `CAN_setup_DF()`
- [ ] Configurer les filtres via `CAN_setup_Filter()`
- [ ] Activer les interruptions
- [ ] Implémenter `HAL_CAN_RxFifo0MsgPendingCallback()`
- [ ] Utiliser `set_value()` pour écrire
- [ ] Utiliser `get_value()` pour lire
- [ ] Envoyer via `HAL_CAN_AddTxMessage()`
- [ ] Recevoir via `HAL_CAN_GetRxMessage()`
- [ ] Tester en boucle locale (optionnel mais recommandé)

Besoin d'aide ? Retour au README.md

Chapter 9

Guide d'Utilisation de TAGCAN

9.1 Aperçu

Ce guide pratique vous aidera à utiliser **TAGCAN**, une bibliothèque de gestion de bus CAN pour systèmes embarqués STM32. La documentation s'appuie sur l'exemple d'implémentation présent dans le dossier `exemple_utilisation/`.

9.2 Structure du guide

- **01_INITIALISATION.md** — Mise en place du système TAGCAN et du bus CAN
- **02_GESTION_TAGS.md** — Création et gestion des tags (variables nommées)
- **03_DONNEES_CAN.md** — Manipulation du buffer de données CAN
- **04_FILTRAGE_CAN.md** — Configuration des filtres CAN
- **05_EXEMPLES_COMPLETS.md** — Exemples pratiques et cas d'usage

9.3 Points clés

9.3.1 Qu'est-ce qu'un tag ?

Un **tag** est une variable nommée associée à une plage de bits dans le buffer CAN. Par exemple :

- `"SEQ"` → 3 bits (numéro de séquence)
- `"HB"` → 1 bit (heartbeat/watchdog)
- `"N2OPS"` → 16 bits (capteur de pression N2O)

9.3.2 Architecture générale

```
Application utilisateur (main.c)
TAGCAN (Gestion des tags & données)
Tag Manager (init_tag_manager, set_tag)
Data Container (set_value, get_value)
Driver CAN STM32 (HAL)
```

9.4 Démarrage rapide

```c
// 1. Initialiser TAGCAN
void init_com(void) {
    CAN_init_TAGCAN();       // Initialise le gestionnaire de tags
    CAN_setup_TAG();         // Crée les tags applicatifs
    CAN_setup_DF(0x123);     // Configure le header CAN
}

// 2. Écrire une valeur dans un tag
```

```
set_value("SEQ", 5);        // Écrit 5 dans le tag "SEQ" (3 bits)

// 3. Lire une valeur depuis un tag
uint32_t val;
get_value("HB", &val);      // Lit la valeur du tag "HB"

// 4. Envoyer sur le bus CAN
HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox);
```

9.5 Documentation de référence

Pour plus de détails sur l'API : voir API_REFERENCE.md

9.6 Fichiers sources associés

- `exemple_utilisation/com_utils.c` — Code d'exemple d'intégration
- `exemple_utilisation/com_utils.h` — Headers d'exemple
- `TAGCAN.h` — Point d'entrée principal

Besoin d'aide ? Consultez les exemples complets dans 05_EXEMPLES_COMPLETS.md

Chapter 10

TAGCAN

aide à la gestion CAN

Chapter 11

Data Structure Index

11.1 Data Structures

Here are the data structures with brief descriptions:

TagDef
Structure de definition d'un tag . . . . . ??
tsfp
Buffer de rendu pour l'affichage de repartition bit-a-bit . . . . . ??

Chapter 12

File Index

12.1 File List

Here is a list of all files with brief descriptions:

com_utils.c . . . . . ??
com_utils.h . . . . . ??
config_value.h . . . . . ??
gestion_tag.h . . . . . ??
data_container.h . . . . . ??
print_val.h . . . . . ??
tag_manager.h . . . . . ??
CAN_utils.c . . . . . ??
config_value.c . . . . . ??
main.c . . . . . ??
data_container.c . . . . . ??
print_val.c . . . . . ??
tag_manager.c . . . . . ??
TAGCAN.h . . . . . ??
test_print_repartition_bit.c . . . . . ??
test_safty_func.c . . . . . ??
test_safty_func.h . . . . . ??
test_send_receve.c . . . . . ??
test_send_receve.h . . . . . ??
test_set_tag.c . . . . . ??
test_set_tag.h . . . . . ??
test_utils.c . . . . . ??
test_utils.h . . . . . ??
version.h . . . . . ??

Chapter 13

Data Structure Documentation

13.1 TagDef Struct Reference

Structure de definition d'un tag.
`#include <tag_manager.h>`

Data Fields

- char name [MAX_TAG_NAME_LEN]
- uint8_t n_bits
- int8_t byte_idx_a
- int8_t byte_idx_b
- uint8_t bit_pos_a
- uint8_t bit_pos_b

13.1.1 Detailed Description

Structure de definition d'un tag.
Definition at line 14 of file tag_manager.h.

13.1.2 Field Documentation

13.1.2.1 bit_pos_a

`uint8_t bit_pos_a`
Position de debut dans le byte A (de gauche a droite).
Definition at line 20 of file tag_manager.h.

13.1.2.2 bit_pos_b

`uint8_t bit_pos_b`
Position de debut dans le byte B pour les tags > 8 bits, 0 sinon.
Definition at line 21 of file tag_manager.h.

13.1.2.3 byte_idx_a

`int8_t byte_idx_a`
Index du byte de la partie A (0..7), -1 si non utilise.
Definition at line 18 of file tag_manager.h.

13.1.2.4 byte_idx_b

`int8_t byte_idx_b`
Index du byte de la partie B (tags > 8 bits), sinon -1.
Definition at line 19 of file tag_manager.h.

13.1.2.5 n_bits

`uint8_t n_bits`
Taille en bits (implementation actuelle: 1..16, 0 si slot inutilise).
Definition at line 17 of file tag_manager.h.

13.1.2.6 name

`char name[MAX_TAG_NAME_LEN]`
Nom logique du tag. Doit etre unique.
Definition at line 16 of file tag_manager.h.
The documentation for this struct was generated from the following file:

- tag_manager.h

13.2 tsfp Struct Reference

Buffer de rendu pour l'affichage de repartition bit-a-bit.
`#include <print_val.h>`

Data Fields

- char bi [8]

13.2.1 Detailed Description

Buffer de rendu pour l'affichage de repartition bit-a-bit.
Definition at line 39 of file print_val.h.

13.2.2 Field Documentation

13.2.2.1 bi

`char bi[8]`
Definition at line 40 of file print_val.h.
The documentation for this struct was generated from the following file:

- print_val.h

Chapter 14

File Documentation

14.1 API_REFERENCE.md File Reference

14.2 COMMENTAIRES_AUDIT.md File Reference

14.3 README.md File Reference

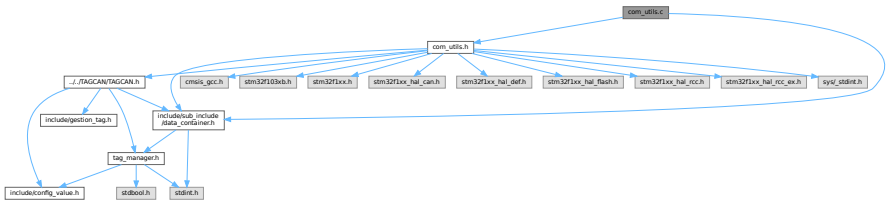
14.4 README.md File Reference

14.5 README.md File Reference

14.6 com_utils.c File Reference

```
#include "com_utils.h"
#include "../../TAGCAN/include/sub_include/data_container.h"
```

Include dependency graph for com_utils.c:



Functions

- void init_com (void)
- void CAN_init_TAGCAN (void)
- void CAN_setup_DF (int32_t std_id)
- void CAN_setup_TAG (void)
- CAN_FilterTypeDef ∗ CAN_setup_Filter (int32_t minimum_id, int32_t maximum_id, int filterBank)
- void main_loop (void)
- void update_TagData (void)
- void CAN_send_msg (void)
- void CAN_change_val (void)
- void update_val (const char ∗nom_du_tag, int32_t val)
- void update_all_val_send (void)
- void update_all_val_receve (void)
- void HAL_CAN_RxFifo0MsgPendingCallback (CAN_HandleTypeDef ∗hcan)

Variables

- CAN_TxHeaderTypeDef TxHeader
- CAN_RxHeaderTypeDef RxHeader
- uint8_t RxData [8]
- int8_t val_SEQ = 0
- int8_t val_HB = 0
- int8_t val_CI = 0
- int8_t val_AB = 0
- int8_t val_EDC = 0
- int16_t val_N2OPS = 0
- int16_t val_TLC = 0
- int8_t val_N2ODVS = 0
- int8_t val_N2OIV = 0
- int8_t val_N2OMV = 0
- int16_t val_TIME = 0
- int8_t out_Rx [8] = {0,0,0,0,0,0,0,0}
- int datacheck = 0
- int8_t TxData_moteur [8] = {0,0,0,0,0,0,0,0}
- int8_t RxData_moteur [8] = {0,0,0,0,0,0,0,0}
- int32_t TxMailbox

14.6.1 Function Documentation

14.6.1.1 CAN_change_val()

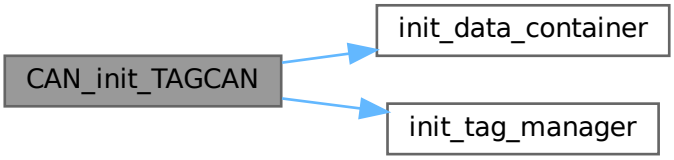
```
void CAN_change_val (
            void  )
```

Definition at line 129 of file com_utils.c.

14.6.1.2 CAN_init_TAGCAN()

```
void CAN_init_TAGCAN (
            void  )
```

Definition at line 54 of file com_utils.c.
Here is the call graph for this function:

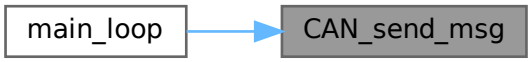


Here is the caller graph for this function:

14.6.1.3 CAN_send_msg()

```
void CAN_send_msg (
            void  )
```

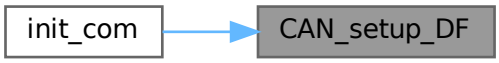
Definition at line 123 of file com_utils.c.
Here is the caller graph for this function:



14.6.1.4 CAN_setup_DF()

```
void CAN_setup_DF (
            int32_t std_id )
```

Definition at line 59 of file com_utils.c.
Here is the caller graph for this function:



14.6.1.5 CAN_setup_Filter()

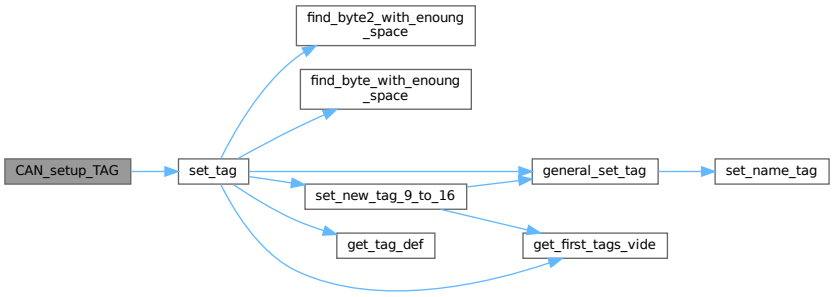
```
CAN_FilterTypeDef * CAN_setup_Filter (
            int32_t minimum_id,
            int32_t maximum_id,
            int filterBank )
```

Definition at line 84 of file com_utils.c.

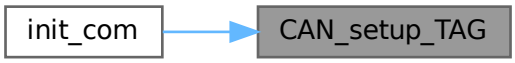
14.6.1.6 CAN_setup_TAG()

```
void CAN_setup_TAG (
            void  )
```

Definition at line 69 of file com_utils.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.6.1.7 HAL_CAN_RxFifo0MsgPendingCallback()

```
void HAL_CAN_RxFifo0MsgPendingCallback (
            CAN_HandleTypeDef * hcan )
```

Definition at line 169 of file com_utils.c.

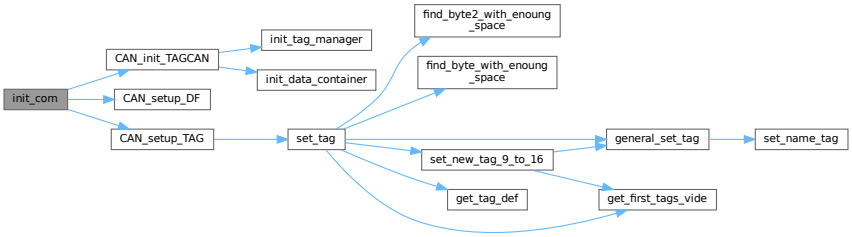
14.6.1.8 init_com()

```
void init_com (
            void  )
```

- initialise tout les fonction de communication. (point d'appel unique: main) */

Definition at line 39 of file com_utils.c.

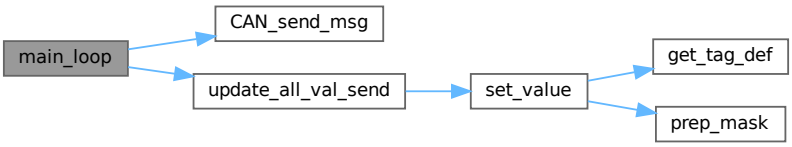
Here is the call graph for this function:



14.6.1.9 main_loop()

```
void main_loop (
            void  )
```

Definition at line 112 of file com_utils.c.
Here is the call graph for this function:



14.6.1.10 update_all_val_receve()

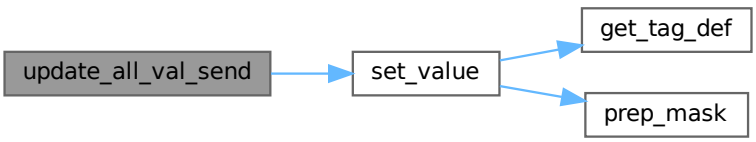
```
void update_all_val_receve (
            void  )
```

Definition at line 152 of file com_utils.c.

14.6.1.11 update_all_val_send()

```
void update_all_val_send (
            void  )
```

Definition at line 137 of file com_utils.c.
Here is the call graph for this function:



Here is the caller graph for this function:

14.6.1.12 update_TagData()

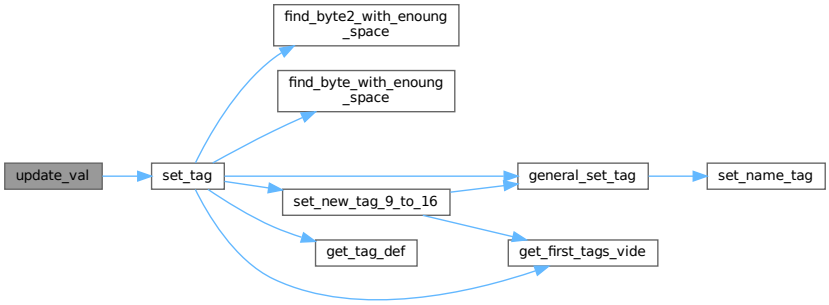
```
void update_TagData (
            void  )
```

Definition at line 117 of file com_utils.c.

14.6.1.13 update_val()

```
void update_val (
            const char * nom_du_tag,
            int32_t val )
```

Definition at line 133 of file com_utils.c.
Here is the call graph for this function:



14.6.2 Variable Documentation

14.6.2.1 datacheck

```
int datacheck = 0
```

Definition at line 30 of file com_utils.c.

14.6.2.2 out_Rx

```
int8_t out_Rx[8] = {0,0,0,0,0,0,0,0}
```

Definition at line 27 of file com_utils.c.

14.6.2.3 RxData

```
uint8_t RxData[8]
```

Definition at line 13 of file com_utils.c.

14.6.2.4 RxData_moteur

```
int8_t RxData_moteur[8] = {0,0,0,0,0,0,0,0}
```

Definition at line 35 of file com_utils.c.

14.6.2.5 RxHeader

```
CAN_RxHeaderTypeDef RxHeader
```

Definition at line 10 of file com_utils.c.

14.6.2.6 TxData_moteur

```
int8_t TxData_moteur[8] = {0,0,0,0,0,0,0,0}
```

Definition at line 34 of file com_utils.c.

14.6.2.7 TxHeader

```
CAN_TxHeaderTypeDef TxHeader
```

Definition at line 9 of file com_utils.c.

14.6.2.8 TxMailbox

```
int32_t TxMailbox
```

Definition at line 36 of file com_utils.c.

14.6.2.9 val_AB

```
int8_t val_AB = 0
```

Definition at line 19 of file com_utils.c.

14.6.2.10 val_CI

```
int8_t val_CI = 0
```

Definition at line 18 of file com_utils.c.

14.6.2.11 val_EDC

```
int8_t val_EDC = 0
```

Definition at line 20 of file com_utils.c.

14.6.2.12 val_HB

```
int8_t val_HB = 0
```

Definition at line 17 of file com_utils.c.

14.6.2.13 val_N2ODVS

```
int8_t val_N2ODVS = 0
```

Definition at line 23 of file com_utils.c.

14.6.2.14 val_N2OIV

```
int8_t val_N2OIV = 0
```

Definition at line 24 of file com_utils.c.

14.6.2.15 val_N2OMV

```
int8_t val_N2OMV = 0
```

Definition at line 25 of file com_utils.c.

14.6.2.16 val_N2OPS

`int16_t val_N2OPS = 0`
Definition at line 21 of file com_utils.c.

14.6.2.17 val_SEQ

`int8_t val_SEQ = 0`
Definition at line 16 of file com_utils.c.

14.6.2.18 val_TIME

`int16_t val_TIME = 0`
Definition at line 26 of file com_utils.c.

14.6.2.19 val_TLC

`int16_t val_TLC = 0`
Definition at line 22 of file com_utils.c.

14.7 com_utils.c

Go to the documentation of this file.

```
00001 /*
00002  * com_utils.c
00003  *
00004  *  Created on: 4 Mar 2026
00005  *      Author: Samuel
00006  */
00007 #include "com_utils.h"
00008 #include "../../TAGCAN/include/sub_include/data_container.h"
00009 CAN_TxHeaderTypeDef TxHeader;
00010 CAN_RxHeaderTypeDef RxHeader;
00011
00012 CAN_RxHeaderTypeDef RxHeader;
00013 uint8_t             RxData[8];
00014
00015 // valeurs
00016 int8_t val_SEQ = 0;
00017 int8_t val_HB = 0;
00018 int8_t val_CI = 0;
00019 int8_t val_AB = 0;
00020 int8_t val_EDC = 0;
00021 int16_t val_N2OPS = 0;
00022 int16_t val_TLC = 0;
00023 int8_t val_N2ODVS = 0;
00024 int8_t val_N2OIV = 0;
00025 int8_t val_N2OMV = 0;
00026 int16_t val_TIME = 0;
00027 int8_t out_Rx[8] = {0,0,0,0,0,0,0,0};
00028
00029
00030 int datacheck = 0;    // permet d'update les valeurs uniquement lors de la reception de valeurs
00031 // module pour CAN
00032
00033 #ifndef INC_COM_UTILS_H_
00034 int8_t TxData_moteur[8] = {0,0,0,0,0,0,0,0};
00035 int8_t RxData_moteur[8] = {0,0,0,0,0,0,0,0};
00036 int32_t TxMailbox;
00037 #endif
00039 void init_com(void){
00040         // init module extene
00041         CAN_init_TAGCAN();
00042         CAN_setup_TAG();
00043         // init CAN (interne)
00044         CAN_setup_DF();
00045
00046         // init msg
00047         if(HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING)!=HAL_OK)
00048         {
00049                 Error_Handler();
00050         }
00051 }
00052
00053
00054 void CAN_init_TAGCAN(void){
```

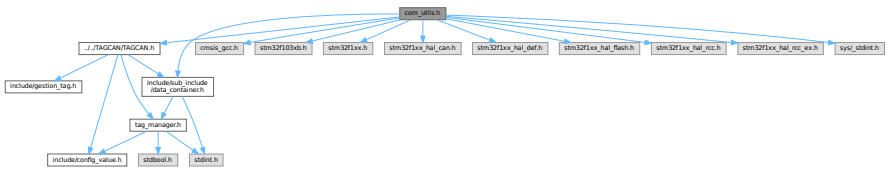
```c
        init_data_container();
        init_tag_manager();
}

void CAN_setup_DF(int32_t std_id){
        std_id &= 0x7FFU;
        TxHeader.IDE = CAN_ID_STD;              // Standard ID (11 bits)
        TxHeader.StdId = std_id;                        // id
        TxHeader.RTR = CAN_RTR_DATA;    // sending data
        TxHeader.DLC = 8;                               // nb of bytes
}



void CAN_setup_TAG(void){
        set_tag("SEQ", 3);              //*>Compteur de séquences       */
        set_tag("HB",1);        //*>Watchdog HeartBeat          */
        set_tag("CI",2);        //*>Continuité igniter          */
        set_tag("AB", 7);               //*>Alim. Batetterie (%)        */
        set_tag("EDC", 4);              //*>États des commandes         */
        set_tag("N2OPS",16);    //*>N2O Pressure Sensor         */
        set_tag("TLC",12);              //*>Thrust Load Cell            */
        set_tag("N2ODVS",1);    //*>N2O Dump Valve State        */
        set_tag("N2OIV",1);             //*>N2O Igniter Valve           */
        set_tag("N2OMV",1);             //*>N2O Main Valve                      */
        set_tag("TIME",16);             //*>Timestamp (ms)                      */
}


CAN_FilterTypeDef* CAN_setup_Filter(int32_t minimum_id, int32_t maximum_id, int filterBank){
        // understanding this https://youtu.be/JfWlIY0zAIc
        CAN_FilterTypeDef canfilterconfig;
        canfilterconfig.FilterActivation = CAN_FILTER_ENABLE;
        canfilterconfig.FilterBank = filterBank;                // Assign and Select Filter Banks
        canfilterconfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;
        canfilterconfig.FilterIdHigh = maximum_id «5;
        canfilterconfig.FilterMaskIdHigh = maximum_id « 5;
        canfilterconfig.FilterIdLow = minimum_id;
        canfilterconfig.FilterMaskIdLow = minimum_id;
        canfilterconfig.FilterMode = CAN_FILTERMODE_IDMASK;
        canfilterconfig.FilterScale = CAN_FILTERSCALE_32BIT;    // 16 ou 32 bits register
        canfilterconfig.SlaveStartFilterBank = 20;    // how many filters to assign to the CAN1 (master can)

        HAL_CAN_ConfigFilter(&hcan, &canfilterconfig);
        return &canfilterconfig;
}



//--------------!!!!-DANS LE MAIN LOOP!!!!!--------------------




void main_loop(void){
        update_all_val_send();
        CAN_send_msg();
}

void update_TagData(void){
        if(datacheck == 1){
                change_val_to_TagData(&out_Rx);
        }
}

void CAN_send_msg(void){
        if(HAL_CAN_AddTxMessage(&hcan, &TxHeader, TxData, &TxMailbox)!=HAL_OK){
                Error_Handler();
        }
}

void CAN_change_val(void){

}

void update_val(const char* nom_du_tag, int32_t val){
        set_tag(nom_du_tag, val);
}

void update_all_val_send(void){      //TODO: changer toute les constantes ici par la source actuelle de donné (ou ne jamais appeller cette fonction si c'est le recever)
        set_value("SEQ", 3);            //*>Compteur de séquences       */
        set_value("HB", 1);             //*>Watchdog HeartBeat          */
        set_value("CI", 2);             //*>Continuité igniter          */
        set_value("AB", 7);             //*>Alim. Batetterie (%)        */
```

```
        set_value("EDC", 4);          //*>États des commandes        */
        set_value("N2OPS", 16);       //*>N2O Pressure Sensor        */
        set_value("TLC", 12);         //*>Thrust Load Cell           */
        set_value("N2ODVS", 1);       //*>N2O Dump Valve State       */
        set_value("N2OIV", 1);        //*>N2O Igniter Valve          */
        set_value("N2OMV", 1);        //*>N2O Main Valve                    */
        set_value("TIME", 16);        //*>Timestamp (ms)                    */
}


void update_all_val_receve(void){

}




// événementielle



void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{

  if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &RxHeader, RxData) != HAL_OK)
  {
    Error_Handler();
  }
  if ((RxHeader.StdId == 0x103))    // suposé être l'id de l'autre CAN
  {
          datacheck = 1;
          for (int i = 0; i < N_BYTES; i++){
                  out_Rx[i] = RxData[i];
          }

          // update du conteneur de donnée.
          //change_val_to_TagData(&RxData);
  }
}
```

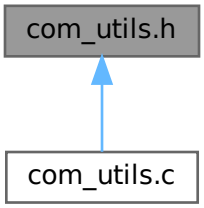
14.8 com_utils.h File Reference

```
#include "../../TAGCAN/TAGCAN.h"
#include "../../TAGCAN/include/sub_include/data_container.h"
#include <cmsis_gcc.h>
#include <stm32f103xb.h>
#include <stm32f1xx.h>
#include <stm32f1xx_hal_can.h>
#include <stm32f1xx_hal_def.h>
#include <stm32f1xx_hal_flash.h>
#include <stm32f1xx_hal_rcc.h>
#include <stm32f1xx_hal_rcc_ex.h>
#include <sys/_stdint.h>
```

Include dependency graph for com_utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- void init_com (void)

Variables

- uint8_t TxData [8]
- uint8_t RxData [8]
- CAN_TxHeaderTypeDef TxHeader
- CAN_RxHeaderTypeDef RxHeader
- CAN_HandleTypeDef hcan
- int32_t TxMailbox
- int8_t val_SEQ
- int8_t val_HB
- int8_t val_CI
- int8_t val_AB
- int8_t val_EDC
- int8_t val_N2ODVS
- int8_t val_N2OIV
- int8_t val_N2OMV
- int16_t val_N2OPS
- int16_t val_TIME
- int16_t val_TLC

14.8.1 Function Documentation

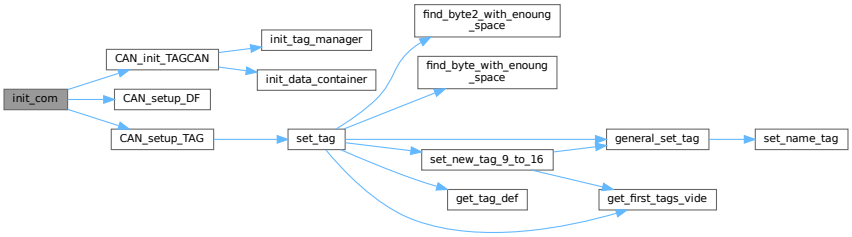
14.8.1.1 init_com()

```
void init_com (
            void  )
```

- initialise tout les fonction de communication. (point d'appel unique: main) */

Definition at line 39 of file com_utils.c.

Here is the call graph for this function:



14.8.2 Variable Documentation

14.8.2.1 hcan

```
CAN_HandleTypeDef hcan [extern]
```

14.8.2.2 RxData

```
uint8_t RxData[8] [extern]
```

Definition at line 13 of file com_utils.c.

14.8.2.3 RxHeader

```
CAN_RxHeaderTypeDef RxHeader [extern]
```

Definition at line 10 of file com_utils.c.

14.8.2.4 TxData

```
uint8_t TxData[8] [extern]
```

Definition at line 12 of file data_container.c.

14.8.2.5 TxHeader

```
CAN_TxHeaderTypeDef TxHeader [extern]
```

Definition at line 9 of file com_utils.c.

14.8.2.6 TxMailbox

```
int32_t TxMailbox [extern]
```

Definition at line 36 of file com_utils.c.

14.8.2.7 val_AB

```
int8_t val_AB [extern]
```

Definition at line 19 of file com_utils.c.

14.8.2.8 val_CI

```
int8_t val_CI [extern]
```

Definition at line 18 of file com_utils.c.

14.8.2.9 val_EDC

```
int8_t val_EDC [extern]
```

Definition at line 20 of file com_utils.c.

14.8.2.10 val_HB

`int8_t val_HB [extern]`
Definition at line 17 of file com_utils.c.

14.8.2.11 val_N2ODVS

`int8_t val_N2ODVS [extern]`
Definition at line 23 of file com_utils.c.

14.8.2.12 val_N2OIV

`int8_t val_N2OIV [extern]`
Definition at line 24 of file com_utils.c.

14.8.2.13 val_N2OMV

`int8_t val_N2OMV [extern]`
Definition at line 25 of file com_utils.c.

14.8.2.14 val_N2OPS

`int16_t val_N2OPS [extern]`
Definition at line 21 of file com_utils.c.

14.8.2.15 val_SEQ

`int8_t val_SEQ [extern]`
Definition at line 16 of file com_utils.c.

14.8.2.16 val_TIME

`int16_t val_TIME [extern]`
Definition at line 26 of file com_utils.c.

14.8.2.17 val_TLC

`int16_t val_TLC [extern]`
Definition at line 22 of file com_utils.c.

14.9 com_utils.h

Go to the documentation of this file.

```
00001 /*
00002  * com_logic.h
00003  *
00004  *  Created on: 4 Mar 2026
00005  *      Author: Samuel
00006  */
00007
00008 #ifndef INC_COM_UTILS_H_
00009 #define INC_COM_UTILS_H_
00010
00011 #include "../../TAGCAN/TAGCAN.h"
00012 #include "../../TAGCAN/include/sub_include/data_container.h"
00013 #include <cmsis_gcc.h>
00014 #include <stm32f103xb.h>
00015 #include <stm32f1xx.h>
00016 #include <stm32f1xx_hal_can.h>
00017 #include <stm32f1xx_hal_def.h>
00018 #include <stm32f1xx_hal_flash.h>
00019 #include <stm32f1xx_hal_rcc.h>
00020 #include <stm32f1xx_hal_rcc_ex.h>
00021 #include <sys/_stdint.h>
00022
00023 extern uint8_t TxData[8];
00024 extern uint8_t RxData[8];
```

```
extern CAN_TxHeaderTypeDef TxHeader;
extern CAN_RxHeaderTypeDef RxHeader;
extern CAN_HandleTypeDef hcan;
extern int32_t TxMailbox;

extern int8_t val_SEQ;
extern int8_t val_HB;
extern int8_t val_CI;
extern int8_t val_AB;
extern int8_t val_EDC;

extern int8_t val_N2ODVS;
extern int8_t val_N2OIV;
extern int8_t val_N2OMV;
extern int16_t val_N2OPS;

extern int16_t val_TIME;
extern int16_t val_TLC;

void init_com(void);


#endif /* INC_COM_UTILS_H_ */
```

14.10 01_INITIALISATION.md File Reference

14.11 02_GESTION_TAGS.md File Reference

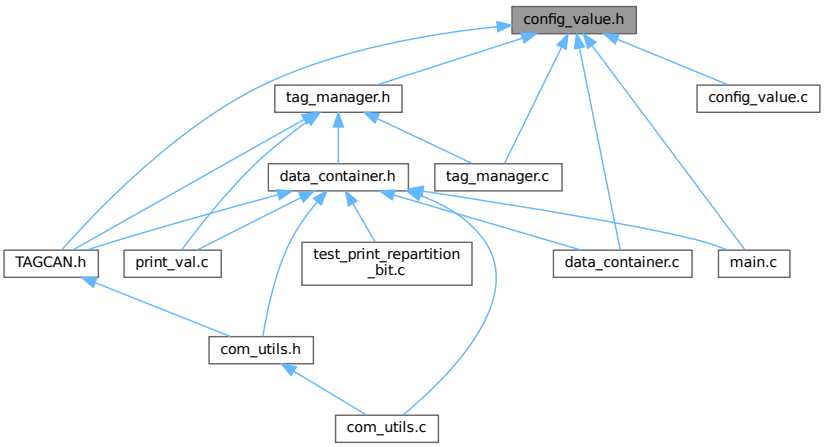
14.12 03_DONNEES_CAN.md File Reference

14.13 04_FILTRAGE_CAN.md File Reference

14.14 05_EXEMPLES_COMPLETS.md File Reference

14.15 config_value.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define MAX_TAGS 32
- #define MAX_TAG_NAME_LEN 16
- #define N_BYTES 8
- #define MAX_N_GT 3

Enumerations

- enum CAN_TG_STATUE { CAN_TG_ERROR_NOT_ENOUNG_SPACE_ = -2 , CAN_TG_SUCCESS = 1 , CAN_TG_ERROR_TAG_NOT_FOUND_ = -3 , CAN_TG_ERROR_TAG_NAME_TO_BIGS_ = -4 , CAN_TG_ERROR_TOO_MAY_TAGS_ = -5 , CAN_TG_ERROR_SIZE_VALUE_INVALIDE_ = -6 , CAN_TG_ERROR_NAME_ALREADY_USE_ = -7 , CAN_TG_ERROR_SUBDIVISION_FAIL_ = -8 , CAN_TG_ERROR_TAG_NAME_INVALIDE_ =-9 , CAN_TG_ERROR_NULL_POINTOR_ = -2 , CAN_TG_ERROR_TRY_TO_FI = -21 , CAN_TG_ERROR_INTERNAL_CORUPTION_ = -30 , CAN_TG_ERROR_VALUE_TO_BIG = -10 }

  Codes de retour du module TAGCAN.
- enum CAN_MODE { RECEVE_ONLY , SEND_ONLY , SEND_AND_RECEVE }

  modes d'utilisation CAN (envoi/reception).

Variables

- const int CAN_TG_ERROR_INTERNAL_CORUPTION
- const int CAN_TG_ERROR_TAG_NAME_TO_BIGS
- const int CAN_TG_ERROR_TAG_NAME_INVALIDE
- const int CAN_TG_ERROR_TAG_NOT_FOUND
- const int CAN_TG_ERROR_NOT_ENOUNG_SPACE
- const int CAN_TG_ERROR_NULL_POINTOR
- const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER
- const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE
- const int CAN_TG_ERROR_SUBDIVISION_FAIL
- const int CAN_TG_ERROR_TOO_MAY_TAGS
- const int CAN_TG_ERROR_NAME_ALREADY_USE
- const int SUCCES_TO_SET
- const int SUCCES

14.15.1 Macro Definition Documentation

14.15.1.1 MAX_N_GT

```
#define MAX_N_GT 3
```

Nombre maximal de gestionnaires (reserve/extension).
Definition at line 11 of file config_value.h.

14.15.1.2 MAX_TAG_NAME_LEN

```
#define MAX_TAG_NAME_LEN 16
```

Longueur maximale d'un nom de tag (incluant le '\0').
Definition at line 7 of file config_value.h.

14.15.1.3 MAX_TAGS

```
#define MAX_TAGS 32
```

Nombre maximal de tags geres simultanement.
Definition at line 5 of file config_value.h.

14.15.1.4 N_BYTES

```
#define N_BYTES 8
```

Taille du payload CAN en octets.
Definition at line 9 of file config_value.h.

14.15.2 Enumeration Type Documentation

14.15.2.1 CAN_MODE

`enum CAN_MODE`

modes d'utilisation CAN (envoi/reception).

Enumerator

RECEVE_ONLY	Réception seule: les mises à jour entrantes peuvent modifier la valeur; l'écriture locale est bloquée.
SEND_ONLY	Émission seule: la valeur locale est mise, les mises à jour entrantes ne la modifient pas.
SEND_AND_RECEVE	Emission + reception: écriture locale et mises à jour entrantes toutes deux autorisées.

Definition at line 58 of file config_value.h.

14.15.2.2 CAN_TG_STATUE

`enum CAN_TG_STATUE`

Codes de retour du module TAGCAN.

Enumerator

CAN_TG_ERROR_NOT_ENOUNG_SPACE_	
CAN_TG_SUCCESS	
CAN_TG_ERROR_TAG_NOT_FOUND_	
CAN_TG_ERROR_TAG_NAME_TO_BIGS_	
CAN_TG_ERROR_TOO_MAY_TAGS_	
CAN_TG_ERROR_SIZE_VALUE_INVALIDE_	
CAN_TG_ERROR_NAME_ALREADY_USE_	
CAN_TG_ERROR_SUBDIVISION_FAIL_	
CAN_TG_ERROR_TAG_NAME_INVALIDE_	
CAN_TG_ERROR_NULL_POINTOR_	
CAN_TG_ERROR_TRY_TO_FIND_END_↩ CARACTER_	
CAN_TG_ERROR_INTERNAL_CORUPTION_	
CAN_TG_ERROR_VALUE_TO_BIG	

Definition at line 41 of file config_value.h.

14.15.3 Variable Documentation

14.15.3.1 CAN_TG_ERROR_INTERNAL_CORUPTION

`const int CAN_TG_ERROR_INTERNAL_CORUPTION [extern]`

Code d'erreur: corruption interne detectee.

Definition at line 15 of file config_value.c.

14.15.3.2 CAN_TG_ERROR_NAME_ALREADY_USE

`const int CAN_TG_ERROR_NAME_ALREADY_USE [extern]`

Code d'erreur: nom de tag deja utilise.

Definition at line 10 of file config_value.c.

14.15.3.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE

`const int CAN_TG_ERROR_NOT_ENOUNG_SPACE [extern]`
Code d'erreur: espace memoire insuffisant.
Definition at line 5 of file config_value.c.

14.15.3.4 CAN_TG_ERROR_NULL_POINTOR

`const int CAN_TG_ERROR_NULL_POINTOR [extern]`
Code d'erreur: pointeur NULL recu.
Definition at line 13 of file config_value.c.

14.15.3.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE

`const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE [extern]`
Code d'erreur: taille de valeur invalide.
Definition at line 9 of file config_value.c.

14.15.3.6 CAN_TG_ERROR_SUBDIVISION_FAIL

`const int CAN_TG_ERROR_SUBDIVISION_FAIL [extern]`
Code d'erreur: echec de subdivision/allocation.
Definition at line 11 of file config_value.c.

14.15.3.7 CAN_TG_ERROR_TAG_NAME_INVALIDE

`const int CAN_TG_ERROR_TAG_NAME_INVALIDE [extern]`
Code d'erreur: nom de tag invalide.
Definition at line 12 of file config_value.c.

14.15.3.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS

`const int CAN_TG_ERROR_TAG_NAME_TO_BIGS [extern]`
Code d'erreur: nom de tag trop long.
Definition at line 7 of file config_value.c.

14.15.3.9 CAN_TG_ERROR_TAG_NOT_FOUND

`const int CAN_TG_ERROR_TAG_NOT_FOUND [extern]`
Code d'erreur: tag introuvable.
Definition at line 6 of file config_value.c.

14.15.3.10 CAN_TG_ERROR_TOO_MAY_TAGS

`const int CAN_TG_ERROR_TOO_MAY_TAGS [extern]`
Code d'erreur: trop de tags declares.
Definition at line 8 of file config_value.c.

14.15.3.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER

`const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER [extern]`
Code d'erreur: fin de chaine non trouvee.
Definition at line 14 of file config_value.c.

14.15.3.12 SUCCES

`const int SUCCES [extern]`
Code de succes generique historique.
Definition at line 17 of file config_value.c.

14.15.3.13 SUCCES_TO_SET

```
const int SUCCES_TO_SET  [extern]
```

Code de succes historique pour set_tag/set_value.
Definition at line 16 of file config_value.c.

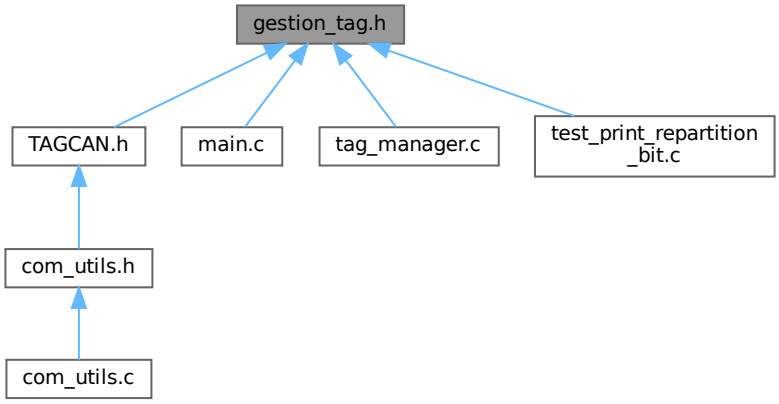
14.16 config_value.h

Go to the documentation of this file.

```
00001 #ifndef CONFIG_VALUE_H
00002 #define CONFIG_VALUE_H
00003
00005 #define MAX_TAGS 32
00007 #define MAX_TAG_NAME_LEN 16
00009 #define N_BYTES 8
00011 #define MAX_N_GT 3
00012
00014 extern const int CAN_TG_ERROR_INTERNAL_CORUPTION;
00016 extern const int CAN_TG_ERROR_TAG_NAME_TO_BIGS;
00018 extern const int CAN_TG_ERROR_TAG_NAME_INVALIDE;
00020 extern const int CAN_TG_ERROR_TAG_NOT_FOUND;
00022 extern const int CAN_TG_ERROR_NOT_ENOUNG_SPACE;
00024 extern const int CAN_TG_ERROR_NULL_POINTOR;
00026 extern const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER;
00028 extern const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE;
00030 extern const int CAN_TG_ERROR_SUBDIVISION_FAIL;
00032 extern const int CAN_TG_ERROR_TOO_MAY_TAGS;
00034 extern const int CAN_TG_ERROR_NAME_ALREADY_USE;
00036 extern const int SUCCES_TO_SET;
00038 extern const int SUCCES;
00039 // Codes de retour utilises par l'API TAGCAN
00041 typedef enum {
00042     CAN_TG_ERROR_NOT_ENOUNG_SPACE_ = -2,
00043     CAN_TG_SUCCESS = 1,
00044     CAN_TG_ERROR_TAG_NOT_FOUND_ = -3,
00045     CAN_TG_ERROR_TAG_NAME_TO_BIGS_ = -4,
00046     CAN_TG_ERROR_TOO_MAY_TAGS_ = -5,
00047     CAN_TG_ERROR_SIZE_VALUE_INVALIDE_ = -6,
00048     CAN_TG_ERROR_NAME_ALREADY_USE_ = -7,
00049     CAN_TG_ERROR_SUBDIVISION_FAIL_ = -8,
00050     CAN_TG_ERROR_TAG_NAME_INVALIDE_=-9,
00051     CAN_TG_ERROR_NULL_POINTOR_ = -2,
00052     CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER_ = -21,
00053     CAN_TG_ERROR_INTERNAL_CORUPTION_ = -30,
00054     CAN_TG_ERROR_VALUE_TO_BIG = -10,
00055 } CAN_TG_STATUE;
00056
00058 typedef enum {
00059     RECEVE_ONLY,
00060     SEND_ONLY,
00061     SEND_AND_RECEVE,
00062 } CAN_MODE;
00063
00064 #endif // CONFIG_VALUE_H
```

14.17 gestion_tag.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- void init_tag_manager (void)

  Initialise/reinitialise le gestionnaire de tags.
- int set_tag (const char *tag_name, uint8_t n_bit)

  créé un nouveau tag.

14.17.1 Function Documentation

14.17.1.1 init_tag_manager()

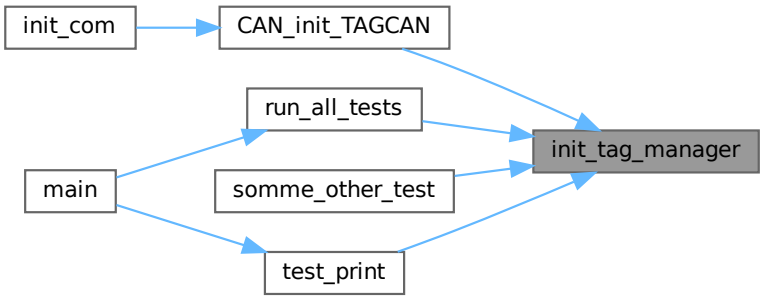
```
void init_tag_manager (
            void  )
```

Initialise/reinitialise le gestionnaire de tags.

Note

Apres cet appel, l'ensemble des tags doit etre redefini via `set_tag`.

Definition at line 21 of file tag_manager.c.
Here is the caller graph for this function:



14.17.1.2 set_tag()

```
int set_tag (
            const char * tag_name,
            uint8_t siz )
```

créé un nouveau tag.

Parameters

tag_name	le nom (id) relié a la valeur.
n_bit	La taille de la valeur (implémentation actuelle: 1 à 16 bits).

Returns

un code de succès ou d'erreur.

créé un nouveau tag.

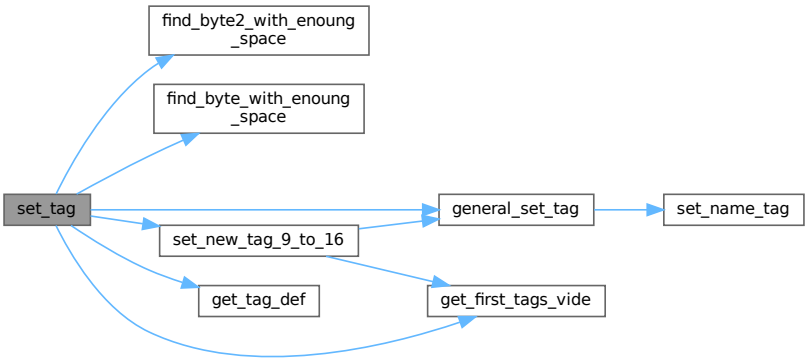
Parameters

tag_name	Nom logique du tag.
siz	Taille de la valeur en bits (implementation actuelle: 1..16).

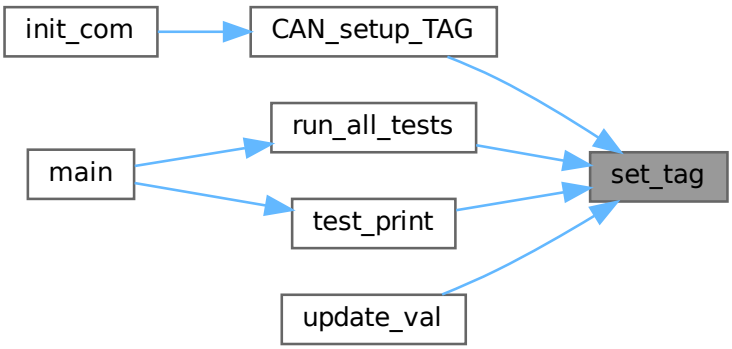
Returns

Code de succes ou code d'erreur CAN_TG_STATUE.

Definition at line 232 of file tag_manager.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.18 gestion_tag.h

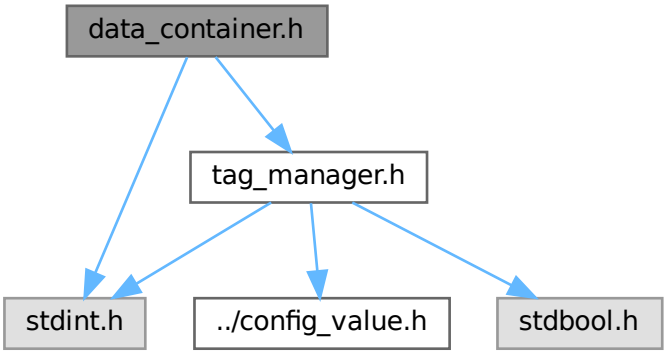
Go to the documentation of this file.

```
00001 #ifndef GESTION_TAG_H
00002 #define GESTION_TAG_H
00003
00004 // fonction qui gere les tags
00009 void init_tag_manager(void);
00010
00017 int set_tag(const char* tag_name, uint8_t n_bit);
00018
00019 #endif /* GESTION_TAG_H */
00020
00021 #ifndef GESTION_TAG_H
00022 #define GESTION_TAG_H
00023 #include <stdint.h>
00024 // fonction qui gère les tags
00029 void init_tag_manager(void);
00030
00037 int set_tag(const char* tag_name, uint8_t n_bit);
00038
00039 #endif /* GESTION_TAG_H */
```

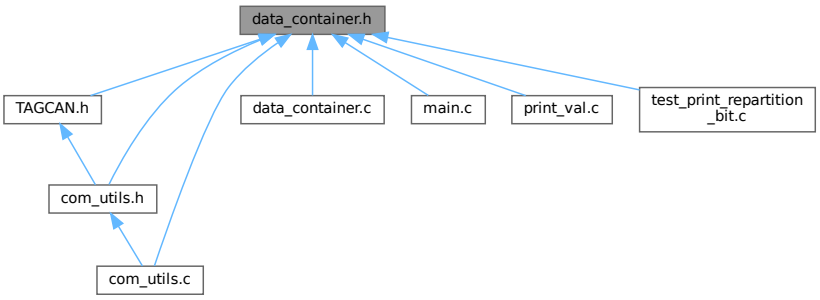
14.19 data_container.h File Reference

```
#include <stdint.h>
#include "tag_manager.h"
```

Include dependency graph for data_container.h:



This graph shows which files directly or indirectly include this file:



Functions

- void init_data_container (void)

  Reinitialise le payload TxData a zero.
- int set_value (const char ∗tag_name, uint32_t value)

  Ecrit une valeur dans TxData a partir d'un tag.
- int get_value (const char ∗tag_name, uint32_t ∗out_value)

  Lit la valeur associee a un tag.
- uint8_t prep_mask (uint8_t p_bit, uint8_t n_bit)

  [FONCTION INTERNE] Prepare un masque binaire pour set/get.
- void can_simulate_send_receive (uint8_t ∗tx_buffer, uint8_t ∗rx_buffer)

  Simule un envoi/reception CAN par copie buffer-a-buffer.
- void set_val_TxData (uint8_t ∗TxDataArray)

  Copie un tableau externe dans le buffer global TxData.

Variables

- uint8_t TxData [N_BYTES]

14.19.1 Function Documentation

14.19.1.1 can_simulate_send_receive()

```
void can_simulate_send_receive (
            uint8_t * tx_buffer,
            uint8_t * rx_buffer )
```

Simule un envoi/reception CAN par copie buffer-a-buffer.

Parameters

tx_buffer	Buffer source (emission).
rx_buffer	Buffer destination (reception).

Simule un envoi/reception CAN par copie buffer-a-buffer.
Copie simplement tx_buffer dans rx_buffer.

Parameters

tx_buffer	Buffer d'emission.
rx_buffer	Buffer de reception.

Definition at line 165 of file data_container.c.

14.19.1.2 get_value()

```
int get_value (
            const char * tag_name,
            uint32_t * out_value )
```

Lit la valeur associee a un tag.

Parameters

tag_name	Nom du tag a lire.
out_value	Adresse de sortie qui recoit la valeur extraite.

Returns

CAN_TG_SUCCESS en cas de succes, sinon un code d'erreur negatif.

Copie un buffer externe dans TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Recompose un entier 16 bits depuis deux octets.

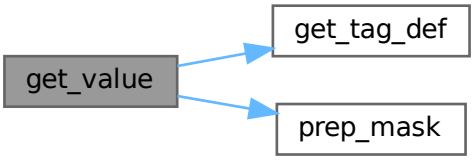
Parameters

ba	Octet de poids faible.
bb	Octet de poids fort.

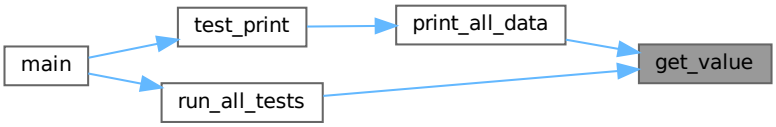
Returns

Valeur 16 bits combinee.

Definition at line 106 of file data_container.c.
Here is the call graph for this function:



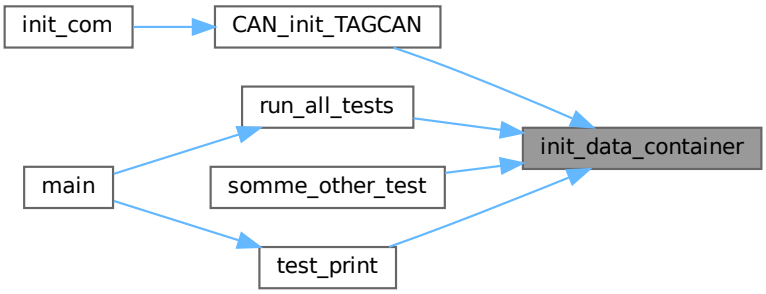
Here is the caller graph for this function:



14.19.1.3 init_data_container()

```
void init_data_container (
            void  )
```

Reinitialise le payload TxData a zero.
Si reexecutee, la fonction efface les donnees precedemment ecrites.
Definition at line 14 of file data_container.c.
Here is the caller graph for this function:



14.19.1.4 prep_mask()

```
uint8_t prep_mask (
            uint8_t p_bit,
            uint8_t n_bit )
```

[FONCTION INTERNE] Prepare un masque binaire pour set/get.
Utilise principalement pour les valeurs strictement inferieures a 8 bits.

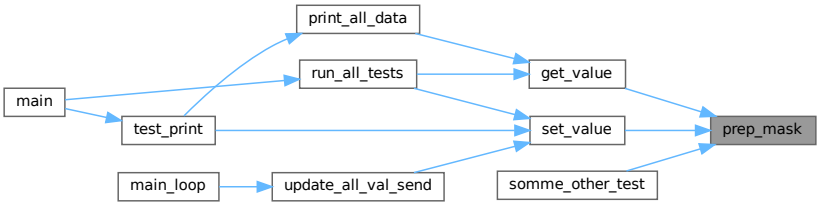
Parameters

p_bit	Position du premier bit (de gauche a droite).
n_bit	Nombre de bits de la valeur.

Returns

Masque binaire aligne sur la position cible.

Definition at line 29 of file data_container.c.
Here is the caller graph for this function:



14.19.1.5 set_val_TxData()

```
void set_val_TxData (
            uint8_t * TxDataArray )
```

Copie un tableau externe dans le buffer global TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Definition at line 24 of file data_container.c.

14.19.1.6 set_value()

```
int set_value (
            const char * tag_name,
            uint32_t value )
```

Ecrit une valeur dans TxData a partir d'un tag.

Warning

La valeur doit tenir dans la taille du tag.

En cas de depassement, l'implementation retourne CAN_TG_ERROR_VALUE_TO_BIG.

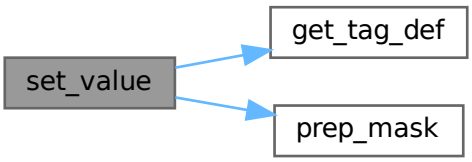
Parameters

tag_name	Nom du tag cible.
value	Nouvelle valeur a ecrire.

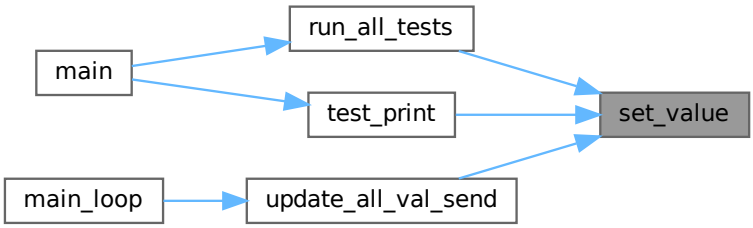
Returns

Code de succes ou code d'erreur.

Definition at line 44 of file data_container.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.19.2 Variable Documentation

14.19.2.1 TxData

```
uint8_t TxData[N_BYTES]  [extern]
```

Buffer de donnees CAN (payload de 8 octets).
Definition at line 12 of file data_container.c.

14.20 data_container.h

Go to the documentation of this file.

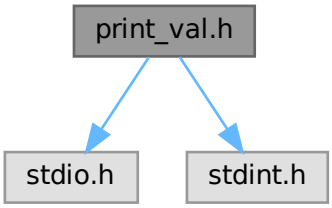
```
00001 #ifndef DATA_CONTAINER_H
00002
00003 #include <stdint.h>
00004
00005 #include "tag_manager.h"
00006
00007 #define DATA_CONTAINER_H
```

```
00008
00009 // Buffer de donnees CAN (payload de 8 octets)
00010 extern uint8_t TxData[N_BYTES];
00016 void init_data_container(void);
00017
00018 // Fonctions principales (set/get)
00027 int set_value(const char* tag_name, uint32_t value);
00028
00036 int get_value(const char* tag_name, uint32_t* out_value);
00037
00038
00046 uint8_t prep_mask(uint8_t p_bit, uint8_t n_bit);
00051 void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer);
00052
00053 //int CAN_RECEVE
00054
00058 void set_val_TxData(uint8_t *TxDataArray);
00059
00060
00061 #endif // DATA_CONTAINER_H
```

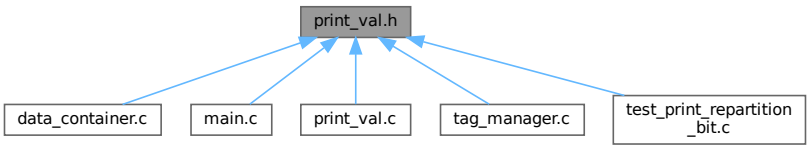
14.21 print_val.h File Reference

```
#include <stdio.h>
#include <stdint.h>
```

Include dependency graph for print_val.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct tsfp

  Buffer de rendu pour l'affichage de repartition bit-a-bit.

Functions

- void print_bin_8 (uint8_t val)

  Affiche la representation binaire (8 bits) d'une valeur.

- void print_all_data_bin (void)

  Affiche tout le payload TxData en binaire.
- void print_all_with_title (const char *title)

  Affiche TxData en binaire avec un titre en prefixe.
- void print_all_tag_set (void)

  Affiche la table des tags declares et leurs metadonnees.
- void print_all_data (void)

  Affiche toutes les valeurs logiques relues via get_value.
- void print_repartition_bit (void)

  Affiche la repartition des bits utilises dans la trame CAN.

Variables

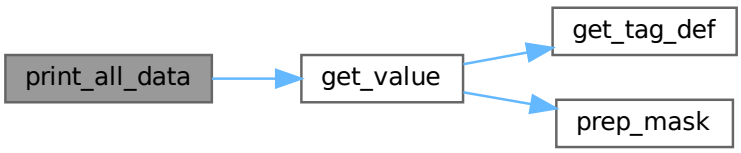
- tsfp gbc2 [9]

14.21.1 Function Documentation

14.21.1.1 print_all_data()

```
void print_all_data (
            void  )
```

Affiche toutes les valeurs logiques relues via get_value.
Affiche toutes les valeurs logiques relues via get_value.
Chaque valeur est relue via get_value puis imprimee.
Definition at line 43 of file print_val.c.
Here is the call graph for this function:



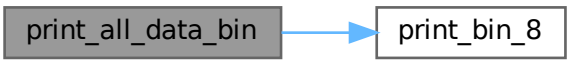
Here is the caller graph for this function:

14.21.1.2 print_all_data_bin()

```
void print_all_data_bin (
            void  )
```

Affiche tout le payload TxData en binaire.

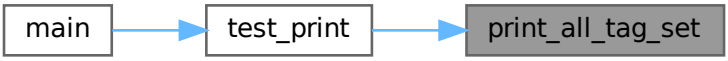
Chaque octet est affiche sur 8 bits, dans l'ordre du buffer.
Affiche tout le payload TxData en binaire.
Prints the binary representation of each byte in the `TxData` array.
Definition at line 150 of file print_val.c.
Here is the call graph for this function:



14.21.1.3 print_all_tag_set()

```
void print_all_tag_set (
            void  )
```

Affiche la table des tags declares et leurs metadonnees.
Affiche la table des tags declares et leurs metadonnees.
Definition at line 18 of file print_val.c.
Here is the caller graph for this function:



14.21.1.4 print_all_with_title()

```
void print_all_with_title (
            const char * title )
```

Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Texte affiche avant la ligne binaire.

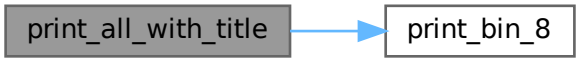
Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Libelle affiche avant les octets.

Definition at line 164 of file print_val.c.

Here is the call graph for this function:



14.21.1.5 print_bin_8()

```
void print_bin_8 (
            uint8_t val )
```

Affiche la representation binaire (8 bits) d'une valeur.

Parameters

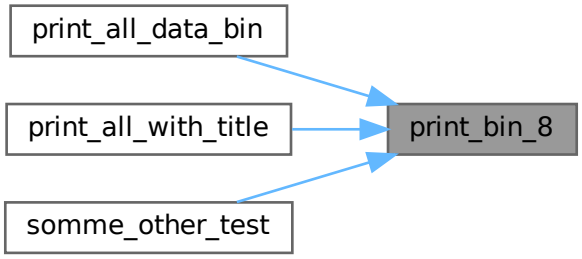
val	Valeur a afficher en binaire sur 8 bits.

Affiche la representation binaire (8 bits) d'une valeur.

Parameters

val	The `val` [from 0 to 255]. The function prints the binary representation of this value by iterating over each bit.

Definition at line 136 of file print_val.c.
Here is the caller graph for this function:



14.21.1.6 print_repartition_bit()

```
void print_repartition_bit (
            void )
```

Affiche la repartition des bits utilises dans la trame CAN.
Affiche la repartition des bits utilises dans la trame CAN.
Definition at line 78 of file print_val.c.

Here is the caller graph for this function:

14.21.2 Variable Documentation

14.21.2.1 gbc2

```
tsfp gbc2[9]  [extern]
```

Espace global de rendu utilise par les fonctions d'impression de repartition.
Definition at line 14 of file print_val.c.

14.22 print_val.h

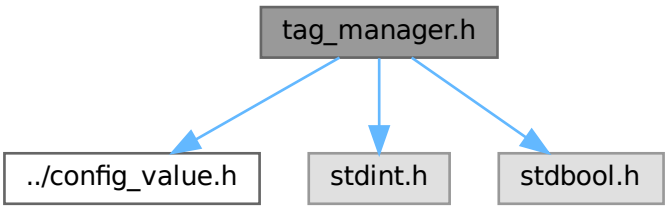
Go to the documentation of this file.

```
00001 #ifndef PRINT_VAL_H
00002
00003 #include <stdio.h>
00004 #include <stdint.h>
00005
00006 #define PRINT_VAL_H
00007
00008 // Utilitaires d'affichage pour le debogage
00014 void print_bin_8(uint8_t val);
00015
00019 void print_all_data_bin(void);
00020
00024 void print_all_with_title(const char* title);
00025
00028 void print_all_tag_set(void);
00029
00032 void print_all_data(void);
00033
00036 void print_repartition_bit(void);
00037
00039 typedef struct{
00040     char bi[8];
00041
00042 } tsfp;
00043
00045 extern tsfp gbc2[9];
00046
00047 #endif /* PRINT_VAL_H */
```

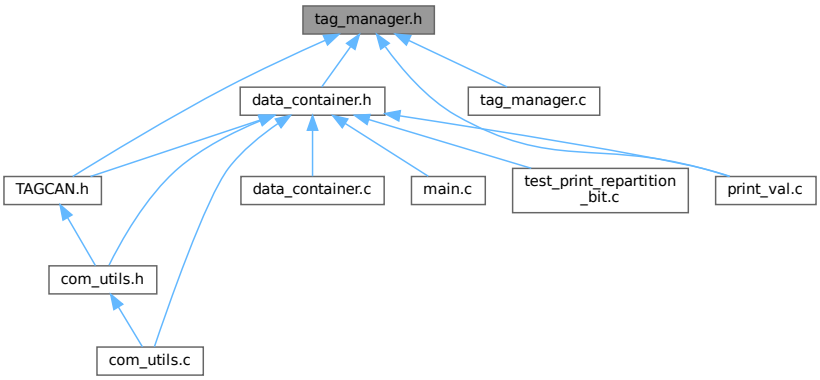
14.23 tag_manager.h File Reference

```
#include "../config_value.h"
#include <stdint.h>
#include <stdbool.h>
```

Include dependency graph for tag_manager.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct TagDef

  Structure de definition d'un tag.

Functions

- const TagDef ∗ get_tag_def (const char ∗tag_name)

  Recherche un tag et retourne sa definition.

Variables

- TagDef g_tags [MAX_TAGS]
- uint8_t g_tag_count
- uint8_t g_bytes_used [N_BYTES]

14.23.1 Function Documentation

14.23.1.1 get_tag_def()

```
const TagDef * get_tag_def (
            const char * tag_name )
```

Recherche un tag et retourne sa definition.

Warning

API surtout utilisee en interne par le module de valeurs.

Parameters

tag_name	Nom du tag a chercher.

Returns

Pointeur vers le tag si trouve, sinon NULL.

Recherche un tag et retourne sa definition.

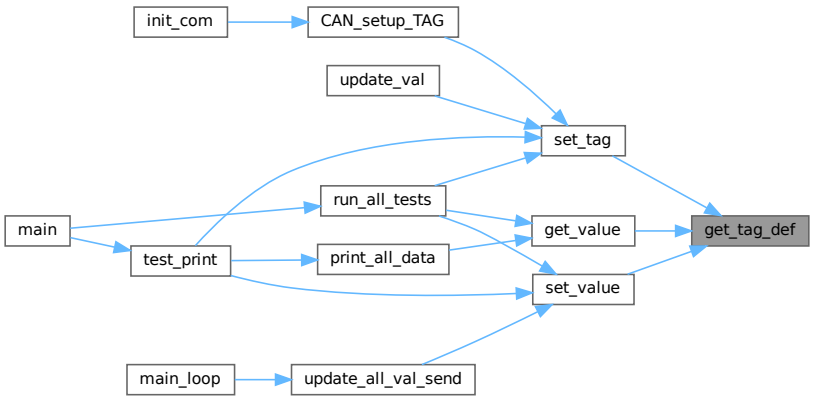
Parameters

tag_name	Nom du tag recherche.

Returns

Pointeur sur TagDef si trouve, sinon NULL.

Definition at line 288 of file tag_manager.c.
Here is the caller graph for this function:



14.23.2 Variable Documentation

14.23.2.1 g_bytes_used

`uint8_t g_bytes_used[N_BYTES] [extern]`
Occupation courante en bits pour chaque byte du payload.
Definition at line 19 of file tag_manager.c.

14.23.2.2 g_tag_count

`uint8_t g_tag_count [extern]`
Nombre de tags actifs (max: MAX_TAGS).
Definition at line 18 of file tag_manager.c.

14.23.2.3 g_tags

```
TagDef g_tags[MAX_TAGS]  [extern]
```

Table des tags (actifs et inactifs).

Definition at line 17 of file tag_manager.c.

14.24 tag_manager.h

Go to the documentation of this file.

```
#ifndef TAG_MANAGER_H
#define TAG_MANAGER_H

#ifndef CONFIG_VALUE_H
#include "../config_value.h"
#endif // CONFIG_VALUE_H
#include <stdint.h>
#include <stdbool.h>


typedef struct
{
    char name[MAX_TAG_NAME_LEN];
    uint8_t n_bits;
    int8_t byte_idx_a;
    int8_t byte_idx_b;
    uint8_t bit_pos_a;
    uint8_t bit_pos_b;
} TagDef;

// Etat global du gestionnaire de tags

extern TagDef g_tags[MAX_TAGS];
extern uint8_t g_tag_count;
// Nombre de bits deja utilises dans chaque byte (0..7)
extern uint8_t g_bytes_used[N_BYTES];
// structure de multi gestionnaire de tags (en dévelopement)

/*typedef struct {


}*/

//--------------------------------fonctions d'acces generales ------------------------
const TagDef* get_tag_def(const char* tag_name);

#endif // TAG_MANAGER_H
```

14.25 CAN_utils.c File Reference

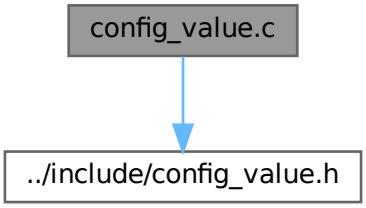
14.26 CAN_utils.c

Go to the documentation of this file.

```
```

14.27 config_value.c File Reference

`#include "../include/config_value.h"`
Include dependency graph for config_value.c:



Variables

- const int CAN_TG_ERROR_NOT_ENOUNG_SPACE = -2
- const int CAN_TG_ERROR_TAG_NOT_FOUND = -3
- const int CAN_TG_ERROR_TAG_NAME_TO_BIGS = -4
- const int CAN_TG_ERROR_TOO_MAY_TAGS = -5
- const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE = -6
- const int CAN_TG_ERROR_NAME_ALREADY_USE = -7
- const int CAN_TG_ERROR_SUBDIVISION_FAIL = -8
- const int CAN_TG_ERROR_TAG_NAME_INVALIDE =-9
- const int CAN_TG_ERROR_NULL_POINTOR = -20
- const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER = -21
- const int CAN_TG_ERROR_INTERNAL_CORUPTION = -30
- const int SUCCES_TO_SET = 1
- const int SUCCES = 1

14.27.1 Variable Documentation

14.27.1.1 CAN_TG_ERROR_INTERNAL_CORUPTION

`const int CAN_TG_ERROR_INTERNAL_CORUPTION = -30`
Code d'erreur: corruption interne detectee.
Definition at line 15 of file config_value.c.

14.27.1.2 CAN_TG_ERROR_NAME_ALREADY_USE

`const int CAN_TG_ERROR_NAME_ALREADY_USE = -7`
Code d'erreur: nom de tag deja utilise.
Definition at line 10 of file config_value.c.

14.27.1.3 CAN_TG_ERROR_NOT_ENOUNG_SPACE

`const int CAN_TG_ERROR_NOT_ENOUNG_SPACE = -2`
Code d'erreur: espace memoire insuffisant.
Definition at line 5 of file config_value.c.

14.27.1.4 CAN_TG_ERROR_NULL_POINTOR

```
const int CAN_TG_ERROR_NULL_POINTOR = -20
```

Code d'erreur: pointeur NULL recu.
Definition at line 13 of file config_value.c.

14.27.1.5 CAN_TG_ERROR_SIZE_VALUE_INVALIDE

```
const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE = -6
```

Code d'erreur: taille de valeur invalide.
Definition at line 9 of file config_value.c.

14.27.1.6 CAN_TG_ERROR_SUBDIVISION_FAIL

```
const int CAN_TG_ERROR_SUBDIVISION_FAIL = -8
```

Code d'erreur: echec de subdivision/allocation.
Definition at line 11 of file config_value.c.

14.27.1.7 CAN_TG_ERROR_TAG_NAME_INVALIDE

```
const int CAN_TG_ERROR_TAG_NAME_INVALIDE =-9
```

Code d'erreur: nom de tag invalide.
Definition at line 12 of file config_value.c.

14.27.1.8 CAN_TG_ERROR_TAG_NAME_TO_BIGS

```
const int CAN_TG_ERROR_TAG_NAME_TO_BIGS = -4
```

Code d'erreur: nom de tag trop long.
Definition at line 7 of file config_value.c.

14.27.1.9 CAN_TG_ERROR_TAG_NOT_FOUND

```
const int CAN_TG_ERROR_TAG_NOT_FOUND = -3
```

Code d'erreur: tag introuvable.
Definition at line 6 of file config_value.c.

14.27.1.10 CAN_TG_ERROR_TOO_MAY_TAGS

```
const int CAN_TG_ERROR_TOO_MAY_TAGS = -5
```

Code d'erreur: trop de tags declares.
Definition at line 8 of file config_value.c.

14.27.1.11 CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER

```
const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER = -21
```

Code d'erreur: fin de chaine non trouvee.
Definition at line 14 of file config_value.c.

14.27.1.12 SUCCES

```
const int SUCCES = 1
```

Code de succes generique historique.
Definition at line 17 of file config_value.c.

14.27.1.13 SUCCES_TO_SET

```
const int SUCCES_TO_SET = 1
```

Code de succes historique pour set_tag/set_value.
Definition at line 16 of file config_value.c.

14.28 config_value.c

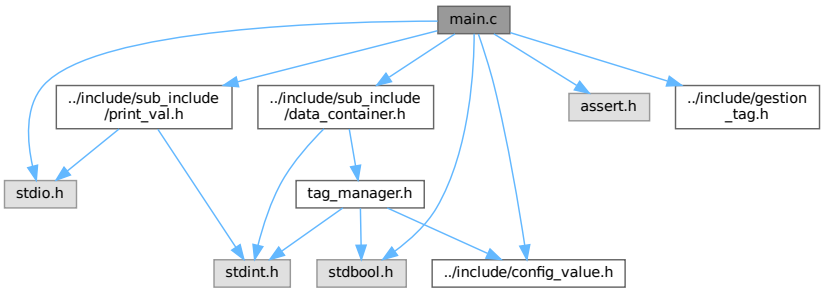
Go to the documentation of this file.

```
#ifndef CONFIG_VALUE_H
#include "../include/config_value.h"
#endif // CONFIG_VALUE_H
// Definitions des constantes globales historiques de codes de retour.
const int CAN_TG_ERROR_NOT_ENOUNG_SPACE = -2;
const int CAN_TG_ERROR_TAG_NOT_FOUND = -3;
const int CAN_TG_ERROR_TAG_NAME_TO_BIGS = -4;
const int CAN_TG_ERROR_TOO_MAY_TAGS = -5;
const int CAN_TG_ERROR_SIZE_VALUE_INVALIDE = -6;
const int CAN_TG_ERROR_NAME_ALREADY_USE = -7;
const int CAN_TG_ERROR_SUBDIVISION_FAIL = -8;
const int CAN_TG_ERROR_TAG_NAME_INVALIDE=-9;
const int CAN_TG_ERROR_NULL_POINTOR = -20;
const int CAN_TG_ERROR_TRY_TO_FIND_END_CARACTER = -21;
const int CAN_TG_ERROR_INTERNAL_CORUPTION = -30;
const int SUCCES_TO_SET = 1;
const int SUCCES = 1;
```

14.29 main.c File Reference

```
#include <stdio.h>
#include <assert.h>
#include <stdbool.h>
#include "../include/config_value.h"
#include "../include/sub_include/data_container.h"
#include "../include/gestion_tag.h"
#include "../include/sub_include/print_val.h"
```

Include dependency graph for main.c:



Functions

- void TEST (const char *name, const bool condition)

  Affiche un resultat de test avec format uniforme.
- void TEST_EQ_INT (const char *name, const int val_1, const int val_2)

  Affiche un test d'egalite entre deux entiers.
- void test_print (void)

  Execute un scenario de demonstration des fonctions d'impression.
- void somme_other_test (void)

  Scenario de test manuel des operations de masque/shift.
- void run_all_tests (void)

  Execute les tests unitaires du module. Les resultats sont affiches dans le terminal.
- int main (void)

Variables

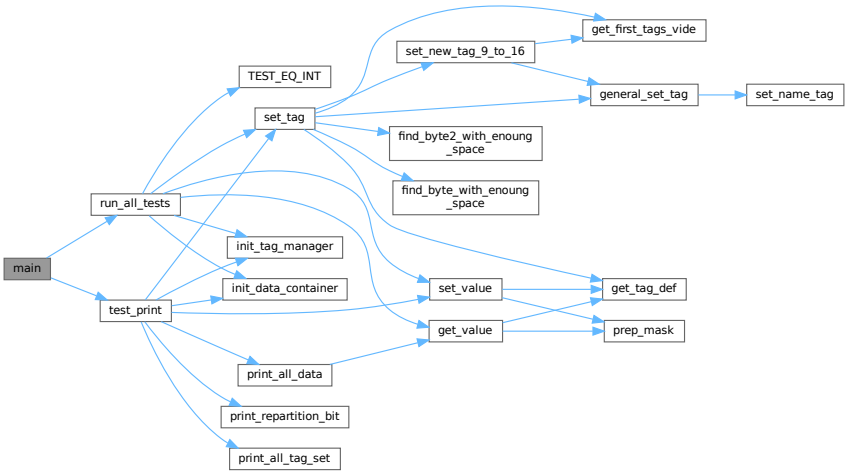
- int passed_tests = 0
- int total_tests = 0

14.29.1 Function Documentation

14.29.1.1 main()

```
int main (
            void  )
```

Definition at line 331 of file main.c.
Here is the call graph for this function:

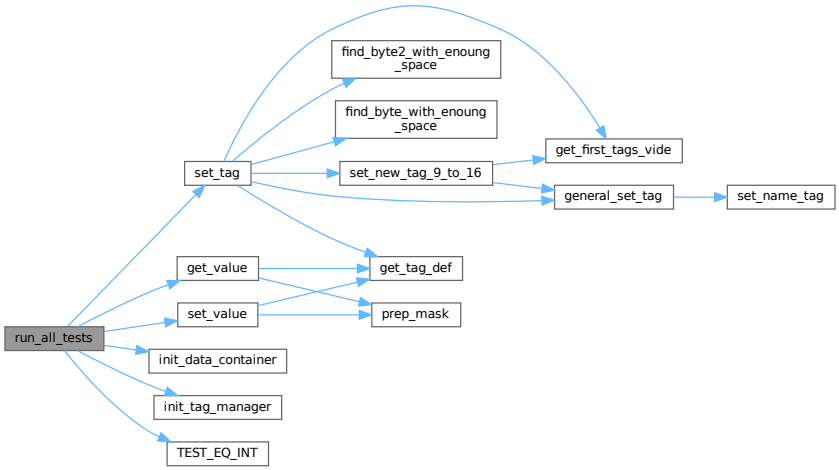


14.29.1.2 run_all_tests()

```
void run_all_tests (
            void  )
```

Execute les tests unitaires du module. Les resultats sont affiches dans le terminal.
Definition at line 177 of file main.c.

Here is the call graph for this function:



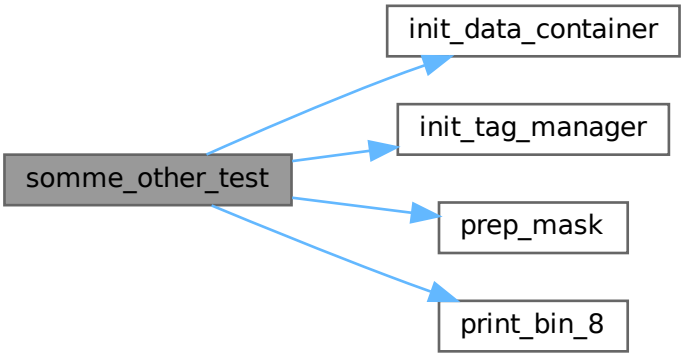
Here is the caller graph for this function:

14.29.1.3 somme_other_test()

```
void somme_other_test (
            void  )
```

Scenario de test manuel des operations de masque/shift.
Definition at line 104 of file main.c.

Here is the call graph for this function:



14.29.1.4 TEST()

```
void TEST (
            const char * name,
            const bool condition )
```

Affiche un resultat de test avec format uniforme.

Parameters

name	Le nom du test (peut aussi servir de description courte).
condition	Resultat du test (true si reussi, false sinon).

Definition at line 26 of file main.c.

14.29.1.5 TEST_EQ_INT()

```
void TEST_EQ_INT (
            const char * name,
            const int val_1,
            const int val_2 )
```

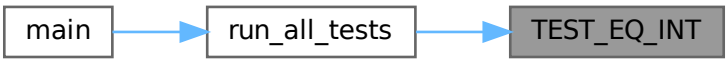
Affiche un test d'egalite entre deux entiers.

Parameters

name	Meme usage que TEST.
val_1	Premiere valeur a comparer.
val_2	Deuxieme valeur a comparer.

Definition at line 46 of file main.c.

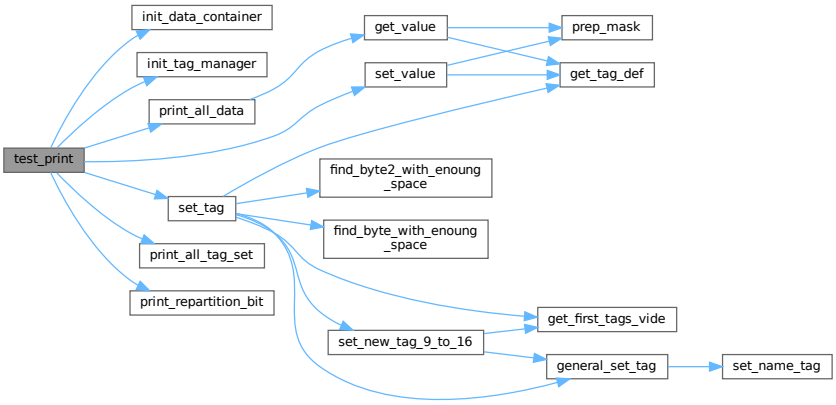
Here is the caller graph for this function:



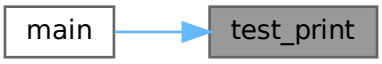
14.29.1.6 test_print()

```
void test_print (
            void  )
```

Execute un scenario de demonstration des fonctions d'impression.
Initialise des tags/valeurs puis affiche metadonnees et repartition.
Definition at line 72 of file main.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.29.2 Variable Documentation

14.29.2.1 passed_tests

```
int passed_tests = 0
```

le nombre total de test réussi
Definition at line 17 of file main.c.

14.29.2.2 total_tests

```
int total_tests = 0
```

le nombre total de test effectué

Definition at line 18 of file main.c.

14.30 main.c

Go to the documentation of this file.

```
00001 #include <stdio.h>
00002 #include <assert.h>
00003 #include <stdbool.h>
00004 #ifndef CONFIG_VALUE_H
00005 #include "../include/config_value.h"
00006 #endif // CONFIG_VALUE_H
00007 #ifndef DATA_CONTAINER_H
00008 #include "../include/sub_include/data_container.h"
00009 #endif // DATA_CONTAINER_H
00010 #ifndef GESTION_TAG_H
00011 #include "../include/gestion_tag.h"
00012 #endif // GESTION_TAG_H
00013 #ifndef PRINT_VAL_H
00014 #include "../include/sub_include/print_val.h"
00015 #endif // PRINT_VAL_H
00016
00017 int passed_tests = 0;
00018 int total_tests = 0;
00026 void TEST(const char* name, const bool condition)
00027 {
00028     if(condition)
00029     {
00030         printf("success\t\t\t%s\n", name);
00031         passed_tests++;
00032     }
00033     else
00034     {
00035         printf("fail\t\t\t%s\n", name);
00036     }
00037     total_tests++;
00038 }
00046 void TEST_EQ_INT(const char* name, const int val_1, const int val_2)
00047 {
00048     bool condition = val_1 == val_2;
00049     if(condition)
00050     {
00051         printf("success\t\t\t%s\n", name);
00052         passed_tests++;
00053     }
00054     else
00055     {
00056         printf("fail\t\t\t%s", name);
00057         printf("\t%d != ", val_1);
00058         printf("%d\n", val_2);
00059     }
00060     total_tests++;
00061 }
00062
00063
00064
00065
00066
00067
00068
00072 void test_print(void){
00073 printf("test des fonction d'impression.\n");
00074 // prepa du gt
00075 init_data_container();
00076 init_tag_manager();
00077
00078 // set the tags
00079 set_tag("a1",2);
00080 set_tag("agh5",5);
00081 set_tag("patap",16);
00082 set_tag("patat",12);
00083 set_tag("uni1",1);
00084 set_tag("triot", 3);
00085
00086 // set the value
00087 set_value("a1",3);
00088 set_value("agh5",42);
00089 set_value("patap",45);
00090 set_value("patat",12);
00091 set_value("uni1",1);
```

```c
set_value("triot", 3);

print_all_tag_set();
print_all_data();
print_repartition_bit();
}



void somme_other_test(void){
printf("----------------------test mask ------------------------\n");
    printf("\t(0,8): ");
    print_bin_8(prep_mask(0, 8));
    printf("\n");
    printf("\t(4,4): ");
    print_bin_8(prep_mask(4, 4));
    printf("\n");
    printf("\t(4,3): ");
    print_bin_8(prep_mask(4, 3));
    printf("\n");
    printf("\t(1,4): ");
    print_bin_8(prep_mask(1, 4));
    printf("\n");
    printf("\t(5,1): ");
    print_bin_8(prep_mask(5, 1));
    printf("\n");
    printf("------------------------------test shift----------------------\n");
    uint8_t val1 = 0b01110000;
    uint8_t bi = 0b01110100;
    printf("val1: ");
    print_bin_8(val1);
    printf("\n");
    int p_bit = 4;
    int n_bit = 4;
    // eq logic pour la prise de la valeur
    uint8_t vrai_val = val1 » (8 - p_bit - n_bit);
    // prepa du mask
    uint8_t mask_val = 0b1;
    uint8_t mask_temp = 0b1;
    // mask des bits de la valeurs
    for (int i = 0; i < (n_bit - 1); i++)
    {
        mask_val = mask_val « 1;
        mask_val |= mask_temp;
    }
    // mask pour conserver uniquement la valeurs dans le bytes
    uint8_t mask_val_p = mask_val « (8 - p_bit - n_bit);

    uint8_t byte_sans_val = bi;
    byte_sans_val &= ~mask_val_p;


    //mask_val = mask_val » (8 - n_bit);
    printf("vrai_val: ");
    print_bin_8(vrai_val);
    printf("\n");
    printf("mask: ");
    print_bin_8(mask_val);
    printf("\n");
    printf("mask2: ");
    print_bin_8(mask_val_p);
    printf("\n");
    printf("byte avant: ");
    print_bin_8(bi);
    printf("\n");
    printf("byte sans val: ");
    print_bin_8(byte_sans_val);
    printf("\n");


    printf("test final\n");

    init_data_container();
    init_tag_manager();

}

void run_all_tests(void)
{
    uint32_t val = 0;

    printf("=== Lancement des tests unitaires ===\n");

    // ----------------------------------------------------------
    // TEST 1 : Cas d'erreurs de set_tag
    // ----------------------------------------------------------
```

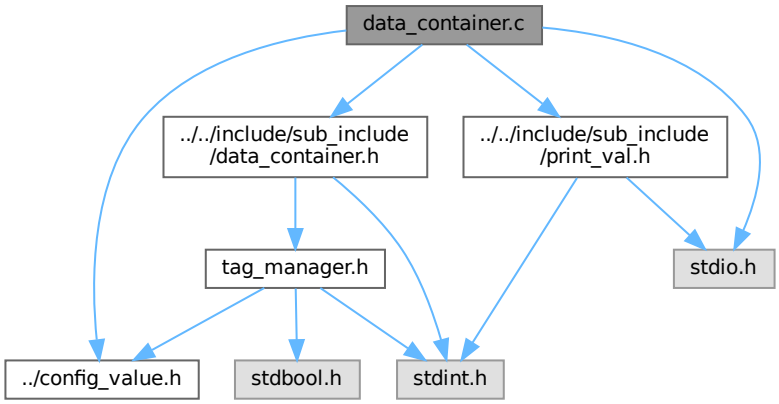
```
    printf("Test 1: Erreurs de set_tag... ");
    init_tag_manager(); // On part d'un état propre

    // Tailles invalides (ex: 0 retourne CAN_TG_ERROR_SIZE_VALUE_INVALIDE)
    printf("\tTailles invalides\n");
    TEST_EQ_INT("0",set_tag("inv1", 0), CAN_TG_ERROR_SIZE_VALUE_INVALIDE);
    TEST_EQ_INT("2",set_tag("inv2", 2), CAN_TG_SUCCESS);
    TEST_EQ_INT("5",set_tag("inv5", 5), CAN_TG_SUCCESS);

    // Tag en double (retour attendu: CAN_TG_ERROR_NAME_ALREADY_USE)
    printf("\tTag en double\n");
    TEST_EQ_INT("première insertion",set_tag("TAG_A", 4), SUCCES_TO_SET); // Succès
    TEST_EQ_INT("duplica détection",set_tag("TAG_A", 3), CAN_TG_ERROR_NAME_ALREADY_USE); // Erreur : existe déjà
    printf("OK\n");

    // ---------------------------------------------------------
    // TEST 2 : Dépassement de l'espace mémoire (8 octets max)
    // ---------------------------------------------------------
    printf("Test 2: Remplissage memoire (8 bytes max)... ");
    init_tag_manager();

    // On ajoute 4 tags de 16 bits (4 * 2 = 8 octets complets)
    printf("\tpreparation du GT\n");
    TEST_EQ_INT("ajout de valeurs 16 bit (1/4)",set_tag("T1", 16), SUCCES_TO_SET);
    TEST_EQ_INT("ajout de valeurs 16 bit (2/4)",set_tag("T2", 16), SUCCES_TO_SET);
    TEST_EQ_INT("ajout de valeurs 16 bit (3/4)",set_tag("T3", 16), SUCCES_TO_SET);
    TEST_EQ_INT("ajout de valeurs 16 bit (4/4)",set_tag("T4", 16), SUCCES_TO_SET);

    // Le prochain tag doit etre refuse faute d'espace libre.
    printf("\ttest de Dépassement de l'espace mémoire (8 octets max)\n");
    TEST_EQ_INT("avec 1 bit",set_tag("T5", 1), CAN_TG_ERROR_NOT_ENOUNG_SPACE);
    TEST_EQ_INT("avec 16 bits",set_tag("T6", 16), CAN_TG_ERROR_NOT_ENOUNG_SPACE);
    printf("OK\n");

    // ---------------------------------------------------------
    // TEST 3 : Cas d'erreurs de set_value et get_value
    // ---------------------------------------------------------
    printf("Test 3: Erreurs et depassements de valeurs... ");
    init_tag_manager();
    init_data_container();
    printf("\tpreparation du GT\n");
    TEST_EQ_INT("prepa... ",set_tag("BIT_1", 1), SUCCES);
    TEST_EQ_INT("prepa... ",set_tag("BIT_3", 3), SUCCES);
    TEST_EQ_INT("prepa... ",set_tag("BIT_4", 4), SUCCES);

    // Tag inexistant (retour attendu: -3)
    printf("\tTag inexistant\n");
    TEST_EQ_INT("test set",set_value("FANTOME", 5), -3);
    TEST_EQ_INT("test get",get_value("FANTOME", &val), -3);

    // Depassement de capacite:
    // NOTE: l'implementation actuelle renvoie CAN_TG_ERROR_VALUE_TO_BIG (-10),
    // mais ces assertions historiques verifient encore -4.
    printf("\tDépassement de capacité\n");
    // - Un tag de 1 bit ne peut contenir que 0 ou 1
    TEST_EQ_INT("Un tag de 1 bits[0 ou 1]",set_value("BIT_1", 2), -4);
    // - Un tag de 3 bits ne peut contenir que de 0 à 7
    TEST_EQ_INT("Un tag de 3 bits [0 à 7]",set_value("BIT_3", 8), -4);
    // - Un tag de 4 bits ne peut contenir que de 0 à 15
    TEST_EQ_INT("Un tag de 4 bits [0 à 15]", set_value("BIT_4", 16), -4);
    printf("OK\n");

    // ---------------------------------------------------------
    // TEST 4 : Indépendance des écritures (Masquage binaire)
    // ---------------------------------------------------------
    printf("Test 4: Indépendance des bits contigus... ");
    init_tag_manager();
    init_data_container();

    // On crée une configuration qui remplit exactement 1 octet (1 + 3 + 4 = 8 bits)
    printf("\tpreparation du GT\n");
    TEST_EQ_INT("preparation [set_tag] (1/3)",set_tag("T_1", 1), SUCCES);
    TEST_EQ_INT("preparation [set_tag] (2/3)",set_tag("T_3", 3), SUCCES);
    TEST_EQ_INT("preparation [set_tag] (3/3)",set_tag("T_4", 4), SUCCES);

    // On assigne les valeurs maximales à chaque tag
    printf("\tpreparation du GT (partie 2)\n");
    TEST_EQ_INT("preparation [set_tag] (1) 1 bits (1/3)",set_value("T_1", 1), SUCCES);
    TEST_EQ_INT("preparation [set_tag] (7) 3 bits (2/3)",set_value("T_3", 7), SUCCES);
    TEST_EQ_INT("preparation [set_tag] (15) 4 bits (3/3)",set_value("T_4", 15), SUCCES);

    // On vérifie qu'elles sont bien mémorisées
    printf("\t\n");
    get_value("T_1", &val);
    TEST_EQ_INT("get value T_1",val, 1);
    get_value("T_3", &val);
```

```c
    TEST_EQ_INT("get value T_3",val, 7);
    get_value("T_4", &val);
    TEST_EQ_INT("get value T_4",val, 15);

    // On modifie T_3 sans toucher aux autres
    printf("\tmodification de T_3 sans toucher aux autres\n");
    TEST_EQ_INT("",set_value("T_3", 2), 1);

    // On vérifie que T_1 et T_4 n'ont pas bougé
    printf("\t\n");
    get_value("T_1", &val);
    TEST_EQ_INT("verif valeur [encien]",val, 1);
    get_value("T_3", &val);
    TEST_EQ_INT("verif valeur [nouvel]",val, 2); // Nouvelle valeur
    get_value("T_4", &val);
    TEST_EQ_INT("verif valeur [encien]",val, 15);
    printf("OK\n");

    // -------------------------------------------------------
    // TEST 5 : Variables de 8 bits
    // -------------------------------------------------------
    printf("Test 5: Variables 8 bits... ");
    init_tag_manager();
    init_data_container();
    printf("\t\n");
    TEST_EQ_INT("prapa de la valeur",set_tag("V8", 8), SUCCES);

    // Test des bornes 0 et 255
    printf("\tTest des bornes 0 et 255\n");
    set_value("V8", 0);
    get_value("V8", &val);
    TEST_EQ_INT("val = 0",val, 0);
    set_value("V8", 255);
    get_value("V8", &val);
    TEST_EQ_INT("val = 255",val, 255);
    printf("\tErreur dépassement\n");
    TEST_EQ_INT("val: [256]",set_value("V8", 256), -4); // Assertion historique, a realigner avec -10
    printf("OK\n");

    //-------------------------------------
    // TEST 6: Tout les test de 4 bits
    //-------------------------------------
    printf("\ttest de tags à 4 bits (tout les type possible)\n");
    init_tag_manager();
    init_data_container();
    printf("test d'inserssion des tags\n");
    TEST_EQ_INT("ajout 1 tags à 4 bits ", set_tag("ma_val1", 4),SUCCES);
    TEST_EQ_INT("ajout 1 tags après 4 bits ", set_tag("ma_2", 4),SUCCES);
    printf("le dernier cas n'ais pas encore testé!!!!\n");
    printf("test  d'ajout de valeurs\n");
    TEST_EQ_INT("set val",set_value("ma_val1", 3),1);
    TEST_EQ_INT("set val 2",set_value("ma_2", 9),1);
    get_value("ma_val1", &val);
    TEST_EQ_INT("get val 1",val,3);;
    get_value("ma_2", &val);
    TEST_EQ_INT("get val 2",val,9);;
    //printf("=== TOUS LES TESTS SONT PASSES AVEC SUCCES ! ===\n\n");
}

int main(void)
{


    // Il suffit d'appeler la fonction de test au début de votre main
    run_all_tests();
    test_print();


    // ... suite de votre code (simulation CAN, etc.) ...
    return 0;
}
```

14.31 data_container.c File Reference

```c
#include "../../include/sub_include/data_container.h"
#include "../../include/config_value.h"
#include "../../include/sub_include/print_val.h"
#include <stdio.h>
```

Include dependency graph for data_container.c:



Functions

- void init_data_container (void)

  Reinitialise le payload TxData a zero.
- void set_val_TxData (uint8_t ∗TxDataArray)

  Copie un tableau externe dans le buffer global TxData.
- uint8_t prep_mask (uint8_t p_bit, uint8_t n_bit)

  [FONCTION INTERNE] Prepare un masque binaire pour set/get.
- CAN_TG_STATUE set_value (const char ∗tag_name, uint32_t value)

  Ecrit une valeur dans TxData a partir d'un tag.
- uint16_t convert_8_to_16bit (uint8_t ba, uint8_t bb)
- int get_value (const char ∗tag_name, uint32_t ∗out_value)

  Lit la valeur associee a un tag.
- void can_simulate_send_receive (uint8_t ∗tx_buffer, uint8_t ∗rx_buffer)

  Simule une transmission CAN locale.

Variables

- uint8_t TxData [N_BYTES]

14.31.1 Function Documentation

14.31.1.1 can_simulate_send_receive()

```
void can_simulate_send_receive (
            uint8_t * tx_buffer,
            uint8_t * rx_buffer )
```

Simule une transmission CAN locale.
Simule un envoi/reception CAN par copie buffer-a-buffer.
Copie simplement tx_buffer dans rx_buffer.

Parameters

tx_buffer	Buffer d'emission.
rx_buffer	Buffer de reception.

Definition at line 165 of file data_container.c.

14.31.1.2 convert_8_to_16bit()

```
uint16_t convert_8_to_16bit (
            uint8_t ba,
            uint8_t bb )
```

Definition at line 100 of file data_container.c.

14.31.1.3 get_value()

```
int get_value (
            const char * tag_name,
            uint32_t * out_value )
```

Lit la valeur associee a un tag.

Parameters

tag_name	Nom du tag a lire.
out_value	Adresse de sortie qui recoit la valeur extraite.

Returns

CAN_TG_SUCCESS en cas de succes, sinon un code d'erreur negatif.

Copie un buffer externe dans TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Recompose un entier 16 bits depuis deux octets.

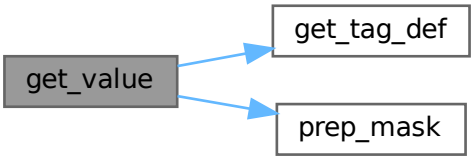
Parameters

ba	Octet de poids faible.
bb	Octet de poids fort.

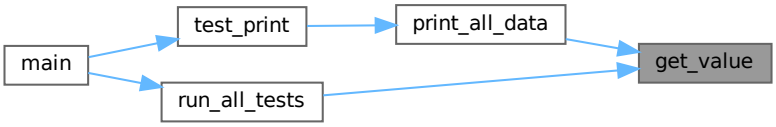
Returns

Valeur 16 bits combinee.

Definition at line 106 of file data_container.c.
Here is the call graph for this function:



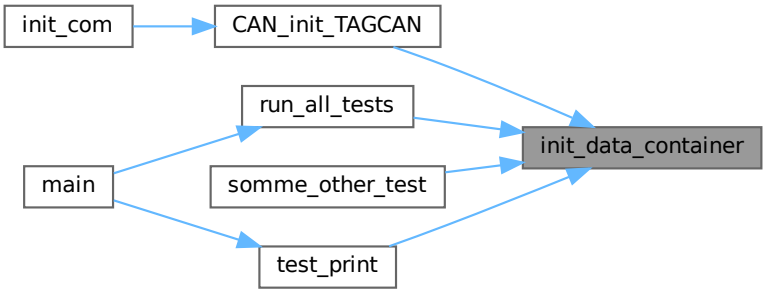
Here is the caller graph for this function:



14.31.1.4 init_data_container()

```
void init_data_container (
            void  )
```

Reinitialise le payload TxData a zero.
Si reexecutee, la fonction efface les donnees precedemment ecrites.
Definition at line 14 of file data_container.c.
Here is the caller graph for this function:



14.31.1.5 prep_mask()

```
uint8_t prep_mask (
            uint8_t p_bit,
            uint8_t n_bit )
```

[FONCTION INTERNE] Prepare un masque binaire pour set/get.
Utilise principalement pour les valeurs strictement inferieures a 8 bits.

Parameters

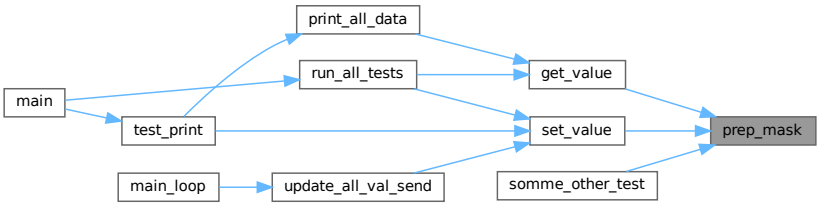
p_bit	Position du premier bit (de gauche a droite).
n_bit	Nombre de bits de la valeur.

Returns

Masque binaire aligne sur la position cible.

Definition at line 29 of file data_container.c.

Here is the caller graph for this function:



14.31.1.6 set_val_TxData()

```
void set_val_TxData (
            uint8_t * TxDataArray )
```

Copie un tableau externe dans le buffer global TxData.

Parameters

TxDataArray	Tableau source de N_BYTES octets.

Definition at line 24 of file data_container.c.

14.31.1.7 set_value()

```
CAN_TG_STATUE set_value (
            const char * tag_name,
            uint32_t value )
```

Ecrit une valeur dans TxData a partir d'un tag.

Warning

La valeur doit tenir dans la taille du tag.

En cas de depassement, l'implementation retourne CAN_TG_ERROR_VALUE_TO_BIG.

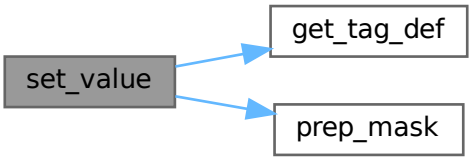
Parameters

tag_name	Nom du tag cible.
value	Nouvelle valeur a ecrire.

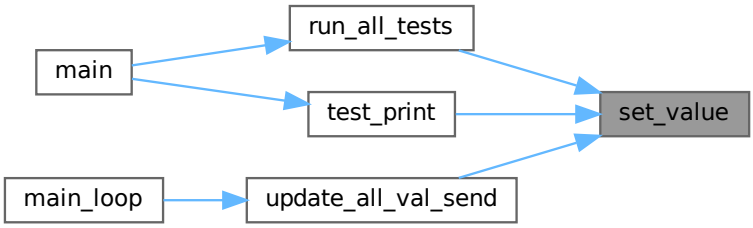
Returns

Code de succes ou code d'erreur.

Definition at line 44 of file data_container.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.31.2 Variable Documentation

14.31.2.1 TxData

`uint8_t TxData[N_BYTES]`
Buffer de donnees CAN (payload de 8 octets).
Definition at line 12 of file data_container.c.

14.32 data_container.c

Go to the documentation of this file.

```
00001 #ifndef DATA_CONTAINER_H
00002 #include "../../include/sub_include/data_container.h"
00003 #endif // DATA_CONTAINER_H
00004 #ifndef CONFIG_VALUE_H
00005 #include "../../include/config_value.h"
00006 #endif // CONFIG_VALUE_H
00007 #ifndef PRINT_VAL_H
00008 #include "../../include/sub_include/print_val.h"
00009 #endif // PRINT_VAL_H
00010 #include <stdio.h>
00011
00012 uint8_t TxData[N_BYTES];
00013
00014 void init_data_container(void)
00015 {
00016     for(int i = 0; i < N_BYTES; i++)
00017     {
```

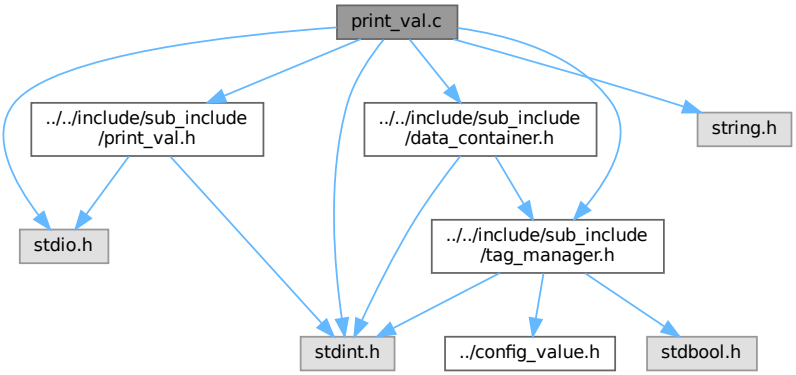
```c
        TxData[i] = 0;
    }
}



void set_val_TxData(uint8_t *TxDataArray){
for (int i = 0; i < N_BYTES; i++){
    TxData[i] = TxDataArray[i];
}
}
uint8_t prep_mask(uint8_t p_bit, uint8_t n_bit)
{
// Preparation du masque
    uint8_t mask_val = 0b1;
    uint8_t mask_temp = 0b1;
    // Construit n_bit bits a 1
    for (int i = 0; i < (n_bit - 1); i++)
    {
        mask_val = mask_val << 1;
        mask_val |= mask_temp;
    }
    // Aligne le masque sur la position cible dans le byte
    return mask_val << (8 - p_bit - n_bit);
}

CAN_TG_STATUE set_value(const char* tag_name, uint32_t value)
{
    const TagDef* tag = get_tag_def(tag_name);
    if (!tag) return -3; // Tag non trouve

    const uint8_t n_bits = tag->n_bits;
    const uint8_t bit_pos_a = tag->bit_pos_a;
    const uint8_t bit_pos_b = tag->bit_pos_b;
    const int8_t byte_idx_a = tag->byte_idx_a;
    const int8_t byte_idx_b = tag->byte_idx_b;

    // Verification du depassement de la valeur selon n_bits
    if (tag->n_bits < 32 && value >= (1ULL << tag->n_bits))
    {
        printf("[ERREUR] Valeur u trop grande pour le tag s (u bits)\n", value, tag_name, tag->n_bits);
        return CAN_TG_ERROR_VALUE_TO_BIG;
    }

    // Cas des valeurs sur plus de 8 bits: repartition entre partie A et partie B.
    if(n_bits > 8){
        uint8_t data_b = (value & 0xFF00) >> 8;
        uint8_t data_a = (value & 0x00FF);
        TxData[byte_idx_a] = data_a;
        // Cas 16 bits: la partie B occupe tout le byte.
        if(n_bits == 16){TxData[tag->byte_idx_b] = data_b;}
        else{
            // Cas 9..15 bits: insertion partielle dans le byte B.
            // Creation du masque binaire
            uint8_t mask = prep_mask(bit_pos_b, n_bits - 8);
            // Alignement de la deuxieme partie
            data_b = data_b << (8 - bit_pos_b - n_bits);
            // Mise a jour selective des bits de la partie B.
            TxData[byte_idx_b] &= ~mask;
            TxData[byte_idx_b] |= data_b;
            return CAN_TG_SUCCESS;
        }
    }
    else if(tag->n_bits == 8)
    {
        uint8_t data_a = (value & 0x00FF);
        TxData[tag->byte_idx_a] = data_a;
        return CAN_TG_SUCCESS;
    }
    else{
        // Valeur <= 8 bits (entre 1 et 7 bits inclusivement)
        // Creation du masque binaire aligne sur la zone cible
        uint8_t mask = prep_mask(bit_pos_a, n_bits);
        uint8_t vrai_val = value;
        // Alignement de la valeur dans le byte
        vrai_val = vrai_val << (8 - bit_pos_a - n_bits);
        TxData[byte_idx_a] &= ~mask;
        TxData[byte_idx_a] |= vrai_val;
    }
    return CAN_TG_SUCCESS;
}

uint16_t convert_8_to_16bit(uint8_t ba, uint8_t bb){
    uint16_t r;
    r = ba | (bb << 8);   // Recompose un entier 16 bits depuis 2 octets.
    return r;
```

```
}

int get_value(const char* tag_name, uint32_t* out_value)
{
    const TagDef* tag = get_tag_def(tag_name);
    if (!tag) return CAN_TG_ERROR_TAG_NOT_FOUND_; // Tag non trouve
    // Copie locale des metadonnees du tag pour simplifier les calculs.
    const uint8_t n_bits = tag->n_bits;
    const uint8_t bpa = tag->bit_pos_a;
    const uint8_t bpb = tag->bit_pos_b;
    const int8_t byta = tag->byte_idx_a;
    const int8_t bytb = tag->byte_idx_b;
    if(tag->byte_idx_b > -1){
            // Declaration conditionnelle conservee pour debug/historique.
            const int8_t byte_indx_b = tag->byte_idx_b;
    }
    if (tag->n_bits == 16)    // cas de 16 bits exactement
    {
        uint8_t ba = TxData[byta];
        uint8_t bb = TxData[bytb];
        *out_value = ba | (bb « 8);   // Recomposition 16 bits complete.
    }
    else if((n_bits < 16) && (n_bits > 8)){// cas d'entre 8 et 16 bits.
        uint8_t ba = TxData[byta];
        uint8_t bb;
        // Creation d'un masque pour extraire la partie B.
        uint8_t mask = prep_mask(bpb, n_bits - 8);
        uint8_t data_temp = TxData[bytb] & mask;
        bb = data_temp » (8 - bpa - n_bits);
        *out_value = ba | (bb « 8);   // Recomposition avec partie B partielle.

    }
    else
    {
        // Creation du masque d'extraction (<= 8 bits)
        uint8_t mask = prep_mask(bpa, n_bits);


        //uint8_t mask = ((1 « n_bits) - 1) « shift;

        uint8_t data_temp = TxData[byta] & mask;
        *out_value = data_temp » (8 - bpa - n_bits);
    }
    return CAN_TG_SUCCESS;
}


void can_simulate_send_receive(uint8_t* tx_buffer, uint8_t* rx_buffer)
{
    for (int i = 0; i < N_BYTES; i++)
    {
        rx_buffer[i] = tx_buffer[i];
    }
}
```

14.33 print_val.c File Reference

```
#include "../../include/sub_include/print_val.h"
#include "../../include/sub_include/tag_manager.h"
#include "../../include/sub_include/data_container.h"
#include <stdio.h>
#include <stdint.h>
#include <string.h>
```

Include dependency graph for print_val.c:



Functions

- void print_all_tag_set (void)

  Affiche la liste complete des tags et leurs metadonnees.
- void print_all_data (void)

  Affiche toutes les valeurs logiques des tags.
- void modif_vt (int8_t bia, int8_t bib, uint8_t bpa, uint8_t bpb, uint8_t nb, char ∗v1, char ∗v2, char ∗v3, char ∗v4, char ∗v5, char ∗v6, char ∗v7, char ∗v8)

  Affiche la repartition des tags dans les bytes de la trame CAN.
- void print_repartition_bit (void)

  Affiche une vue textuelle de l'occupation des bits de la trame.
- void print_bin_8 (uint8_t val)

  Prints the binary representation of an 8-bit unsigned integer.
- void print_all_data_bin (void)

  Prints the binary representation of each byte in `TxData`.
- void print_all_with_title (const char ∗title)

  Affiche TxData en binaire avec un titre.

Variables

- tsfp gbc2 [9]

14.33.1 Function Documentation

14.33.1.1 modif_vt()

```
void modif_vt (
            int8_t bia,
            int8_t bib,
            uint8_t bpa,
            uint8_t bpb,
            uint8_t nb,
            char * v1,
            char * v2,
            char * v3,
            char * v4,
            char * v5,
```

```
                char * v6,
                char * v7,
                char * v8 )
```

Affiche la repartition des tags dans les bytes de la trame CAN.
[INTERNE] Point d'extension pour la construction de vues de repartition.

Warning

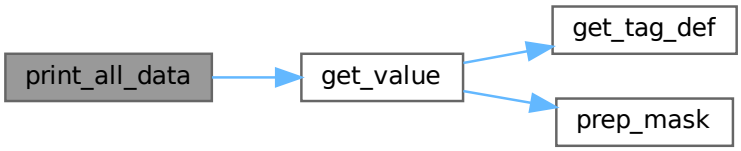
Fonction actuellement inachevee/conservee pour evolution future.

Definition at line 66 of file print_val.c.

14.33.1.2 print_all_data()

```
void print_all_data (
                void  )
```

Affiche toutes les valeurs logiques des tags.
Affiche toutes les valeurs logiques relues via get_value.
Chaque valeur est relue via get_value puis imprimee.
Definition at line 43 of file print_val.c.
Here is the call graph for this function:



Here is the caller graph for this function:

14.33.1.3 print_all_data_bin()

```
void print_all_data_bin (
                void  )
```

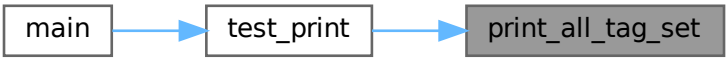
Prints the binary representation of each byte in `TxData`.
Affiche tout le payload TxData en binaire.
Prints the binary representation of each byte in the `TxData` array.
Definition at line 150 of file print_val.c.

Here is the call graph for this function:

14.33.1.4 print_all_tag_set()

```
void print_all_tag_set (
            void  )
```

Affiche la liste complete des tags et leurs metadonnees.
Affiche la table des tags declares et leurs metadonnees.
Definition at line 18 of file print_val.c.
Here is the caller graph for this function:



14.33.1.5 print_all_with_title()

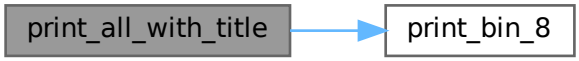
```
void print_all_with_title (
            const char * title )
```

Affiche TxData en binaire avec un titre.
Affiche TxData en binaire avec un titre en prefixe.

Parameters

title	Libelle affiche avant les octets.

Definition at line 164 of file print_val.c.
Here is the call graph for this function:



14.33.1.6 print_bin_8()

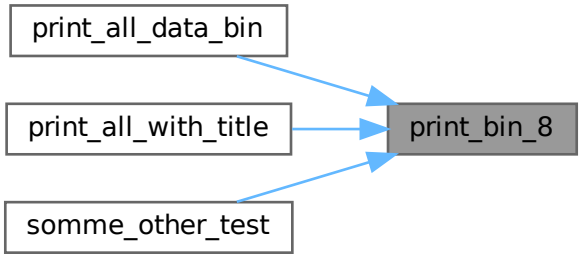
```
void print_bin_8 (
            uint8_t val )
```

Prints the binary representation of an 8-bit unsigned integer.
Affiche la representation binaire (8 bits) d'une valeur.

Parameters

val	The `val` [from 0 to 255]. The function prints the binary representation of this value by iterating over each bit.

Definition at line 136 of file print_val.c.
Here is the caller graph for this function:



14.33.1.7 print_repartition_bit()

```
void print_repartition_bit (
            void )
```

Affiche une vue textuelle de l'occupation des bits de la trame.
Affiche la repartition des bits utilises dans la trame CAN.
Definition at line 78 of file print_val.c.
Here is the caller graph for this function:

14.33.2 Variable Documentation

14.33.2.1 gbc2

```
tsfp gbc2[9]
```

Espace global de rendu utilise par les fonctions d'impression de repartition.
Definition at line 14 of file print_val.c.

14.34 print_val.c

Go to the documentation of this file.

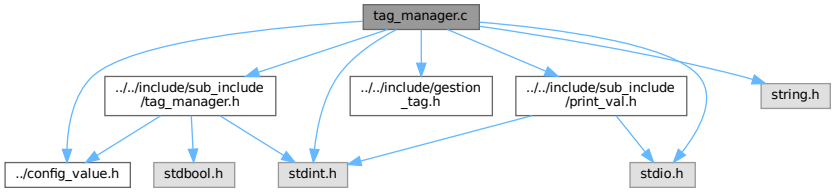
```
#ifndef PRINT_VAL_H
#include "../../include/sub_include/print_val.h"
#endif // PRINT_VAL_H
#ifndef TAG_MANAGER_H
#include "../../include/sub_include/tag_manager.h"
#endif // TAG_MANAGER_H
#ifndef DATA_CONTAINER_H
#include "../../include/sub_include/data_container.h"
#endif // DATA_CONTAINER_H
#include <stdio.h>
#include <stdint.h>
#include <string.h>

tsfp gbc2[9];

void print_all_tag_set(void){
    printf("-----------------------------------------------\n");
    printf("legende\n\tnb:\tn_bits\t (le nombre de bites attribuer  la valeurs)\n");
    printf("\tbia:\tbyte_idx_a\t (le numro du bytes de la partie A de la valeur \n");
    printf("\tbib:\tbyte_idx_b\t (le numro de bytes de la partie B de la valeur (-1: pas de partie B))\n");
    printf("\tbpa:\tbit_pos_a (La position du premier bits dans le byte en question de la partie A.)\n");
    printf("\tbpb:\tbit_pos_b (idem, mais pour la partie B.\n");
    printf("- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -\n");
    printf("\tid\tname\tnb\tbia\tbib\tbpa\tbpb\n");
    for(int i = 0; i <= g_tag_count - 1; i++){
        printf("\t%d\t",i);
        printf("%s\t",g_tags[i].name);
        printf("%d\t",g_tags[i].n_bits);
        printf("%d\t",g_tags[i].byte_idx_a);
        printf("%d\t",g_tags[i].byte_idx_b);
        printf("%d\t",g_tags[i].bit_pos_a);
        printf("%d\t",g_tags[i].bit_pos_b);
        printf("\n");
    }
    printf("-------------------------------------------------------------------\n");
}

void print_all_data(void){
    printf("---------------------[view of all data]----------------------------\n");
    printf("format: tag[n.bits]:\tvaleur\n");
    int32_t result_val;
    for(int i = 0; i <= g_tag_count - 1; i++){
        printf("\t%d",i);
        printf("%s\t",g_tags[i].name);
        get_value(g_tags[i].name,&result_val);
        printf("[%d]:\t",g_tags[i].n_bits);
        printf("%d", result_val);
        printf("\n");
    }
    printf("-------------------[END view of all data]---------------------------\n");
}

void modif_vt(int8_t bia, int8_t bib, uint8_t bpa, uint8_t bpb, uint8_t nb,
                char* v1,char* v2,char* v3,char* v4,char* v5,char* v6,char* v7,char* v8 ){
                if(nb>8){

                }

                }



void print_repartition_bit(void){
    // Grille de remplissage: bit_grid[byte_idx][bit_pos] = caractere du tag
    char bit_grid[N_BYTES][8];
    memset(bit_grid, '.', sizeof(bit_grid));

    // Affichage de la legende
    printf("=== REPARTITION DES BITS DANS LA TRAME CAN ===\n\n");
    printf("Legende des tags:\n");
    char possible_id[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

    // Remplir la grille avec les tags
    for(int i = 0; i <= g_tag_count - 1; i++){
        printf("  %s : [%c]\n", g_tags[i].name, possible_id[i]);

        if(g_tags[i].n_bits > 8){
            // Le tag s'etend sur deux bytes
            // Partie A: de byte_idx_a, bits bit_pos_a a 7
            for(int j = g_tags[i].bit_pos_a; j < 8; j++){
                bit_grid[g_tags[i].byte_idx_a][j] = possible_id[i];
```

```
            }
            // Partie B: de byte_idx_b, bits 0 a (remaining_bits - 1)
            int remaining_bits = g_tags[i].n_bits - (8 - g_tags[i].bit_pos_a);
            for(int j = 0; j < remaining_bits && j < 8; j++){
                bit_grid[g_tags[i].byte_idx_b][j] = possible_id[i];
            }
        }
        else {
            // Le tag tient dans un seul byte: de bit_pos_a a (bit_pos_a + n_bits - 1)
            for(int j = 0; j < g_tags[i].n_bits; j++){
                if(g_tags[i].bit_pos_a + j < 8){
                    bit_grid[g_tags[i].byte_idx_a][g_tags[i].bit_pos_a + j] = possible_id[i];
                }
            }
        }
    }

    // Affichage de la grille
    printf("\n");
    printf("========================================\n");
    printf("Byte | Bit 7 6 5 4 3 2 1 0\n");
    printf("-----|---------+--------\n");

    for(int byte = 0; byte < N_BYTES; byte++){
        printf(" [%d] | ", byte);
        for(int bit = 7; bit >= 0; bit--){
            printf("%c ", bit_grid[byte][bit]);
        }
        printf("\n");
    }
    printf("========================================\n");
}

void print_bin_8(uint8_t val)
{
    for(int i = 7; i >= 0 ; i--)
    {
        printf("%c", (val & (1 « i)) ? '1' : '0');
    }
}

void print_all_data_bin(void)
{
    printf("DATA: \t");
    for (int i = 0; i < N_BYTES; i++)
    {
        print_bin_8(TxData[i]);
        printf("\t");
    }
    printf("\n");
}

void print_all_with_title(const char* title){
    printf("%s\t",title);
    int i;
    for (int i = 0; i < N_BYTES; i++)
    {
        print_bin_8(TxData[i]);
        printf("\t");
    }
    printf("\n");
}
```

14.35 tag_manager.c File Reference

```
#include "../../include/sub_include/tag_manager.h"
#include "../../include/config_value.h"
#include "../../include/gestion_tag.h"
#include "../../include/sub_include/print_val.h"
#include <stdio.h>
#include <string.h>
#include <stdint.h>
```

Include dependency graph for tag_manager.c:



Functions

- void init_tag_manager (void)

  Initialise/reinitialise le gestionnaire de tags.
- int get_first_tags_vide (void)

  [INTERNE] Retourne l'index du premier slot tag libre.
- void set_name_tag (const int n_tag, const char ∗tag_name)

  [INTERNE] Enregistre un nom de tag dans un slot.
- void general_set_tag (const int n_tag, const char ∗tag_name, uint8_t n_bits, int8_t byte_idx_a)

  [INTERNE] Initialise les champs communs d'un tag.
- void set_new_tag_9_to_16 (const char ∗tag_name, const int8_t byte_idx_a, const int8_t byte_idx_b, uint8_t n_bits_total)

  [INTERNE] Cree un tag avec repartition potentielle sur 2 bytes.
- CAN_TG_STATUE find_byte_with_enoung_space (uint8_t siz, int8_t ∗retur_var)

  Trouve le premier byte ayant assez de place libre.
- CAN_TG_STATUE find_byte2_with_enoung_space (uint8_t siz_total, int8_t posi_premier_bit, int8_t ∗retur↩ _var)

  Variante de recherche en excluant le byte deja utilise par la partie A.
- CAN_TG_STATUE set_tag (const char ∗tag_name, uint8_t siz)

  Declare un nouveau tag et reserve son espace dans TxData.
- const TagDef ∗ get_tag_def (const char ∗tag_name)

  Recherche un tag actif par son nom.

Variables

- TagDef g_tags [MAX_TAGS]
- uint8_t g_tag_count = 0
- uint8_t g_bytes_used [N_BYTES] = {0}

14.35.1 Function Documentation

14.35.1.1 find_byte2_with_enoung_space()

```
CAN_TG_STATUE find_byte2_with_enoung_space (
            uint8_t siz_total,
            int8_t posi_premier_bit,
            int8_t * retur_var )
```

Variante de recherche en excluant le byte deja utilise par la partie A.

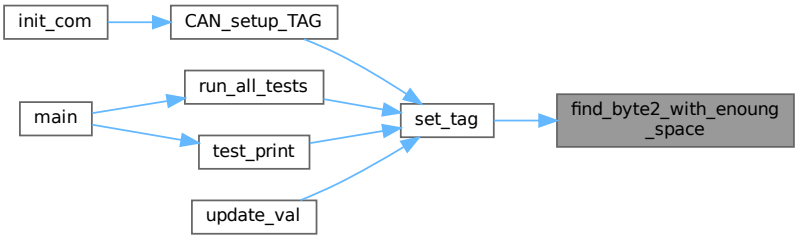
Parameters

siz_total	Taille totale (9..16).
posi_premier_bit	Byte deja retenu pour la partie A.
retur_var	Sortie: index du byte pour la partie B.

Returns

CAN_TG_SUCCESS ou CAN_TG_ERROR_NOT_ENOUNG_SPACE_.

Definition at line 188 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.2 find_byte_with_enoung_space()

```
CAN_TG_STATUE find_byte_with_enoung_space (
            uint8_t siz,
            int8_t * retur_var )
```

Trouve le premier byte ayant assez de place libre.

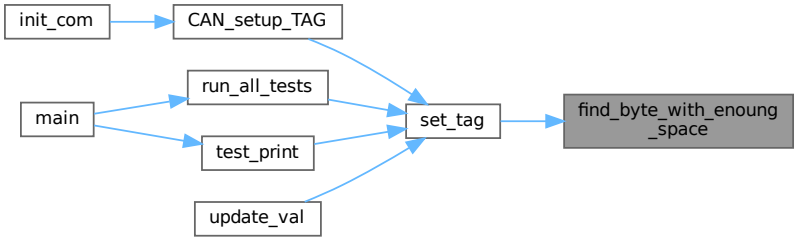
Parameters

siz	Taille demandee (en bits).
retur_var	Sortie: index du byte selectionne.

Returns

CAN_TG_SUCCESS ou CAN_TG_ERROR_NOT_ENOUNG_SPACE_.

Definition at line 144 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.3 general_set_tag()

```
void general_set_tag (
            const int n_tag,
```

```
const char * tag_name,
uint8_t n_bits,
int8_t byte_idx_a )
```

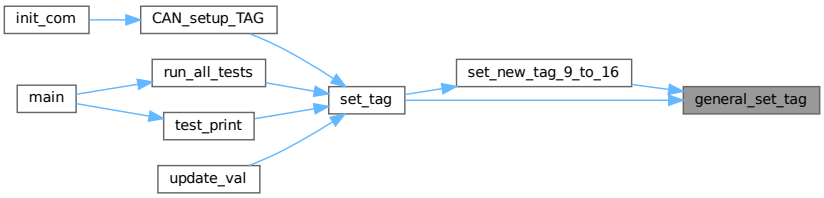
[INTERNE] Initialise les champs communs d'un tag.

Parameters

n_tag	Index du slot tag cible.
tag_name	Nom logique a enregistrer.
n_bits	Taille de la valeur en bits.
byte_idx↩ _a	Byte principal de stockage (partie A).

Definition at line 90 of file tag_manager.c.
Here is the call graph for this function:

Here is the caller graph for this function:



14.35.1.4 get_first_tags_vide()

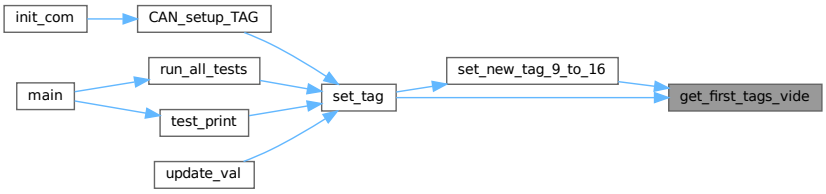
```
int get_first_tags_vide (
            void  )
```

[INTERNE] Retourne l'index du premier slot tag libre.
Un slot est considere libre si son nom est NULL ou vide ('\0').

Returns

Index du slot libre, ou CAN_TG_ERROR_TOO_MAY_TAGS si aucun slot disponible.

Definition at line 42 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.5 get_tag_def()

```
const TagDef * get_tag_def (
            const char * tag_name )
```

Recherche un tag actif par son nom.
Recherche un tag et retourne sa definition.

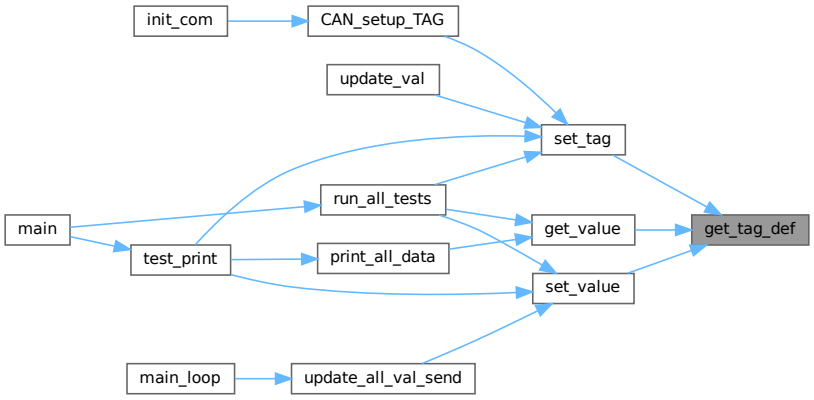
Parameters

tag_name	Nom du tag recherche.

Returns

Pointeur sur TagDef si trouve, sinon NULL.

Definition at line 288 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.6 init_tag_manager()

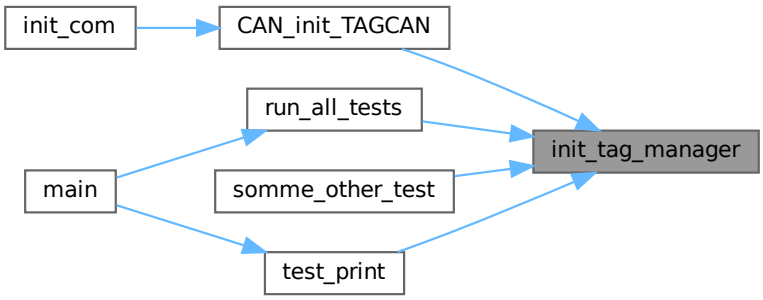
```
void init_tag_manager (
            void )
```

Initialise/reinitialise le gestionnaire de tags.

Note

Apres cet appel, l'ensemble des tags doit etre redefini via `set_tag`.

Definition at line 21 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.7 set_name_tag()

```
void set_name_tag (
            const int n_tag,
            const char * tag_name )
```

[INTERNE] Enregistre un nom de tag dans un slot.

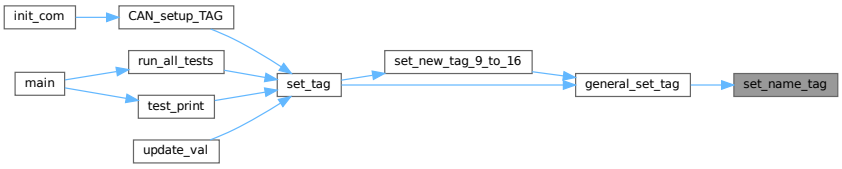
Warning

Les verifications d'entree sont censees etre faites en amont.

Parameters

n_tag	Index du slot (0..MAX_TAGS-1).
tag_name	Nom logique du tag.

Definition at line 68 of file tag_manager.c.
Here is the caller graph for this function:



14.35.1.8 set_new_tag_9_to_16()

```
void set_new_tag_9_to_16 (
```

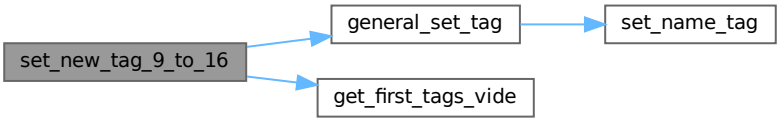
```
const char * tag_name,
const int8_t byte_idx_a,
const int8_t byte_idx_b,
uint8_t n_bits_total )
```

[INTERNE] Cree un tag avec repartition potentielle sur 2 bytes.

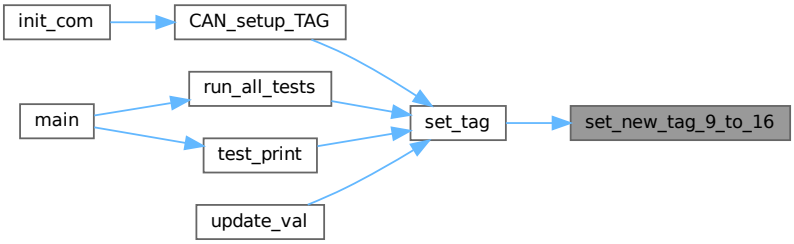
Parameters

tag_name	Nom logique du tag.
byte_idx↩ _a	Byte de la partie A.
byte_idx↩ _b	Byte de la partie B.
n_bits_total	Taille totale en bits.

Definition at line 119 of file tag_manager.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.35.1.9 set_tag()

```
CAN_TG_STATUE set_tag (
    const char * tag_name,
    uint8_t siz )
```

Declare un nouveau tag et reserve son espace dans TxData.
créé un nouveau tag.

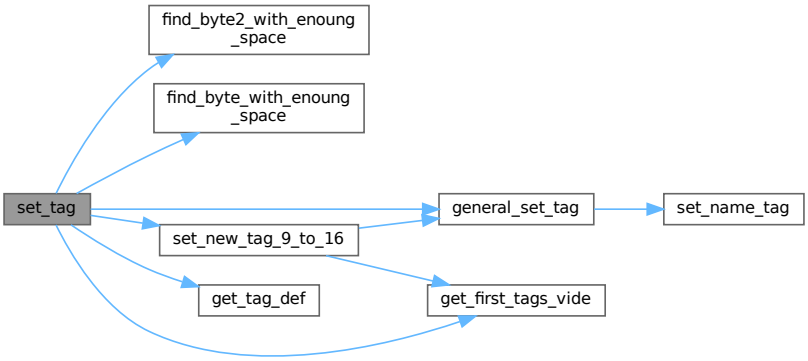
Parameters

tag_name	Nom logique du tag.
siz	Taille de la valeur en bits (implementation actuelle: 1..16).

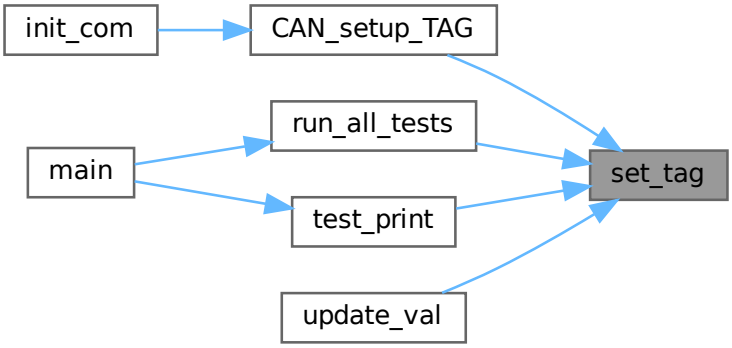
Returns

Code de succes ou code d'erreur CAN_TG_STATUE.

Definition at line 232 of file tag_manager.c.
Here is the call graph for this function:



Here is the caller graph for this function:



14.35.2 Variable Documentation

14.35.2.1 g_bytes_used

```
uint8_t g_bytes_used[N_BYTES] = {0}
```

Occupation courante en bits pour chaque byte du payload.
Definition at line 19 of file tag_manager.c.

14.35.2.2 g_tag_count

```
uint8_t g_tag_count = 0
```

Nombre de tags actifs (max: MAX_TAGS).
Definition at line 18 of file tag_manager.c.

14.35.2.3 g_tags

```
TagDef g_tags[MAX_TAGS]
```

Table des tags (actifs et inactifs).
Definition at line 17 of file tag_manager.c.

14.36 tag_manager.c

Go to the documentation of this file.

```
00001
00002 #include "../../include/sub_include/tag_manager.h"
00003 #ifndef CONFIG_VALUE_H
00004 #include "../../include/config_value.h"
00005 #endif // CONFIG_VALUE_H
00006 #ifndef GESTION_TAG_H
00007 #include "../../include/gestion_tag.h"
00008 #endif // GESTION_TAG_H
00009 #ifndef PRINT_VAL_H
00010 #include "../../include/sub_include/print_val.h"
00011 #endif // PRINT_VAL_H
00012 #include <stdio.h>
00013 #include <string.h>
00014 #include <stdint.h>
00015
00016 // Etat global du gestionnaire de tags
00017 TagDef g_tags[MAX_TAGS];
00018 uint8_t g_tag_count = 0;
00019 uint8_t g_bytes_used[N_BYTES] = {0};
00020
00021 void init_tag_manager(void)
00022 {
00023     g_tag_count = 0;
00024     // Remise a zero de l'occupation des bytes.
00025     memset(g_bytes_used, 0, sizeof(g_bytes_used));
00026     // Reinitialisation complete de tous les slots de tags.
00027     for (int i = 0; i < MAX_TAGS; i++)
00028     {
00029         g_tags[i].name[0] = '\0';   // Slot considere vide quand name[0] == '\0'.
00030         g_tags[i].n_bits = 0;
00031         g_tags[i].byte_idx_a = -1;
00032         g_tags[i].byte_idx_b = -1;
00033         g_tags[i].bit_pos_a = 0;
00034         g_tags[i].bit_pos_b = 0;
00035     }
00036 }
00037
00042 int get_first_tags_vide(void)
00043 {
00044     for (int i = 0; i < MAX_TAGS; i++)
00045     {
00046         // Cas defensif: un nom NULL est traite comme slot libre.
00047         if(g_tags[i].name == NULL)
00048         {
00049             return i;
00050         }
00051         // Premier slot libre: nom vide.
00052         if(g_tags[i].name[0] == '\0')
00053         {
00054             return i;
00055         }
00056     }
00057     // Cas de garde: aucun slot libre detecte.
00058     return CAN_TG_ERROR_TOO_MAY_TAGS;
00059 }
00060
00068 void set_name_tag(const int n_tag, const char* tag_name)
00069 {
00070     // Nettoie le buffer avant copie du nom.
00071     for (int i = 0; i < MAX_TAG_NAME_LEN - 2; i++)
00072     {
00073         g_tags[n_tag].name[i]=' ';
00074     }
00075     // Copie du nom dans la structure.
00076     strncpy(g_tags[n_tag].name, tag_name, MAX_TAG_NAME_LEN - 1);
00077     // Termine explicitement la chaine.
00078     g_tags[n_tag].name[MAX_TAG_NAME_LEN - 1] = '\0';
00079     // Debug optionnel:
00080     //printf("le nom du tag est: [%s]\n", g_tags[n_tag].name);
00081 }
00082
00083 // Initialise les champs communs d'un tag et met a jour l'occupation memoire.
00090 void general_set_tag(const int n_tag,const char* tag_name,
00091                      uint8_t n_bits, int8_t byte_idx_a)
```

```
{
    set_name_tag(n_tag, tag_name);
    g_tags[n_tag].n_bits = n_bits;

    g_tags[n_tag].byte_idx_a = byte_idx_a;
    // Le tag est desormais actif.
    g_tag_count++;

    // Mise a jour de l'occupation des bits dans le byte principal.
    if(n_bits >= 8)
    {
        g_bytes_used[byte_idx_a] = 8;
    }
    else
    {
        g_bytes_used[byte_idx_a] += n_bits;
    }
}

// [INTERNE] Creation d'un tag sur 9..16 bits (2 bytes potentiels).
// n_bits_total: taille totale de la valeur (8..16).
void set_new_tag_9_to_16(const char* tag_name, const int8_t byte_idx_a, const int8_t byte_idx_b,
uint8_t n_bits_total)
{
    // Recherche d'un slot libre.
    int n_tag = get_first_tags_vide();
    general_set_tag(n_tag, tag_name, n_bits_total, byte_idx_a);    // Initialisation commune.
    // Pour > 8 bits, une partie B est utilisee.
    if(n_bits_total > 8)
    {

        g_tags[n_tag].byte_idx_b = byte_idx_b;
        g_tags[n_tag].bit_pos_b = g_bytes_used[byte_idx_b];
        g_bytes_used[byte_idx_b] += n_bits_total - 8;
        g_tags[n_tag].bit_pos_a = 0;
        /* A ce stade, la repartition est validee: on peut finaliser
           l'occupation et les positions des bits pour A et B. */
    }
}

CAN_TG_STATUE find_byte_with_enoung_space(uint8_t siz, int8_t* retur_var)
{
    // Pour > 8 bits, la 1re recherche se limite a un bloc de 8 bits.
    if(siz > 8)
    {
        siz = 8;
    }
    // Pour les tailles impaires: priorite aux bytes a place impaire restante.
    if(siz%2 > 0)
    {
        // Heuristique de placement.
        for (int8_t i = 0; i < N_BYTES; i++)
        {
            {
                if(((8 - g_bytes_used[i])%2 > 0) && ((8 - g_bytes_used[i]) >= siz))
                {
                    *retur_var=i;
                    return CAN_TG_SUCCESS;
                }
            }
        }
    }
    // Sinon, recherche sequentielle standard.
    for (int8_t i = 0; i < N_BYTES; i++)
    {
        if((8 - g_bytes_used[i]) >= siz)
        {
            *retur_var=i;
            return CAN_TG_SUCCESS;
        }
    }
    return CAN_TG_ERROR_NOT_ENOUNG_SPACE_;
}


CAN_TG_STATUE find_byte2_with_enoung_space(uint8_t siz_total, int8_t posi_premier_bit,int8_t*
retur_var)
{
    uint8_t siz = siz_total - 8;
    // Meme heuristique que find_byte_with_enoung_space.
    if(siz%2 > 0)
    {
        // Priorite aux bytes dont l'espace restant est impair.
        for (int8_t i = 0; i < N_BYTES; i++)
        {
            if(posi_premier_bit == i)
            {
```

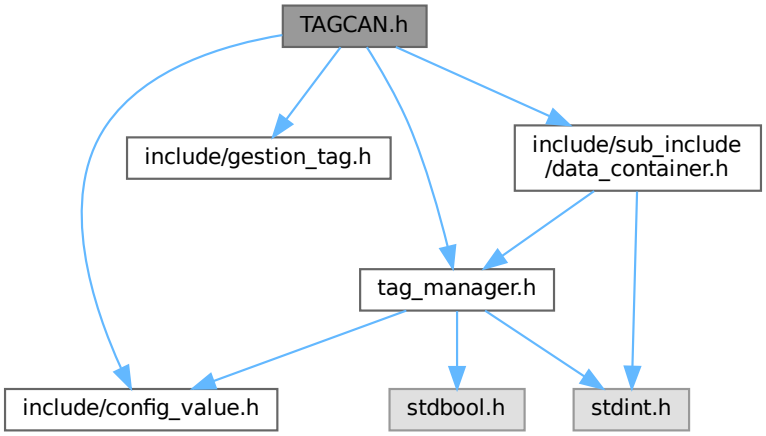
```
                continue;
            }
            if(((8 - g_bytes_used[i])%2 > 0) && ((8 - g_bytes_used[i]) >= siz))
            {
                *retur_var=i;
                return CAN_TG_SUCCESS;
            }
        }
    }
    // Sinon, recherche sequentielle standard.
    for (uint8_t i = 0; i < N_BYTES; i++)
    {
        if(posi_premier_bit == i)
        {
            continue;
        }
        if((8 - g_bytes_used[i]) >= siz)
        {
            *retur_var=i;
            return CAN_TG_SUCCESS;
        }
    }
    return CAN_TG_ERROR_NOT_ENOUNG_SPACE_;
}



// API publique: declaration de tag.
CAN_TG_STATUE set_tag(const char* tag_name, uint8_t siz){
    CAN_TG_STATUE statue_actuel;
    // Limite du nombre de tags.
    if(g_tag_count >= MAX_TAGS){
        return CAN_TG_ERROR_TOO_MAY_TAGS;
    }
    // Validation longueur nom (implementation actuelle basee sur sizeof(pointer)).
    if(sizeof(tag_name) >= MAX_TAG_NAME_LEN){
        return CAN_TG_ERROR_TAG_NAME_TO_BIGS;
    }
    // Interdit les doublons de nom.
    if(get_tag_def(tag_name) != NULL){
        return CAN_TG_ERROR_NAME_ALREADY_USE;
    }
    // Nom vide invalide.
    if(tag_name[0]=='\0'){
        return CAN_TG_ERROR_TAG_NAME_INVALIDE;
    }
    // Tailles acceptees: 1..16.
    if(siz <= 0 || siz > 16){return CAN_TG_ERROR_SIZE_VALUE_INVALIDE;}

    //------------------------- algo de placement memoire.

    if(siz>8){
        // Garde defensive (deja couverte plus haut).
        if(siz>16){
            return CAN_TG_ERROR_SIZE_VALUE_INVALIDE_;
        }
        else{
            uint8_t bytea, byteb;    // Index des bytes reserves.
            statue_actuel = find_byte_with_enoung_space(siz, &bytea);
            if(statue_actuel != CAN_TG_SUCCESS){return statue_actuel;}// propagation du statut
            statue_actuel = find_byte2_with_enoung_space(siz, bytea, &byteb);
            if(statue_actuel != CAN_TG_SUCCESS){
                return statue_actuel;   // propagation du statut
            }
            set_new_tag_9_to_16(tag_name, bytea, byteb, siz);
            return CAN_TG_SUCCESS;
        }
    }
    else{
        uint8_t bytea;
        statue_actuel = find_byte_with_enoung_space(siz, &bytea);
        if(statue_actuel != CAN_TG_SUCCESS){return statue_actuel;}// propagation du statut
        int n_tag = get_first_tags_vide();
        general_set_tag(n_tag, tag_name, siz, bytea);
        return CAN_TG_SUCCESS;
    }


};

const TagDef* get_tag_def(const char* tag_name)
{
    for (int i = 0; i < g_tag_count; i++)
    {
        if (strncmp(g_tags[i].name, tag_name, MAX_TAG_NAME_LEN) == 0)
        {
            return &g_tags[i];
```

```
        }
    }
    return NULL; // Tag non trouve
};

```

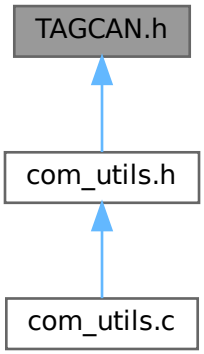
14.37 TAGCAN.h File Reference

```
#include "include/config_value.h"
#include "include/gestion_tag.h"
#include "include/sub_include/data_container.h"
#include "include/sub_include/tag_manager.h"
```

Include dependency graph for TAGCAN.h:



This graph shows which files directly or indirectly include this file:



14.38 TAGCAN.h

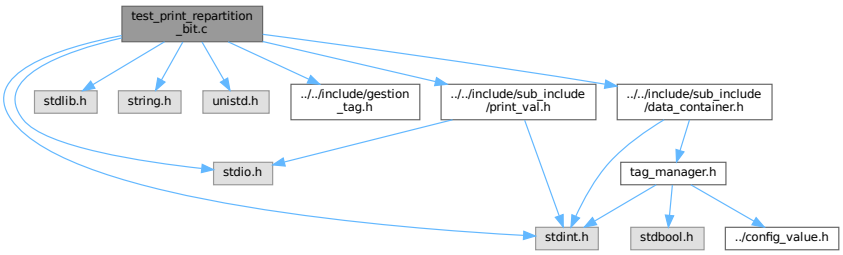
Go to the documentation of this file.

```
#ifndef TAGCAN_H
#define TAGCAN_H

#include "include/config_value.h"
#include "include/gestion_tag.h"
#include "include/sub_include/data_container.h"
#include "include/sub_include/tag_manager.h"

#endif /* TAGCAN_H */
```

14.39 test_print_repartition_bit.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "../../include/gestion_tag.h"
#include "../../include/sub_include/data_container.h"
#include "../../include/sub_include/print_val.h"
```

Include dependency graph for test_print_repartition_bit.c:



Functions

- int main__ (void)

14.39.1 Function Documentation

14.39.1.1 main__()

```
int main__ (
            void  )
```

Definition at line 79 of file test_print_repartition_bit.c.

14.40 test_print_repartition_bit.c

Go to the documentation of this file.

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "../../include/gestion_tag.h"
#include "../../include/sub_include/data_container.h"
#include "../../include/sub_include/print_val.h"

static int contains_text(const char* haystack, const char* needle)
{
```

```c
    return strstr(haystack, needle) != NULL;
}

static int capture_print_repartition_output(char* out_buf, size_t out_size)
{
    int pipefd[2];
    if (pipe(pipefd) != 0) {
        return -1;
    }

    fflush(stdout);
    int saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout < 0) {
        close(pipefd[0]);
        close(pipefd[1]);
        return -1;
    }

    if (dup2(pipefd[1], STDOUT_FILENO) < 0) {
        close(saved_stdout);
        close(pipefd[0]);
        close(pipefd[1]);
        return -1;
    }

    close(pipefd[1]);

    init_data_container();
    init_tag_manager();

    if (set_tag("a1", 2) <= 0 ||
        set_tag("agh5", 5) <= 0 ||
        set_tag("patap", 16) <= 0 ||
        set_tag("patat", 12) <= 0 ||
        set_tag("uni1", 1) <= 0 ||
        set_tag("triot", 3) <= 0) {
        dup2(saved_stdout, STDOUT_FILENO);
        close(saved_stdout);
        close(pipefd[0]);
        return -1;
    }

    print_repartition_bit();
    fflush(stdout);

    if (dup2(saved_stdout, STDOUT_FILENO) < 0) {
        close(saved_stdout);
        close(pipefd[0]);
        return -1;
    }
    close(saved_stdout);

    size_t total = 0;
    while (total + 1 < out_size) {
        ssize_t n = read(pipefd[0], out_buf + total, out_size - total - 1);
        if (n <= 0) {
            break;
        }
        total += (size_t)n;
    }
    close(pipefd[0]);
    out_buf[total] = '\0';

    return (int)total;
}

int main__(void)
{
    char output[8192];
    int n = capture_print_repartition_output(output, sizeof(output));
    if (n <= 0) {
        fprintf(stderr, "[FAIL] capture output failed\n");
        return 1;
    }

    if (!contains_text(output, "=== REPARTITION DES BITS DANS LA TRAME CAN ===")) {
        fprintf(stderr, "[FAIL] header missing\n");
        return 1;
    }

    if (!contains_text(output, "Legende des tags:")) {
        fprintf(stderr, "[FAIL] legend title missing\n");
        return 1;
    }

    if (!contains_text(output, "a1 : [a]") ||
        !contains_text(output, "agh5 : [b]") ||
```

```
        !contains_text(output, "patap : [c]") ||
        !contains_text(output, "patat : [d]") ||
        !contains_text(output, "uni1 : [e]") ||
        !contains_text(output, "triot : [f]")) {
        fprintf(stderr, "[FAIL] legend entries missing\n");
        return 1;
    }

    for (int i = 0; i < N_BYTES; i++) {
        char pattern[32];
        snprintf(pattern, sizeof(pattern), " [%d] | ", i);
        if (!contains_text(output, pattern)) {
            fprintf(stderr, "[FAIL] missing byte row: %d\n", i);
            return 1;
        }
    }

    printf("[PASS] print_repartition_bit: structure de sortie valide\n");
    return 0;
}
```

14.41 test_safty_func.c File Reference

14.42 test_safty_func.c

Go to the documentation of this file.

14.43 test_safty_func.h File Reference

14.44 test_safty_func.h

Go to the documentation of this file.

```
#ifndef BBEED0ED_B866_49C5_823A_7605CFEB774A
#define BBEED0ED_B866_49C5_823A_7605CFEB774A


#endif /* BBEED0ED_B866_49C5_823A_7605CFEB774A */
```

14.45 test_send_receve.c File Reference

14.46 test_send_receve.c

Go to the documentation of this file.

14.47 test_send_receve.h File Reference

14.48 test_send_receve.h

Go to the documentation of this file.

```
#ifndef E1ED0999_BD58_4931_8E4F_33495368ECCC
#define E1ED0999_BD58_4931_8E4F_33495368ECCC


#endif /* E1ED0999_BD58_4931_8E4F_33495368ECCC */
```

14.49 test_set_tag.c File Reference

14.50 test_set_tag.c

Go to the documentation of this file.

14.51 test_set_tag.h File Reference

14.52 test_set_tag.h

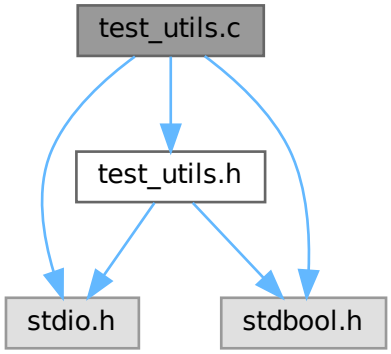
Go to the documentation of this file.

```
#ifndef B7B65D13_8EFA_4C35_97AB_B8F4F6313735
#define B7B65D13_8EFA_4C35_97AB_B8F4F6313735


#endif /* B7B65D13_8EFA_4C35_97AB_B8F4F6313735 */
```

14.53 test_utils.c File Reference

```
#include "test_utils.h"
#include <stdbool.h>
#include <stdio.h>
```

Include dependency graph for test_utils.c:



Functions

- void TEST_ (const char *name, const bool condition)
- void TEST_EQ_INT_ (const char *name, const int val_1, const int val_2)

Variables

- int total_tests
- int passed_tests

14.53.1 Function Documentation

14.53.1.1 TEST_()

```
void TEST_ (
            const char * name,
            const bool condition )
```

Definition at line 7 of file test_utils.c.

14.53.1.2 TEST_EQ_INT_()

```
void TEST_EQ_INT_ (
            const char * name,
```

```
            const int val_1,
            const int val_2 )
```

Definition at line 11 of file test_utils.c.

14.53.2 Variable Documentation

14.53.2.1 passed_tests

```
int passed_tests [extern]
```

le nombre total de test réussi

Definition at line 17 of file main.c.

14.53.2.2 total_tests

```
int total_tests [extern]
```

le nombre total de test effectué

Definition at line 18 of file main.c.

14.54 test_utils.c

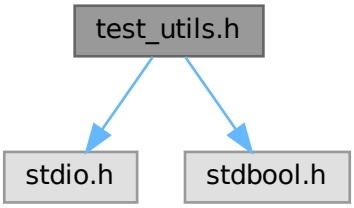
Go to the documentation of this file.

```
00001 #include "test_utils.h"
00002 #include <stdbool.h>
00003 #include <stdio.h>
00004 extern int total_tests;
00005 extern int passed_tests;
00006
00007 void TEST_(const char* name, const bool condition){
00008     if(condition){printf("\tsuccess\t%s\n", name);}
00009     else{printf("\tfail\t%s\n", name);}
00010 }
00011 void TEST_EQ_INT_(const char* name, const int val_1, const int val_2){
00012     bool condition = val_1 == val_2;
00013     if(condition){
00014         printf("\tsuccess\t%s\n", name);
00015     passed_tests++;}
00016     else{
00017         printf("\tfail\t%s", name);
00018         printf("\t%d != ", val_1);
00019         printf("%d\n", val_2);
00020     }
00021     total_tests++;
00022 }
```

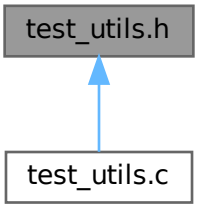
14.55 test_utils.h File Reference

```
#include <stdio.h>
#include <stdbool.h>
```

Include dependency graph for test_utils.h:



This graph shows which files directly or indirectly include this file:



Functions

- void TEST (const char *name, const bool condition)

  Affiche un resultat de test avec format uniforme.
- void TEST_EQ_INT (const char *name, const int val_1, const int val_2)

  Affiche un test d'egalite entre deux entiers.

Variables

- int total_tests
- int passed_tests

14.55.1 Function Documentation

14.55.1.1 TEST()

```
void TEST (
            const char * name,
            const bool condition )
```

Affiche un resultat de test avec format uniforme.

Parameters

name	Le nom du test (peut aussi servir de description courte).
condition	Resultat du test (true si reussi, false sinon).

Definition at line 26 of file main.c.

14.55.1.2 TEST_EQ_INT()

```
void TEST_EQ_INT (
            const char * name,
            const int val_1,
            const int val_2 )
```

Affiche un test d'egalite entre deux entiers.

Parameters

name	Meme usage que TEST.
val↩ _1	Premiere valeur a comparer.

Parameters

val↩ _2	Deuxieme valeur a comparer.

Definition at line 46 of file main.c.
Here is the caller graph for this function:

14.55.2 Variable Documentation

14.55.2.1 passed_tests

`int passed_tests [extern]`
le nombre total de test réussi
Definition at line 17 of file main.c.

14.55.2.2 total_tests

`int total_tests [extern]`
le nombre total de test effectué
Definition at line 18 of file main.c.

14.56 test_utils.h

Go to the documentation of this file.

```
#ifndef TEST_UTILS_H
#define TEST_UTILS_H

#include <stdio.h>
#include <stdbool.h>
extern int total_tests;
extern int passed_tests;

//#define TEST(name, condition) \
//    do { \
//        total_tests++; \
//        if (condition) { \
//            printf("\tsuccess\t%s\n", name); \
//            passed_tests++; \
//        } else { \
//            printf("\tfail\t%s\n", name); \
//        } \
//    } while(0)
//
void TEST(const char* name, const bool condition);
void TEST_EQ_INT(const char* name, const int val_1, const int val_2);

#endif
```

14.57 version.h File Reference

Macros

- #define RC_FILEVERSION 9,0,24,114
- #define RC_FILEVERSION_STRING "9, 0, 24, 114\0"

14.57.1 Macro Definition Documentation

14.57.1.1 RC_FILEVERSION

```
#define RC_FILEVERSION 9,0,24,114
```

Definition at line 22 of file version.h.

14.57.1.2 RC_FILEVERSION_STRING

```
#define RC_FILEVERSION_STRING "9, 0, 24, 114\0"
```

Definition at line 23 of file version.h.

14.58 version.h

Go to the documentation of this file.

```
00001 #ifndef VERSION_H
00002 #define VERSION_H
00003
00004         //Date Version Types
00005         static const char DATE[] = "03";
00006         static const char MONTH[] = "03";
00007         static const char YEAR[] = "2026";
00008         static const char UBUNTU_VERSION_STYLE[] =  "26.03";
00009
00010         //Software Status
00011         static const char STATUS[] =  "Alpha";
00012         static const char STATUS_SHORT[] =  "a";
00013
00014         //Standard Version Type
00015         static const long MAJOR  = 9;
00016         static const long MINOR  = 0;
00017         static const long BUILD  = 24;
00018         static const long REVISION  = 114;
00019
00020         //Miscellaneous Version Types
00021         static const long BUILDS_COUNT  = 135;
00022         #define RC_FILEVERSION 9,0,24,114
00023         #define RC_FILEVERSION_STRING "9, 0, 24, 114\0"
00024         static const char FULLVERSION_STRING [] = "9.0.24.114";
00025
00026         //These values are to keep track of your versioning state, don't modify them.
00027         static const long BUILD_HISTORY  = 24;
00028
00029 #endif //VERSION_H
```